

ĐẠI HỌC TRÀ VINH
TRƯỜNG KỸ THUẬT VÀ CÔNG NGHỆ



ĐỒ ÁN TỐT NGHIỆP
THIẾT KẾ VÀ TRIỂN KHAI HỆ THỐNG QUẢN LÝ API
VỚI CƠ CHẾ XÁC THỰC VÀ GIỚI HẠN TRUY CẬP
(API RATE LIMITING)

Vĩnh Long, tháng 6 năm 2026

ĐẠI HỌC TRÀ VINH
TRƯỜNG KỸ THUẬT VÀ CÔNG NGHỆ



ĐỒ ÁN TỐT NGHIỆP
THIẾT KẾ VÀ TRIỂN KHAI
HỆ THỐNG QUẢN LÝ API VỚI CƠ CHẾ
XÁC THỰC VÀ GIỚI HẠN TRUY CẬP
(API RATE LIMITING)

Giảng viên hướng dẫn: **GVC. ThS. NGUYỄN KHẮC QUỐC**

Sinh viên thực hiện: **PHẠM TRUNG HIẾU**

Mã số sinh viên: **110122076**

Lớp: **DA22TTD**

Khoá: **2022**

Vĩnh Long, tháng 06 năm 2026

ĐẠI HỌC TRÀ VINH
TRƯỜNG KỸ THUẬT VÀ CÔNG NGHỆ



ĐỒ ÁN TỐT NGHIỆP
THIẾT KẾ VÀ TRIỂN KHAI
HỆ THỐNG QUẢN LÝ API VỚI CƠ CHẾ
XÁC THỰC VÀ GIỚI HẠN TRUY CẬP
(API RATE LIMITING)

Giảng viên hướng dẫn: **GVC. ThS. NGUYỄN KHẮC QUỐC**

Sinh viên thực hiện: **PHẠM TRUNG HIẾU**

Mã số sinh viên: **110122076**

Lớp: **DA22TTD**

Khoá: **2022**

Vĩnh Long, tháng 06 năm 2026

MỤC LỤC

LỜI MỞ ĐẦU	viii
CHƯƠNG 1. TỔNG QUAN	1
1.1. Đặt vấn đề	1
1.2. Mục đích nghiên cứu.....	1
1.3. Phạm vi nghiên cứu và chức năng của hệ thống	2
1.3.1 Phạm vi nghiên cứu.....	2
1.3.2 Chức năng của hệ thống.....	3
1.4. Ý nghĩa thực tiễn	3
CHƯƠNG 2. NGHIÊN CỨU LÝ THUYẾT.....	5
2.1. Tìm hiểu về Microservices.....	5
2.1.1 Microservices	5
2.1.2 Kiến trúc một khối	5
2.1.3 Kiến trúc dịch vụ nhỏ.....	6
2.1.4 Ưu nhược điểm của Microservices	8
2.1.4.1 Ưu điểm	8
2.1.4.2 Nhược điểm	8
2.2. Tổng quan Spring Boot.....	9
2.2.1 Spring Boot	9
2.2.2 Lịch sử phát triển của Spring Boot	9
2.2.3 Sự khác biệt giữa Spring và Spring Boot.....	9
2.2.4 Những tính năng nổi bật của Spring Boot.....	10
2.3. Giới thiệu Spring Cloud	12
2.3.1 Spring Cloud	12
2.3.2 Mối quan hệ với Spring Boot.....	12

2.3.3 Spring Cloud hỗ trợ cho quá trình chuyển đổi sang microservice.....	13
2.3.4 Các vấn đề phát sinh mà Spring Cloud giải quyết.....	14
2.3.5 Spring Cloud giải quyết các vấn đề	15
2.3.6 Hướng dẫn thực hành.....	17
2.4. Tổng quan về RESTful API	20
2.4.1 RESTful API	20
2.4.2 Diễn giải các thành phần.....	20
2.4.3 RESTful hoạt động.....	21
2.4.4 Authentication và dữ liệu trả về	22
2.4.5 Status code	22
2.4.6 Xây dựng API với Laravel.....	23
2.5. Tìm hiểu về JSON Web Token	23
2.5.1 Json Web Token - jwt	23
2.5.2 Cấu trúc của JSON Web Token:	24
2.5.3 Signature	25
2.6. Tổng quan API Gateway	26
2.6.1 API Gateway	26
2.6.2 Lợi ích của việc sử dụng API Gateway	27
2.7. Tổng quan Redis.....	28
2.7.1 Giới thiệu.....	28
2.7.2 Lợi ích của Redis	29
2.8. Tìm hiểu về hệ quản trị cơ sở dữ liệu PostgreSQL.....	30
2.8.1 PostgreSQL	30
2.8.2 Tính năng PostgreSQL.....	31
2.9. Tìm hiểu và tổng quan về Docker.....	33

2.9.1 Docker	33
2.9.2 Docker giải quyết vấn đề	33
2.9.3 Container	34
2.9.4 Docker và Virtual Machine.....	34
2.9.5 Các thành phần quan trọng của Docker	34
2.9.6 Docker Compose	35
2.10. Tìm hiểu Apache Jmeter	36
2.10.1 Apache Jmeter	36
2.10.2 Apache JMeter có thể làm.....	36
2.10.3 Đặc trưng của Apache JMeter.....	37
2.11. Tổng quan Swagger	38
2.11.1 Swagger.....	38
2.11.2 Lợi ích của Swagger với nhà phát triển API.....	41
CHƯƠNG 3. HIỆN THỰC HÓA NGHIÊN CỨU	42
3.1. Mô tả bài toán.....	42
3.1.1 Bối cảnh thực tế	42
3.1.2 Các vấn đề cần giải quyết	42
3.1.3 Mục tiêu của hệ thống.....	43
3.1.4 Yêu cầu chức năng.....	43
3.1.5 Yêu cầu phi chức năng.....	44
3.2. Kiến Trúc Hệ Thống.....	45
3.2.1 Mô hình kiến trúc Microservice.....	45
3.2.2 Các thành phần trong hệ thống	46
3.2.2.1 Eureka Server.....	46
3.2.2.2 API Gateway	46

3.2.2.3 IAM Service	47
3.2.2.4 HR Service.....	48
3.2.2.5 Redis	49
3.2.2.6 PostgreSQL	49
3.2.3 Luồng xử lý của hệ thống.....	50
3.2.3.1 Luồng đăng ký tài khoản	50
3.2.3.2 Luồng đăng nhập	51
3.2.3.3 Luồng xác thực JWT	52
3.2.3.4 Luồng Rate Limiting.....	53
3.2.3.5 Luồng quản lý sinh viên	54
3.2.4 Kiến trúc bảo mật.....	54
3.2.4.1 JWT RS256	55
3.2.4.2 Bcrypt	55
3.2.4.3 HMAC Signature	56
3.2.4.4 Rate Limiting	56
3.2.4.5 Brute-force Protection	57
3.2.4.6 Account Locking.....	57
3.3. Phân tích tác nhân và Use Case	57
3.3.1 Tác nhân của hệ thống	57
3.3.2 Danh sách Use Case	59
3.3.3 Sơ đồ Use Case tổng quát	61
3.4. Thiết kế chức năng hệ thống	61
3.4.1 Chức năng quản lý tài khoản.....	61
3.4.2 Chức năng phân quyền.....	63
3.4.3 Chức năng quản lý sinh viên.....	63

3.5. Cơ sở dữ liệu	64
3.5.1 Phân tích dữ liệu.....	64
3.5.2 Mô hình ERD	65
3.5.3 Danh sách bảng dữ liệu	66
3.5.4 Thiết kế bảng nguoi_dung.....	66
3.5.5 Thiết kế bảng sinh_vien	67
3.5.6 Thiết kế bảng giang_vien.....	67
3.5.7 Thiết kế bảng lop_hoc.....	68
3.5.8 Thiết kế bảng dang_ky_lop_hoc	68
3.5.9 Thiết kế bảng nhat_ky_bao_mat.....	69
3.6. Thiết kế API.....	69
3.6.1 API Authentication	69
3.6.2 Student API	70
3.6.3 Teacher API	72
3.6.4 Class API.....	73
3.6.5 Security API.....	74
CHƯƠNG 4. KẾT QUẢ NGHIÊN CỨU	76
4.1. Môi trường triển khai	76
4.1.1 Yêu cầu phần cứng.....	76
4.1.2 Yêu cầu phần mềm.....	76
4.1.3 Công nghệ sử dụng.....	77
4.2. Cài đặt và cấu hình hệ thống.....	79
4.2.1 Clone project từ repository	79
4.2.2 Cấu hình Environment Variables	79
4.2.3 Build và khởi động services.....	79

4.2.4 Verify deployment	79
4.3. Demo chức năng hệ thống	80
4.3.1 Demo Authentication - Đăng ký và Đăng nhập.....	80
4.3.2 Demo Student Management - Quản lý sinh viên.....	82
4.3.3 Demo Class Enrollment - Quản lý đăng ký lớp.....	84
4.4. Demo giao diện Website quản lý sinh viên	88
4.4.1 Giao diện đăng nhập/ đăng xuất.....	88
4.4.2 Giao diện Admin.....	89
4.4.3 Giao diện Giảng viên	89
4.4.4 Giao diện Sinh viên.....	90
4.5. Kiểm thử bảo mật.....	90
4.5.1 Test Progressive Account Locking	90
4.5.2 Test Rate Limiting	92
4.5.3 Test JWT Authentication	93
CHƯƠNG 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN.....	94
5.1. Kết luận	94
5.2. Hướng phát triển.....	94
TÀI LIỆU THAM KHẢO	96

LỜI MỞ ĐẦU

Hiện nay, cùng với sự phát triển mạnh mẽ của công nghệ thông tin và Internet, các hệ thống quản lý trên nền tảng web ngày càng được sử dụng rộng rãi trong nhiều lĩnh vực như giáo dục, doanh nghiệp và dịch vụ trực tuyến. Đặc biệt, đối với các hệ thống có nhiều người dùng truy cập cùng lúc thì vấn đề bảo mật, quản lý truy cập và đảm bảo hệ thống hoạt động ổn định là điều rất quan trọng.

Trong quá trình tìm hiểu và học tập về lập trình backend, tôi nhận thấy rằng nhiều hệ thống hiện nay không chỉ đơn giản là xây dựng các chức năng quản lý dữ liệu mà còn phải đảm bảo được khả năng bảo mật, phân quyền người dùng, giới hạn truy cập và chống các hành vi spam hoặc tấn công vào hệ thống. Bên cạnh đó, kiến trúc Microservice cũng đang được sử dụng phổ biến trong các hệ thống doanh nghiệp vì có khả năng mở rộng tốt, dễ quản lý và dễ triển khai hơn so với mô hình Monolithic truyền thống.

Xuất phát từ những lý do trên, tôi quyết định thực hiện đề tài: “Thiết kế và triển khai hệ thống quản lý API với cơ chế xác thực và giới hạn truy cập (API Rate Limiting)”. Đề tài tập trung xây dựng một hệ thống backend theo mô hình Microservice sử dụng Spring Boot kết hợp với các công nghệ như Redis, PostgreSQL và Docker nhằm mô phỏng một hệ thống doanh nghiệp thực tế.

Hệ thống được xây dựng với các chức năng chính như xác thực người dùng bằng JWT, quản lý phân quyền theo vai trò, giới hạn số lượng request bằng Redis Rate Limiting, khóa tài khoản khi đăng nhập sai nhiều lần, chặn IP có dấu hiệu spam request và ghi log hoạt động của hệ thống. Ngoài ra, đề tài còn xây dựng mô hình quản lý sinh viên với các vai trò như Admin, Giáo viên và Sinh viên để mô phỏng việc quản lý người dùng trong thực tế.

Thông qua đề tài này, tôi mong muốn củng cố thêm kiến thức về lập trình backend, kiến trúc Microservice, bảo mật hệ thống và quy trình xây dựng API theo hướng doanh nghiệp. Đồng thời, đây cũng là cơ hội để em tiếp cận và tìm hiểu sâu hơn về các công nghệ hiện đại đang được sử dụng phổ biến trong thực tế hiện nay.

LỜI CẢM ƠN

Trong suốt quá trình thực hiện đồ án tốt nghiệp, tôi đã nhận được rất nhiều sự quan tâm, hướng dẫn và hỗ trợ quý báu từ quý thầy cô, gia đình và bạn bè. Đây là nguồn động lực lớn giúp tôi hoàn thành tốt đề tài của mình.

Trước hết, tôi xin gửi lời cảm ơn chân thành và sâu sắc đến thầy Nguyễn Khắc Quốc, giảng viên hướng dẫn của tôi. Thầy đã tận tình hướng dẫn, góp ý, định hướng và hỗ trợ tôi trong suốt quá trình thực hiện đề tài. Những kiến thức chuyên môn, kinh nghiệm thực tế và những lời nhận xét quý báu của thầy không chỉ giúp tôi hoàn thiện đồ án mà còn giúp tôi nâng cao tư duy, phương pháp nghiên cứu và kỹ năng trong lĩnh vực Công nghệ thông tin.

Tôi xin gửi lời tri ân đến quý thầy cô thuộc Bộ môn Khoa Công nghệ Thông tin đã luôn tận tâm giảng dạy, truyền đạt những kiến thức nền tảng cũng như chuyên môn trong suốt quá trình học tập. Những kiến thức và sự chỉ bảo của quý thầy cô là hành trang quan trọng giúp tôi có thể áp dụng vào quá trình thực hiện đồ án và định hướng phát triển trong tương lai.

Cuối cùng, tôi xin cảm ơn gia đình và bạn bè đã luôn động viên, hỗ trợ và tạo điều kiện để tôi có thể hoàn thành tốt nhiệm vụ học tập của mình.

Mặc dù đã cố gắng hoàn thiện đồ án với tinh thần nghiêm túc và trách nhiệm, nhưng do kiến thức và kinh nghiệm thực tế còn hạn chế, bài báo cáo khó tránh khỏi những thiếu sót. Tôi rất mong nhận được sự góp ý từ quý thầy cô để đề tài được hoàn thiện hơn.

Tôi xin chân thành cảm ơn!

Sinh viên thực hiện

Phạm Trung Hiếu

This image shows a full page of white paper with horizontal ruling lines. The lines are evenly spaced and extend across the width of the page, typical of notebook or legal stationery. There are no margins, text, or other markings on the page.

Giáo viên hướng dẫn

BẢN NHẬN XÉT ĐỒ ÁN, KHÓA LUẬN TỐT NGHIỆP

(Của giảng viên hướng dẫn)

Họ và tên sinh viên: Phạm Trung Hiếu

MSSV: 110122076

Ngành: Công Nghệ Thông Tin

Khóa: 2022

Tên đề tài: Thiết kế và triển khai hệ thống quản lý api với cơ chế xác thực và giới hạn truy cập (api rate limiting).

Họ và tên Giáo viên hướng dẫn: Nguyễn Khắc Quốc

Chức danh: Học vị:

NHẬN XÉT

1. Nội dung đề tài:

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

2. Ưu điểm:

.....

.....

.....

3. Nhược điểm:

.....

.....

.....

.....

4. Điểm mới đề tài:

.....

.....

.....

.....

.....

5. Giá trị thực trên đề tài:

.....

.....

.....

.....

.....

.....

.....

7. Đề nghị sửa chữa bổ sung:

.....

.....

.....

.....

.....

.....

.....

8. Đánh giá:

.....

.....

.....

.....

.....

.....

Vĩnh Long, ngày 29 tháng 06 năm 2026
Giảng viên hướng dẫn

(Của giảng viên chấm trong đề án, khoá luận của sinh viên)

Giảng viên chấm

BẢN NHẬN XÉT ĐỒ ÁN, KHÓA LUẬN TỐT NGHIỆP
(Của cán bộ chấm đồ án, khóa luận)

Họ và tên người nhận xét:

Chức danh: Học vị:

Chuyên ngành:

Cơ quan công tác:

Tên sinh viên:

Tên đề tài đồ án, khóa luận tốt nghiệp:

.....

.....

I. Ý KIẾN NHẬN XÉT

1. Nội dung:

.....

.....

.....

.....

.....

.....

.....

.....

2. Điểm mới các kết quả của đồ án, khóa luận:

.....

.....

.....

3. Ứng dụng thực tế:

.....

.....

.....

.....

.....

II. CÁC VẤN ĐỀ CẦN LÀM RÕ

(Các câu hỏi của giáo viên phản biện)

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

III. KẾT LUẬN

(Ghi rõ đồng ý hay không đồng ý cho bảo vệ đồ án khóa luận tốt nghiệp)

.....

.....

.....

.....

.....

Vĩnh Long, ngày 29 tháng 06 năm 2026

Người nhận xét

DANH MỤC HÌNH ẢNH

HÌNH 2.1: Kiến trúc Microservices	5
HÌNH 2.2: Kiến trúc dịch vụ.....	7
HÌNH 2.3: Sự khác nhau giữa Microservices với SOA.....	14
HÌNH 2.4: RESTful API.....	20
HÌNH 2.5: RESTful hoạt động.....	21
HÌNH 2.6: JSON Web Token	24
HÌNH 2.7: Cấu trúc của JSON Web Token	25
HÌNH 2.8: API Gateway	26
HÌNH 2.9: Redis.....	28
HÌNH 3.1: Kiến trúc hệ thống quản lý sinh viên	45
HÌNH 3.2: Luồng đăng ký tài khoản.....	50
HÌNH 3.3: Luồng đăng nhập.....	51
HÌNH 3.4: Luồng xác thực JWT	52
HÌNH 3.5: Luồng Rate Limiting.....	53
HÌNH 3.6: Luồng quản lý sinh viên.....	54
HÌNH 3.7: Mức độ khóa tăng dần theo số lần đăng nhập sai	57
HÌNH 3.8: Use Case Diagram - Hệ thống quản lý sinh viên	61
HÌNH 3.9: Entity Relationship Diagram.....	65
HÌNH 3.10: Danh sách bảng dữ liệu.....	66
HÌNH 3.11: Bảng thiết kế bảng nguoi_dung	66
HÌNH 3.12: Bảng thiết kế bảng sinh_vien.....	67
HÌNH 3.13: Bảng thiết kế bảng giang_vien.....	67
HÌNH 3.14: Bảng thiết kế bảng lop_hoc.....	68
HÌNH 3.15: Bảng thiết kế dang_ky_lop_hoc.....	68

HÌNH 3.16: Bảng thiết kế nhật_ky_bao_mat.....	69
HÌNH 4.1: Postman - Đăng ký user mới thành công với status 201	81
HÌNH 4.2: Postman - Đăng nhập thành công với JWT token	82
HÌNH 4.3: Postman - POST Student API với JWT Authorization.....	83
HÌNH 4.4: Postman - GET Students với pagination params	84
HÌNH 4.5: Postman - POST Enroll Student API	86
HÌNH 4.6: Postman - GET Students in Class	87
HÌNH 4.7: Giao diện đăng nhập	88
HÌNH 4.8: Giao diện đăng ký	88
HÌNH 4.9: Giao diện Admin.....	89
HÌNH 4.10: Giao diện Giảng viên	89
HÌNH 4.11: Giao diện Sinh viên.....	90

DANH MỤC BẢNG

BẢNG 2.1: Cách Spring Cloud giải quyết các vấn đề.....	15
BẢNG 2.2: Docker và Virtual Machine	34
BẢNG 3.1: Tác nhân của hệ thống	57
BẢNG 3.2: Use Case Admin	59
BẢNG 3.3: Use Case Teacher	60
BẢNG 3.4: Use Case Student	60
BẢNG 3.5: Phân quyền.....	63
BẢNG 3.6: Student API.....	71
BẢNG 3.7: Teacher API.....	72
BẢNG 3.8: Class API	73
BẢNG 3.9: Security API.....	75
BẢNG 4.1: Phần cứng	76
BẢNG 4.2: Phần mềm	76
BẢNG 4.3: Bảng các server đã chạy	80
BẢNG 4.1: Test Progressive Account Locking.....	90

CỤM TỪ VIẾT TẮT

STT	Viết tắt	Tên đầy đủ	Dịch theo chuyên ngành
1	AES	Advanced Encryption Standard	Chuẩn mã hóa tiên tiến
2	API	Application Programming Interface	Giao diện lập trình ứng dụng
3	CPU	Central Processing Unit	Bộ xử lý trung tâm
4	CRUD	Create, Read, Update, Delete	Tạo, đọc, cập nhật, xóa dữ liệu
5	DB	Database	Cơ sở dữ liệu
6	DTO	Data Transfer Object	Đối tượng truyền dữ liệu
7	HTTP	HyperText Transfer Protocol	Giao thức truyền tải siêu văn bản
8	IAM	Identity and Access Management	Quản lý định danh và quyền truy cập
9	IP	Internet Protocol	Giao thức Internet
10	JSON	JavaScript Object Notation	Định dạng dữ liệu JSON
11	JWT	JSON Web Token	Chuỗi token xác thực dạng JSON
12	LTS	Long Term Support	Phiên bản hỗ trợ dài hạn
13	MVC	Model - View - Controller	Mô HÌNH Model - View - Controller
14	ORM	Object Relational Mapping	Công nghệ ánh xạ đối tượng - quan hệ
15	REST	Representational State Transfer	Kiến trúc chuyển trạng thái đại diện
16	RESTful API	Representational State Transfer API	API tuân theo kiến trúc REST
17	SQL	Structured Query Language	Ngôn ngữ truy vấn có cấu trúc
18	SSL	Secure Sockets Layer	Lớp công bảo mật
19	TCP	Transmission Control Protocol	Giao thức điều khiển truyền tải

20	URL	Uniform Resource Locator	Bộ định vị tài nguyên thống nhất
21	XML	Extensible Markup Language	Ngôn ngữ đánh dấu mở rộng
22	YAML	YAML Ain't Markup Language	Định dạng cấu HÌNH dữ liệu

Các thuật ngữ riêng trong đồ án:

STT	Viết tắt	Tên đầy đủ	Dịch CNTT chuẩn
23	ACID	Atomicity, Consistency, Isolation, Durability	Tính nguyên tử, nhất quán, cô lập, bền vững của giao dịch
24	BCrypt	BCrypt Password Hashing Algorithm	Thuật toán băm mật khẩu BCrypt
25	CI/CD	Continuous Integration / Continuous Deployment	Tích hợp liên tục / Triển khai liên tục
26	CORS	Cross-Origin Resource Sharing	Cơ chế chia sẻ tài nguyên khác nguồn
27	DNS	Domain Name System	Hệ thống phân giải tên miền
28	IDE	Integrated Development Environment	Môi trường phát triển tích hợp
29	JDK	Java Development Kit	Bộ công cụ phát triển Java
30	JVM	Java Virtual Machine	Máy ảo Java
31	JPA	Java Persistence API	API duy trì dữ liệu Java
32	MFA	Multi-Factor Authentication	Xác thực đa yếu tố
33	OAuth	Open Authorization	Chuẩn cấp quyền truy cập mở
34	RBAC	Role-Based Access Control	Kiểm soát truy cập dựa trên vai trò
35	Redis	Remote Dictionary Server	Hệ thống lưu trữ dữ liệu dạng khóa - giá trị
36	RDBMS	Relational Database Management System	Hệ quản trị cơ sở dữ liệu quan hệ
37	SDK	Software Development Kit	Bộ công cụ phát triển phần mềm
38	UI	User Interface	Giao diện người dùng

39	UUID	Universally Unique Identifier	Mã định danh duy nhất toàn cầu
40	VM	Virtual Machine	Máy ảo

CHƯƠNG 1. TỔNG QUAN

1.1. Đặt vấn đề

Hiện nay, cùng với sự phát triển mạnh mẽ của công nghệ thông tin và Internet, các hệ thống quản lý trên nền tảng web ngày càng được sử dụng rộng rãi trong nhiều lĩnh vực như giáo dục, doanh nghiệp và các dịch vụ trực tuyến. Đối với các hệ thống có số lượng người dùng lớn, việc đảm bảo tính bảo mật, ổn định và khả năng quản lý truy cập là một yêu cầu rất quan trọng.

Trong thực tế, nhiều hệ thống hiện nay vẫn còn tồn tại các vấn đề như spam request, tấn công brute-force, truy cập trái phép hoặc quản lý quyền người dùng chưa hiệu quả. Ngoài ra, khi số lượng người dùng tăng cao thì mô hình Monolithic truyền thống cũng gặp nhiều hạn chế về khả năng mở rộng, bảo trì và triển khai hệ thống.

Bên cạnh đó, kiến trúc Microservice hiện đang được nhiều doanh nghiệp sử dụng vì có khả năng tách biệt các service độc lập, dễ mở rộng và dễ quản lý hơn. Đồng thời, việc kết hợp các công nghệ như API Gateway, Redis, JWT và Docker giúp nâng cao khả năng bảo mật cũng như hiệu suất hoạt động của hệ thống.

Xuất phát từ những lý do trên, tôi quyết định thực hiện đề tài “Thiết kế và triển khai hệ thống quản lý API với cơ chế xác thực và giới hạn truy cập (API Rate Limiting)” nhằm xây dựng một hệ thống backend theo mô hình Microservice có áp dụng các cơ chế bảo mật và quản lý truy cập theo hướng doanh nghiệp thực tế.

1.2. Mục đích nghiên cứu

Đề tài được thực hiện nhằm các mục đích sau:

- Tìm hiểu kiến trúc Microservice trong phát triển hệ thống backend.
- Tìm hiểu cơ chế xác thực và phân quyền người dùng bằng JWT.
- Xây dựng API Gateway để quản lý request từ client.
- Áp dụng Redis Rate Limiting để giới hạn số lượng request và chống spam API.
- Xây dựng cơ chế khóa tài khoản và chặn IP khi đăng nhập sai nhiều lần.
- Xây dựng hệ thống quản lý sinh viên với các vai trò Admin, Giáo viên và Sinh viên.
- Nâng cao kiến thức về Spring Boot, Spring Security và các công nghệ backend hiện đại.

- Mô phỏng quy trình xây dựng hệ thống backend theo hướng doanh nghiệp thực tế.

1.3. Phạm vi nghiên cứu và chức năng của hệ thống

1.3.1 Phạm vi nghiên cứu

Đề tài tập trung nghiên cứu và xây dựng hệ thống quản lý sinh viên theo kiến trúc Microservice, hướng đến việc phát triển một hệ thống backend có tính bảo mật, khả năng mở rộng và dễ dàng bảo trì. Hệ thống được xây dựng bằng Spring Boot 3.4.1 trên nền tảng Java 21, sử dụng PostgreSQL để quản lý dữ liệu và Redis nhằm hỗ trợ tăng hiệu năng cũng như triển khai các cơ chế bảo mật.

Hệ thống được thiết kế theo mô hình Microservice gồm các thành phần chính như Eureka Server, API Gateway, IAM Service và HR Service. Trong đó, Eureka Server có nhiệm vụ quản lý các dịch vụ trong hệ thống, giúp các Service có thể tìm kiếm và kết nối với nhau. API Gateway đóng vai trò là cổng trung gian tiếp nhận mọi yêu cầu từ phía người dùng, thực hiện kiểm tra bảo mật, xác thực JWT, giới hạn truy cập API và điều hướng request đến đúng Service xử lý.

IAM Service chịu trách nhiệm quản lý tài khoản, xác thực và phân quyền người dùng. HR Service xử lý các nghiệp vụ chính liên quan đến quản lý sinh viên, giáo viên, lớp học, điểm học tập và thông tin cố vấn học tập.

Đề tài tập trung nghiên cứu các vấn đề về Authentication, Authorization, API Security, Rate Limiting, Logging và Monitoring. Hệ thống áp dụng JWT để xác thực người dùng, BCrypt để bảo vệ mật khẩu, AES-GCM để mã hóa dữ liệu nhạy cảm và Redis để kiểm soát số lượng request nhằm hạn chế tình trạng spam API.

Bên cạnh đó, hệ thống còn triển khai cơ chế bảo vệ tài khoản thông qua việc giới hạn số lần đăng nhập sai, tự động khóa tài khoản khi phát hiện hành vi bất thường và ghi nhận log để phục vụ việc kiểm tra, giám sát hệ thống.

Trong phạm vi đề tài, hệ thống tập trung phát triển trên nền tảng Web, chưa nghiên cứu ứng dụng Mobile, trí tuệ nhân tạo (AI), Machine Learning hoặc triển khai trên môi trường Cloud thực tế.

1.3.2 Chức năng của hệ thống

Hệ thống được xây dựng với ba nhóm người dùng chính gồm Admin, Teacher và Student, mỗi nhóm có quyền hạn khác nhau nhằm đảm bảo tính bảo mật và phù hợp với nhu cầu quản lý.

Đối với người dùng, hệ thống hỗ trợ các chức năng cơ bản như đăng ký tài khoản, đăng nhập, đổi mật khẩu và cấp lại JWT Token. Sau khi đăng nhập thành công, hệ thống thực hiện xác thực thông qua JWT và kiểm tra quyền truy cập dựa trên vai trò của người dùng.

Về bảo mật, hệ thống triển khai các cơ chế như xác thực JWT, mã hóa mật khẩu bằng BCrypt, mã hóa dữ liệu quan trọng bằng AES-GCM và giới hạn số lượng request thông qua Redis Rate Limiting. Ngoài ra, hệ thống có khả năng theo dõi số lần đăng nhập sai, khóa tài khoản tạm thời và chặn IP trong trường hợp phát hiện hành vi truy cập bất thường.

Đối với chức năng quản lý sinh viên, hệ thống hỗ trợ quản lý các thông tin như mã sinh viên, họ tên, thông tin cá nhân, lớp học, ngành học, kết quả học tập và điểm rèn luyện. Giáo viên có thể quản lý danh sách sinh viên thuộc lớp mình phụ trách, cập nhật điểm và theo dõi thông tin học tập.

Admin có quyền quản lý toàn bộ hệ thống bao gồm quản lý tài khoản, phân quyền người dùng, quản lý sinh viên, giáo viên, lớp học, theo dõi log hoạt động và xử lý các vấn đề bảo mật như mở khóa tài khoản hoặc gỡ chặn IP.

Thông qua các chức năng trên, hệ thống không chỉ giải quyết bài toán quản lý sinh viên mà còn mô phỏng một hệ thống backend doanh nghiệp có khả năng bảo mật và mở rộng trong thực tế.

1.4. Ý nghĩa thực tiễn

Đề tài giúp người thực hiện củng cố kiến thức về lập trình backend, bảo mật hệ thống và kiến trúc Microservice. Đồng thời, đề tài còn giúp tiếp cận các công nghệ hiện đại đang được sử dụng phổ biến trong doanh nghiệp như Spring Boot, Redis, JWT, Docker và API Gateway.

Ngoài ra, hệ thống còn mô phỏng được các cơ chế bảo mật thực tế như giới hạn request, chống spam API, khóa tài khoản khi đăng nhập sai nhiều lần và quản lý

phân quyền người dùng. Đây là những tính năng quan trọng thường được áp dụng trong các hệ thống doanh nghiệp hiện nay.

Thông qua việc xây dựng đề tài, người thực hiện có cơ hội rèn luyện kỹ năng phân tích hệ thống, thiết kế API, xây dựng backend và triển khai các giải pháp bảo mật cho hệ thống web theo hướng thực tế.

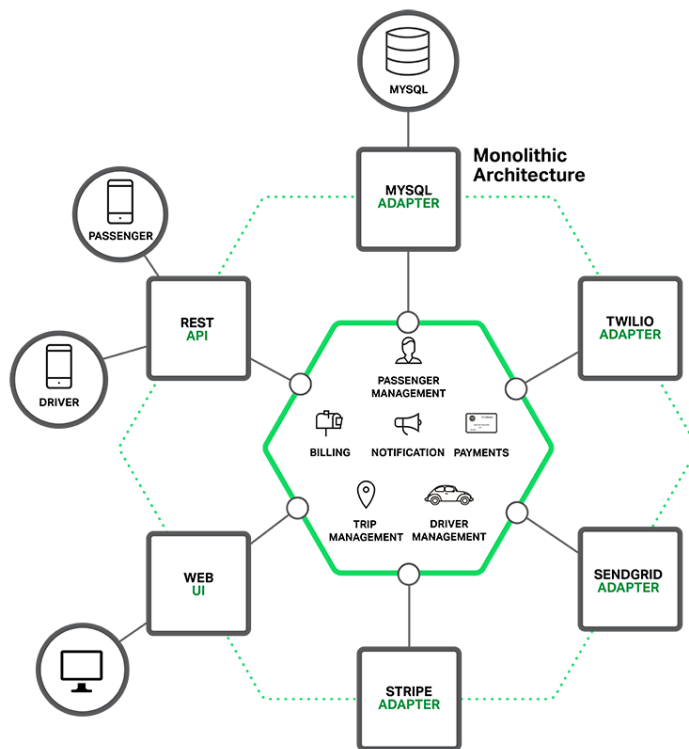
CHƯƠNG 2. NGHIÊN CỨU LÝ THUYẾT

2.1. Tìm hiểu về Microservices

2.1.1 Microservices

Thực tế có nhiều định nghĩa khác nhau đối với Microservices nhưng hiểu theo cách đơn giản thì, microservice là một kiểu kiến trúc phần mềm. Các module trong phần mềm này được chia thành các service rất nhỏ (microservice). Mỗi service sẽ được đặt trên một server riêng để dàng để nâng cấp và scale ứng dụng.[9]

2.1.2 Kiến trúc một khối



HÌNH 2.1: Kiến trúc Microservices

Một dịch vụ gọi taxi qua di động, với business logic được thể hiện bởi các khối dịch vụ, đối tượng cho từng vùng nghiệp vụ (domain objects) và các sự kiện (events: khách đặt xe, khách hủy xe, xe nhận khách...) Xung quanh lõi là bộ chuyển đổi (adapter) ví dụ như kết nối vào cơ sở dữ liệu, gửi nhận thông điệp (messaging), web service hoặc giao diện web frontend.

Trong nhiều trường hợp, người ta có thể xây dựng các services độc lập nhưng chúng lại được triển khai chung. Mặc dù có cấu trúc module hóa hợp lý, nhưng ứng dụng kiểu này sẽ đóng gói và cài đặt thành một khối (monolithic). Mã chạy cụ thể tùy thuộc vào ngôn ngữ lập trình hay thư viện framework.

Tương tự như ứng dụng monolithic, những services này to và phức tạp lên theo thời gian vì thường xuyên thêm các tính năng. Và thế là những ứng dụng này lại trở thành một mớ các services monolithic, cũng không còn khác mấy so với kiến trúc một khối thông thường. Hình trên thể hiện một ứng dụng gồm nhiều services. Những services này được triển khai cùng một lúc vào 1 ứng dụng lớn. Dù bên trong có gồm các services thì đây là một ứng dụng monolithic. Một số tính chất của kiến trúc một khối:

- Được thiết kế, phát triển và triển khai theo một khối duy nhất.
- Ứng dụng monolithic phức tạp và to gây khó khăn cho việc bảo trì, nâng cấp và thêm tính năng mới.
- Khó áp dụng phát triển kiểu agile.
- Phải triển khai lại toàn bộ hệ thống dù chỉ cập nhật hay nâng cấp một phần.
- Mở rộng: phải mở rộng cả khối ứng dụng, gặp khó khăn nếu có các yêu cầu về tài nguyên khác nhau (ví dụ một service yêu cầu thêm CPU, service khác lại yêu cầu nhiều memory).
- Độ tin cậy: một service không ổn định có thể sập cả hệ thống.
- Khó đổi mới: ứng dụng monolithic phải sử dụng chung công nghệ nên khó thay đổi hay áp dụng công nghệ mới.
- Những tính chất giới hạn trên của kiến trúc Monolithic dẫn tới sự phát triển của kiến trúc Microservices.

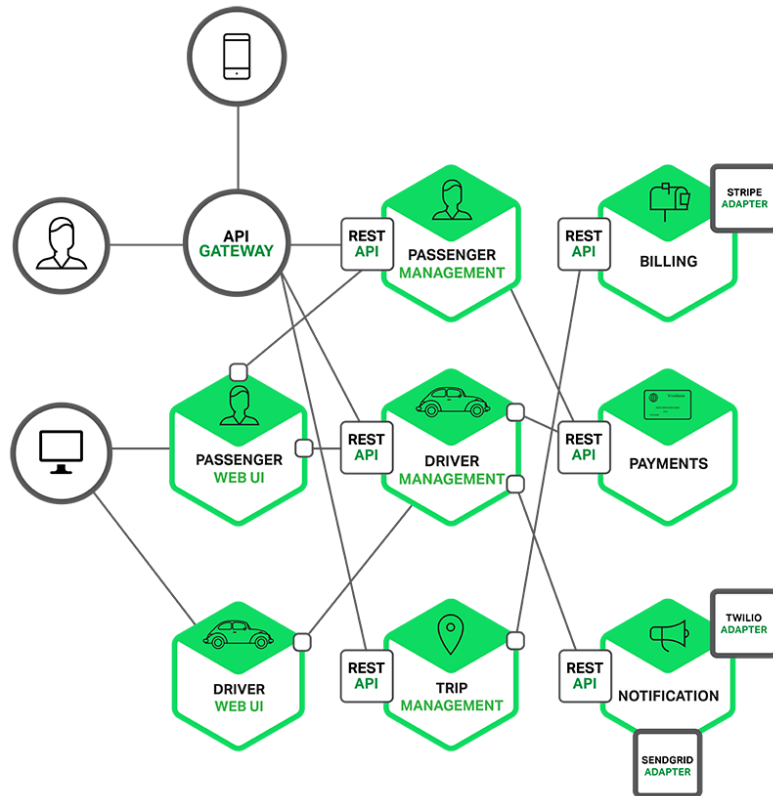
2.1.3 Kiến trúc dịch vụ nhỏ

Nền tảng của kiến trúc microservices là xây dựng một ứng dụng mà ứng dụng này là tổng hợp của nhiều services nhỏ và độc lập có thể chạy riêng biệt, phát triển và triển khai độc lập.

Ta có thể giải quyết các vấn đề của ứng dụng một khối bằng kiến trúc microservices (nhiều dịch vụ nhỏ). Ý tưởng là chia nhỏ ứng dụng lớn ra thành các dịch vụ nhỏ kết nối với nhau.

Mỗi dịch vụ nhỏ thực hiện một tập các chức năng chuyên biệt như quản lý đơn hàng, quản lý khách hàng. Mỗi dịch vụ là một ứng dụng nhỏ có kiến trúc đa diện lõi là business logic kết nối ra các adapter khác nhau. Một số dịch vụ nhỏ lộ ra giao tiếp

lập trình API cho dịch vụ nhỏ khác hay ứng dụng client gọi tới. Khi vận hành, mỗi dịch vụ nhỏ được chạy trong một máy ảo hoặc Docker container.



HÌNH 2.2: Kiến trúc dịch vụ

Mỗi vùng chức năng giờ được thực thi bởi một dịch vụ nhỏ. Ứng dụng web cũng có thể chia nhỏ hơn chuyên cho từng đối tượng người dùng (hành khách/tài xế). Thiết kế giao diện cho từng đối tượng người dùng giúp tối ưu trải nghiệm tốt hơn, tốc độ nhanh hơn, dễ tương thích hơn trong khi chức năng tối giản hơn.

Một số khái niệm về microservices nói về quá trình chia tách ứng dụng monolithic thành nhóm các services độc lập. Tuy nhiên, theo quan điểm của tôi, microservices không chỉ về chia tách các services sẵn có trong monolithic.

Điều quan trọng chính là nhìn vào các tính năng trong một ứng dụng monolithic, ta có thể nhận biết, xác định các yêu cầu và khả năng cần thiết để đáp ứng một nghiệp vụ. Sau đó từng nghiệp vụ này sẽ được xây dựng thành những service nhỏ, độc lập. Những services này có thể sử dụng các nền tảng công nghệ khác nhau và phục vụ một mục đích cụ thể và có giới hạn. Ưu nhược điểm của Microservices

2.1.4 Ưu nhược điểm của Microservices

2.1.4.1 Ưu điểm

- Cho phép dễ dàng continuous delivery và deployment các ứng dụng lớn, phức tạp:
 - + Cải thiện khả năng bảo trì - mỗi service tương đối nhỏ do đó dễ hiểu và thay đổi hơn
 - + Khả năng testing dễ dàng hơn - các services nhỏ hơn và nhanh hơn để test
 - + Khả năng triển khai tốt hơn - các services có thể được triển khai độc lập
 - + Cho phép các services được phát triển bởi những team khác nhau. Mỗi team có thể phát triển, thử nghiệm, triển khai và mở rộng quy mô dịch vụ của mình một cách độc lập với tất cả các team khác.
- Giảm thiểu rủi ro: Nếu có lỗi trong một service thì chỉ có service đó bị ảnh hưởng. Các services khác sẽ tiếp tục xử lý các yêu cầu. Trong khi đó, một thành phần hoạt động sai của kiến trúc một khối có thể làm ảnh hưởng toàn bộ hệ thống.
- Dễ dàng thay đổi sử dụng các công nghệ mới: Khi triển khai các services bạn có thể lựa chọn nhiều công nghệ mới. Tương tự khi có thay đổi lớn đối với các services hiện có bạn có thể dễ dàng thay đổi công nghệ.

2.1.4.2 Nhược điểm

- Các nhà phát triển phải đối phó với sự phức tạp của việc tạo ra một hệ thống phân tán:
 - + Cần implement việc communication giữa các inter-services.
 - + Handle partial failure là rất phức tạp vì một luồng xử lý cần đi qua nhiều services.
 - + Việc thực hiện các requests trải rộng trên nhiều services khó khăn hơn, điều này cũng đòi hỏi sự phối hợp cẩn thận giữa các teams.
 - + Khó khăn trong việc đảm bảo toàn vẹn CSDL nếu triển khai theo kiến trúc cơ sở dữ liệu phân vùng.
- Triển khai và quản lý microservices nếu làm thủ công theo cách đã làm với ứng dụng một khối phức tạp hơn rất nhiều.
- Phải xử lý sự cố khi kết nối chậm, lỗi khi thông điệp không gửi được hoặc thông điệp gửi đến nhiều đích đến vào các thời điểm khác nhau.

2.2. Tổng quan Spring Boot

2.2.1 Spring Boot

Spring Boot được xây dựng dựa trên nền của Spring Framework, phát triển bởi ngôn ngữ Java. Nó là một phiên bản có tính tự động hoá cao hơn và đơn giản hơn của Spring. Spring Boot khiến cho việc tạo và khởi chạy ứng dụng chỉ diễn ra trong vòng vài phút.[1]

2.2.2 Lịch sử phát triển của Spring Boot

Spring Boot được phát triển bởi Pivotal Software, với phiên bản đầu tiên (Spring Boot 1.0) được phát hành vào tháng 4 năm 2014. Mục tiêu ban đầu của Spring Boot là giải quyết những hạn chế và phức tạp trong việc cấu hình Spring Framework truyền thống. Spring Framework, mặc dù rất mạnh mẽ và linh hoạt, nhưng yêu cầu nhiều cấu hình thủ công và có thể trở nên cồng kềnh khi xây dựng các ứng dụng phức tạp.

Trước khi Spring Boot ra đời, việc cấu hình một ứng dụng Spring thường đòi hỏi nhiều file XML hoặc các class Java cấu hình, gây khó khăn cho việc bảo trì và phát triển. Spring Boot được giới thiệu như một cách tiếp cận đơn giản và hiệu quả hơn, với các cấu hình tự động và các công cụ hỗ trợ như Spring Initializr, giúp nhà phát triển bắt đầu dự án mới nhanh chóng chỉ với một vài cú nhấp chuột.

Từ khi ra mắt, Spring Boot đã nhanh chóng trở thành một trong những framework phổ biến nhất trong cộng đồng Java, được sử dụng rộng rãi trong nhiều loại ứng dụng khác nhau, từ các ứng dụng web đơn giản đến các hệ thống microservices phức tạp. Sự phát triển và cải tiến liên tục của Spring Boot đã giúp nó trở thành một phần không thể thiếu trong hệ sinh thái Spring, giúp các nhà phát triển xây dựng các ứng dụng hiện đại một cách dễ dàng và nhanh chóng hơn.

2.2.3 Sự khác biệt giữa Spring và Spring Boot

Nếu bạn đã từng phát triển ứng dụng Java, chắc hẳn bạn đã trải qua sự phức tạp và tốn công sức khi thực hiện các tác vụ với cơ sở dữ liệu. Đó thực sự là một quá trình không mấy dễ dàng! Tuy nhiên, sự ra đời của Spring Boot đã giúp đơn giản hóa các tác vụ phức tạp liên quan đến cơ sở dữ liệu.

Spring là một tập hợp các framework con, bao gồm nhiều module khác nhau như Spring MVC, Spring JDBC, Spring AOP, Spring ORM và Spring Test. Tính

năng cốt lõi của Spring Framework chính là Dependency Injection (DI), cho phép các đối tượng được tiêm (injected) vào nhau một cách linh hoạt mà không cần phải tạo mới hoặc quản lý chúng trực tiếp. Điều này làm cho ứng dụng trở nên dễ dàng quản lý và mở rộng hơn.

Mặc dù Spring đã giúp cho việc phát triển các ứng dụng Java trở nên dễ dàng hơn, nhưng vẫn còn đó sự phức tạp và độ rộng lớn mà framework này mang lại. Việc học Spring Framework từ đầu có thể là một hành trình dài và đầy thách thức đối với nhiều lập trình viên. Đó cũng chính là lúc Spring Boot tỏa sáng với những cải tiến vượt trội.

Spring Boot ra đời với mục tiêu đơn giản hóa việc phát triển các ứng dụng Java bằng cách giảm thiểu cấu hình thủ công và cung cấp các thiết lập mặc định hợp lý. Thay vì phải cấu hình một cách phức tạp và chi tiết như trong Spring, Spring Boot giúp lập trình viên khởi tạo và chạy ứng dụng một cách nhanh chóng chỉ với một vài dòng mã. Spring Boot còn tích hợp sẵn các công cụ như máy chủ nhúng (embedded server), hỗ trợ cấu hình bằng YAML, và tự động hóa nhiều tác vụ mà trước đây cần phải làm thủ công.

2.2.4 Những tính năng nổi bật của Spring Boot

Dưới đây là những tính năng nổi bật của Spring Boot giúp nó trở thành một lựa chọn hàng đầu cho các nhà phát triển.

❖ Web Development

Spring Boot rất phù hợp cho phát triển các ứng dụng web nhờ module Spring chuyên dụng cho web. Framework này hỗ trợ tạo ra các máy chủ HTTP độc lập (self-contained HTTP server) bằng cách tích hợp các máy chủ nhúng như Tomcat, Jetty hoặc Undertow. Với module spring-boot-starter-web, bạn có thể nhanh chóng bắt đầu và vận hành ứng dụng của mình.

❖ SpringApplication

SpringApplication là một lớp cung cấp cách tiện lợi để khởi động ứng dụng Spring từ phương thức main của Java. Bạn có thể bắt đầu ứng dụng của mình chỉ bằng cách gọi phương thức tĩnh run():

```
public static void main(String[] args) {  
    SpringApplication.run(ClassName.class, args);  
}
```

```
}
```

❖ Application Events and Listeners

Spring Boot sử dụng các sự kiện để xử lý nhiều tác vụ khác nhau, cho phép bạn tạo các tệp factory để thêm các listener vào ứng dụng. Bạn có thể tham chiếu các listener này bằng cách sử dụng khóa `ApplicationListener`. Điều này giúp bạn dễ dàng quản lý và theo dõi các sự kiện xảy ra trong ứng dụng.

❖ Admin Support

Spring Boot cung cấp các tính năng hỗ trợ quản trị cho ứng dụng, cho phép bạn truy cập và quản lý ứng dụng từ xa. Tính năng này có thể được kích hoạt đơn giản bằng cách sử dụng thuộc tính `spring.application.admin.enabled`.

❖ Externalized Configuration

Spring Boot cho phép bạn “externalize” cấu hình của ứng dụng để có thể dễ dàng làm việc với các môi trường khác nhau. Ứng dụng sử dụng các tệp YAML để externalize cấu hình, giúp bạn dễ dàng quản lý và thay đổi cấu hình mà không cần chỉnh sửa mã nguồn.

❖ Properties Files

Spring Boot cung cấp một tập hợp phong phú các thuộc tính ứng dụng có thể được cấu hình thông qua các tệp properties của dự án. Ví dụ, bạn có thể cấu hình cổng máy chủ bằng cách đặt `server-port = 8082`. Điều này giúp tổ chức các thuộc tính của ứng dụng một cách dễ dàng và có hệ thống.

❖ YAML Support

YAML cung cấp một cách tiện lợi để xác định cấu hình theo cấu trúc phân cấp. Nó là một tập hợp con của JSON và được Spring Boot hỗ trợ tự động thông qua lớp `SpringApplication`. YAML là một sự thay thế hiệu quả cho các tệp properties truyền thống, đặc biệt khi cần cấu hình phức tạp.

❖ Type-safe Configuration

Spring Boot cung cấp cấu hình kiểu an toàn, giúp quản lý và xác thực cấu hình của ứng dụng một cách chính xác. Điều này đảm bảo rằng các cấu hình của ứng dụng luôn an toàn và không gây ra lỗi tại runtime. Bạn cũng có thể sử dụng các annotation được cung cấp bởi thư viện này để tăng cường khả năng cấu hình.

❖ Logging

Spring Boot sử dụng Common logging cho tất cả các hoạt động ghi log nội bộ. Các phụ thuộc về logging được quản lý mặc định và không nên thay đổi trừ khi cần thiết phải tùy chỉnh. Điều này giúp quá trình ghi log của ứng dụng trở nên ổn định và dễ dàng quản lý.

❖ Spring Security

Các ứng dụng Spring Boot, vốn là các ứng dụng web dựa trên Spring, mặc định được bảo mật với xác thực cơ bản (basic authentication) trên tất cả các endpoint HTTP. Spring Boot cung cấp một tập hợp phong phú các endpoint để phát triển các ứng dụng bảo mật, giúp bảo vệ dữ liệu và quyền truy cập vào hệ thống.

Giờ đây với Spring Boot, chúng ta có thể tạo dự án Spring một cách nhanh chóng và cấu hình cũng đơn giản dùng Sublime Text để phát triển luôn khỏi cần cài đặt eclipse hay netbean.

2.3. Giới thiệu Spring Cloud

2.3.1 Spring Cloud

Spring Cloud là một framework cung cấp bộ công cụ toàn diện để xây dựng và vận hành các ứng dụng theo kiến trúc microservices. Mục tiêu chính của nó là đơn giản hóa sự phức tạp vốn có của các hệ thống phân tán bằng cách cung cấp các giải pháp đã được kiểm chứng cho những bài toán phổ biến. Cụ thể, Spring Cloud cung cấp các implementation cho những design pattern quan trọng như:[3]

- Service Discovery (Khám phá dịch vụ).
- Centralized Configuration (Quản lý cấu hình tập trung).
- Circuit Breakers (Cơ chế ngắt mạch).
- Intelligent Routing (Định tuyến thông minh).
- Distributed Tracing (Truy vết phân tán).

2.3.2 Môi quan hệ với Spring Boot

Mối quan hệ và sự phân chia vai trò giữa Spring Boot và Spring Cloud rất rõ ràng:

- Spring Boot tập trung vào việc đơn giản hóa quá trình phát triển và đóng gói một service độc lập. Nó cung cấp cơ chế auto-configuration, quản lý dependency và máy chủ nhúng (embedded server) để bạn có thể nhanh chóng tạo ra một ứng dụng

sẵn sàng chạy (production-ready). Phạm vi của Spring Boot là bên trong một microservice.

- Spring Cloud giải quyết các vấn đề phát sinh khi nhiều service được tạo bởi Spring Boot cần tương tác với nhau trong một môi trường phân tán. Nó cung cấp hạ tầng và các pattern để các service này có thể giao tiếp, chịu lỗi và phối hợp một cách hiệu quả. Phạm vi của Spring Cloud là giữa các microservice.

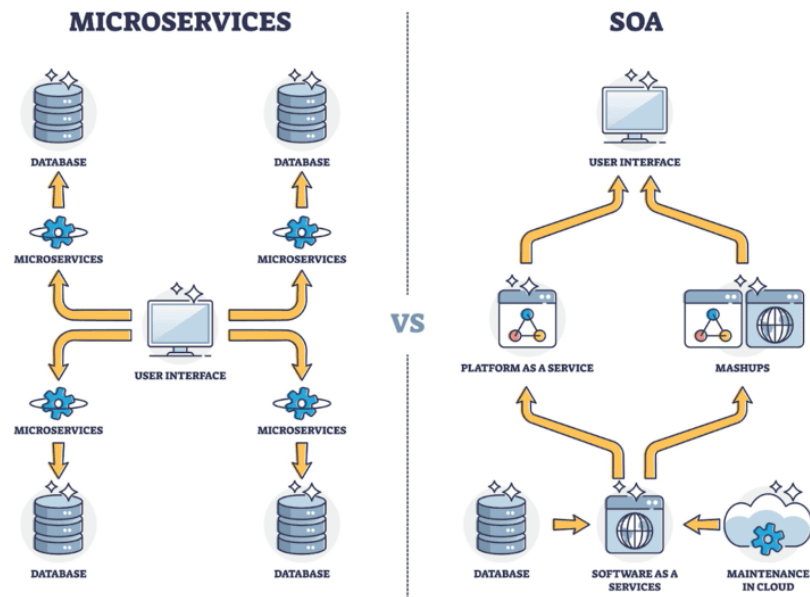
Nói một cách khác, Spring Boot cho phép bạn xây dựng các service, còn Spring Cloud cung cấp nền tảng để các service đó kết nối và vận hành một cách tin cậy như một hệ thống thống nhất.

2.3.3 Spring Cloud hỗ trợ cho quá trình chuyển đổi sang microservice

Kiến trúc microservices là một mô hình kiến trúc phần mềm, trong đó một ứng dụng được cấu thành từ một tập hợp các dịch vụ (services) nhỏ, độc lập và có khả năng-triển-khai-độc-lập (independently deployable). Mỗi service thường được xây dựng xoay quanh một nghiệp vụ (business capability), chạy trong tiến trình riêng và giao tiếp với các service khác thông qua các API được định nghĩa rõ ràng, thường là qua giao thức HTTP/REST hoặc gRPC.

Mô hình này đối lập với kiến trúc Monolithic (nguyên khối), nơi mà tất cả các thành phần chức năng được đóng gói và triển khai như một đơn vị duy nhất. Trong khi Monolithic đơn giản hơn về mặt vận hành ban đầu, nó bộc lộ nhiều nhược điểm khi hệ thống phát triển về quy mô và độ phức tạp, chẳng hạn như:

- Khó mở rộng: Phải nhân bản toàn bộ ứng dụng thay vì chỉ mở rộng thành phần đang chịu tải cao.
- Ràng buộc công nghệ: Khó áp dụng công nghệ mới cho một phần của hệ thống.
- Thiếu khả năng phục hồi: Lỗi ở một module có thể làm sập toàn bộ ứng dụng.
- Chu trình phát triển chậm: Mọi thay đổi nhỏ đều yêu cầu build và triển khai lại toàn bộ khối ứng dụng lớn.



HÌNH 2.3: Sự khác nhau giữa Microservices với SOA

2.3.4 Các vấn đề phát sinh mà Spring Cloud giải quyết

Khi chia tách một ứng dụng nguyên khối thành các microservice, chúng ta phải giải quyết các vấn đề phức tạp về tương tác và vận hành giữa các service này.

- Service Discovery (Độ phân giải vị trí dịch vụ): Trong một môi trường linh hoạt như Cloud hoặc containerized (Docker/Kubernetes), các instance của service được tạo ra và hủy đi một cách tự động. Địa chỉ IP và port của chúng là động và không cố định. Vậy làm thế nào một service client có thể tìm ra được địa chỉ mạng (network location) chính xác của một service mà nó cần gọi tại một thời điểm bất kỳ?
- Externalized Configuration (Ngoại vi hóa cấu hình): Mỗi service cần các thông tin cấu hình như chuỗi kết nối database, credentials, địa chỉ của các dịch vụ khác, feature flags... Quản lý các file cấu hình này phân tán ở từng service là một cơn ác mộng. Khi một thuộc tính chung (ví dụ: mật khẩu database) thay đổi, làm thế nào để cập nhật cho hàng chục service một cách đồng bộ mà không cần phải restart lại toàn bộ?
- Fault Tolerance (Khả năng chịu lỗi và chống lỗi dây chuyền): Trong một hệ thống phân tán, lỗi mạng hoặc một service bị quá tải/sập là điều không thể tránh khỏi. Một lời gọi từ service A đến service B bị treo có thể làm cạn kiệt tài nguyên (thread

pool) của A. Nếu nhiều service cùng gọi đến B, sự cố của B có thể lan truyền và gây ra lỗi dây chuyền (cascading failure), làm sập cả một phần hoặc toàn bộ hệ thống.

- API Gateway: Việc public hàng chục endpoint của các microservice ra bên ngoài cho client (web, mobile) là không thực tế và thiếu an toàn. Cần một điểm vào (entry point) duy nhất để xử lý các tác vụ chung (cross-cutting concerns) như xác thực (authentication), giới hạn truy cập (rate limiting), giám sát (monitoring), và định tuyến (routing) các request đến đúng service nội bộ.

- Cân bằng tải phía: Để đảm bảo tính sẵn sàng cao (high availability), mỗi service thường chạy nhiều instance. Khi service A gọi service B, làm thế nào nó có thể phân phối các yêu cầu một cách thông minh qua các instance của B để tránh tình trạng một instance bị quá tải trong khi các instance khác lại đang rảnh rỗi?

2.3.5 Spring Cloud giải quyết các vấn đề

Spring Cloud cung cấp một bộ các project được xây dựng dựa trên hệ sinh thái Spring Boot để đưa ra các giải pháp đã được chuẩn hóa cho những thách thức trên.

BẢNG 2.1: Cách Spring Cloud giải quyết các vấn đề

Thách thức Kỹ thuật	Giải pháp của Spring Cloud	Cơ chế hoạt động
Độ phân giải vị trí dịch vụ	Spring Cloud Netflix Eureka	Cung cấp một Service Registry. Mỗi microservice khi khởi động sẽ đăng ký (register) thông tin địa chỉ mạng của nó với Eureka Server và gửi các tín hiệu “heartbeat” định kỳ. Các service client sẽ truy vấn (query) Eureka Server để lấy danh sách các instance đang hoạt động của service mà nó cần gọi.

Ngoại vi hóa cấu hình	Spring Cloud Config	Cung cấp một Config Server quản lý cấu hình tập trung, thường được sao lưu bởi một Git repository. Các microservice trở thành client, lấy cấu hình từ Config Server khi khởi động và có thể được cấu hình để tự động làm mới (refresh) các thuộc tính mà không cần khởi động lại.
Khả năng chịu lỗi và chống lỗi dây chuyền	Spring Cloud Circuit Breaker (với Resilience4j)	Triển khai mô hình Circuit Breaker. Nó bao bọc các lời gọi mạng trong một state machine (CLOSED, OPEN, HALF-OPEN). Khi số lượng lỗi vượt ngưỡng, breaker sẽ “mở” (OPEN), các lời gọi sau đó sẽ thất bại ngay lập tức (fail-fast) thay vì cố gắng kết nối đến service đang lỗi, đồng thời cung cấp một phương án dự phòng (fallback).
API Gateway	Spring Cloud Gateway	Đóng vai trò là một Reverse Proxy và API Gateway. Nó định nghĩa các routes để ánh xạ request từ client tới các microservice tương ứng dựa trên các điều kiện (predicates) như đường dẫn, header... Gateway cũng áp dụng các

		filters để xử lý các tác vụ cross-cutting concerns.
Cân bằng tải	Spring Cloud LoadBalancer	Cung cấp một cơ chế Client-Side Load Balancing. Khi được tích hợp với service discovery (như Eureka), nó sẽ lấy danh sách các instance có sẵn và áp dụng một thuật toán (ví dụ: Round Robin) để chọn ra một instance cho mỗi request, phân phối đều tải trọng.

2.3.6 Hướng dẫn thực hành

Chúng ta sẽ xây dựng một ví dụ đơn giản gồm 1 Config Server và 1 Config Client.

Yêu cầu: Chuẩn bị một Git repository (trên GitHub, GitLab,) chứa các file cấu hình. Ví dụ, tạo một repo có cấu trúc:

```
/my-configs
├── application.yml
└── order-service.yml
```

Nội dung file `application.yml` (cấu hình chung):

```
common:
  message: "Hello from default config!"
```

Nội dung file `order-service.yml` (cấu hình riêng cho order-service):

```
order:
  discount:
    percentage: 15
```

❖ Cài đặt Config Server

Đây là một Spring Boot application độc lập.

Thêm Dependencies (pom.xml):

```
<dependencies>
```

```

<dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-config-server</artifactId>
</dependency>
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
</dependency>
</dependencies>

```

Kích hoạt Config Server: Thêm annotation `@EnableConfigServer` vào class Application chính.

```

@SpringBootApplication
@EnableConfigServer
public class ConfigServerApplication {
    public static void main(String[] args) {
        SpringApplication.run(ConfigServerApplication.class,
args);
    }
}

```

Cấu hình Server (application.yml): File này chỉ định port cho server và đường dẫn đến Git repository chứa cấu hình.

```

server:
    port: 8888 # Port mặc định cho config server

spring:
    cloud:
        config:
            server:
                git:
                    uri:https://github.com/your-username/my-configs.git
# Thay bằng URI repo của bạn
                    # clone-on-start: true # (Optional) Kéo repo về ngay
khi khởi động

```

Bây giờ, hãy khởi chạy Config Server. Bạn có thể kiểm tra bằng cách truy cập `http://localhost:8888/order-service/default`. Bạn sẽ thấy một JSON trả về chứa nội dung của cả 2 file `application.yml` và `order-service.yml`.

❖ Cấu hình Config Client

Đây là microservice `order-service`.

- Thêm Dependencies (`pom.xml`):

```
<dependencies>
  <dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-starter-config</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
  </dependency>
</dependencies>
```

- Cấu hình Bootstrap:

Để client biết địa chỉ Config Server ngay khi bắt đầu khởi động, chúng ta cần tạo một file `src/main/resources/bootstrap.yml` (thay vì `application.yml`).

```
spring:
  application:
    name: order-service # Tên này phải trùng với tên file cấu
    hình trong repo Git
  cloud:
    config:
      uri: http://localhost:8888 # Địa chỉ của Config Server
```

Lưu ý: `spring.application.name` rất quan trọng, Config Server sẽ dựa vào tên này để tìm file cấu hình tương ứng (`order-service.yml`).

❖ Cấu hình Client:

Thêm dependency `spring-boot-starter-actuator`.

Trong `application.yml` của `order-service`, thêm dòng sau để expose endpoint refresh:

```
management:
  endpoints:
    web:
      exposure:
        include: "refresh"
```

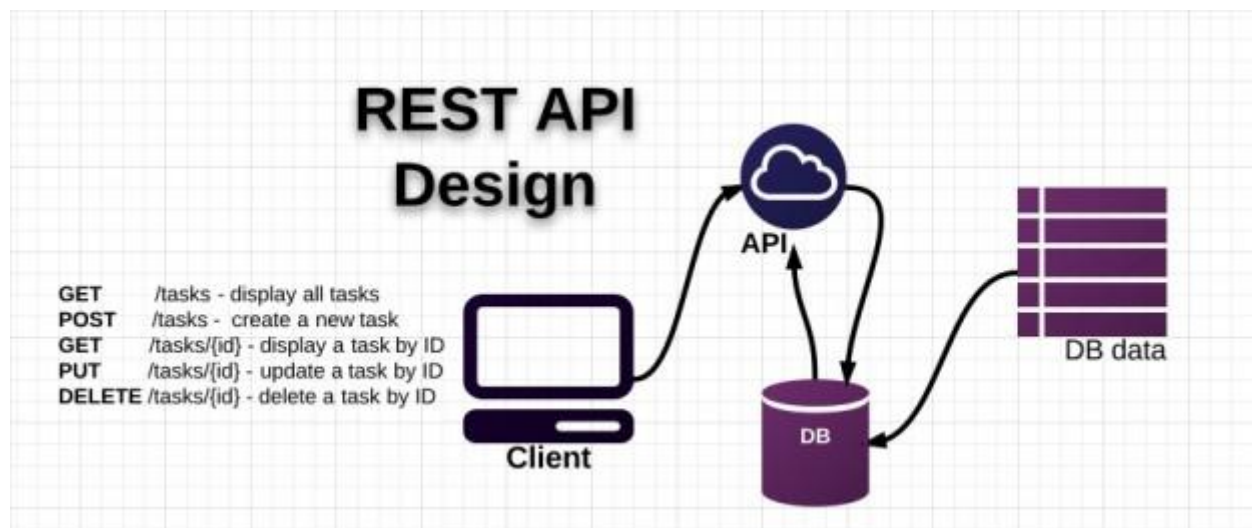
Thêm annotation `@RefreshScope` vào class Controller. Annotation này báo cho Spring biết rằng bean này có thể được tạo lại khi có sự kiện refresh.

```
@RestController
@RefreshScope // Rất quan trọng!
public class ConfigController {
    // ... nội dung như cũ
}
```

2.4. Tổng quan về RESTful API

2.4.1 RESTful API

RESTful API là một tiêu chuẩn dùng trong việc thiết kế API cho các ứng dụng web (thiết kế Web services) để tiện cho việc quản lý các resource. Nó chú trọng vào tài nguyên hệ thống (tệp văn bản, ảnh, âm thanh, video, hoặc dữ liệu động...), bao gồm các trạng thái tài nguyên được định dạng và được truyền tải qua HTTP.



HÌNH 2.4: RESTful API

2.4.2 Diễn giải các thành phần

API (Application Programming Interface) là một tập các quy tắc và cơ chế mà theo đó, một ứng dụng hay một thành phần sẽ tương tác với một ứng dụng hay thành

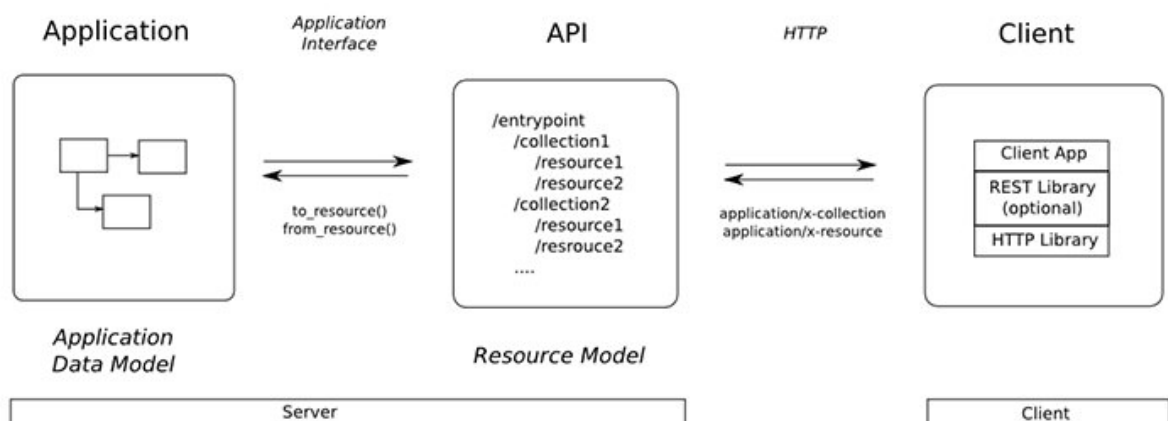
phần khác. API có thể trả về dữ liệu mà bạn cần cho ứng dụng của mình ở những kiểu dữ liệu phổ biến như JSON hay XML.

REST (Representational State Transfer) là một dạng chuyển đổi cấu trúc dữ liệu, một kiểu kiến trúc để viết API. Nó sử dụng phương thức HTTP đơn giản để tạo cho giao tiếp giữa các máy. Vì vậy, thay vì sử dụng một URL cho việc xử lý một số thông tin người dùng, REST gửi một yêu cầu HTTP như GET, POST, DELETE, vv đến một URL để xử lý dữ liệu.

RESTful API là một tiêu chuẩn dùng trong việc thiết kế các API cho các ứng dụng web để quản lý các resource. RESTful là một trong những kiểu thiết kế API được sử dụng phổ biến ngày nay để cho các ứng dụng (web, mobile...) khác nhau giao tiếp với nhau.

Chức năng quan trọng nhất của REST là quy định cách sử dụng các HTTP method (như GET, POST, PUT, DELETE...) và cách định dạng các URL cho ứng dụng web để quản các resource. RESTful không quy định logic code ứng dụng và không giới hạn bởi ngôn ngữ lập trình ứng dụng, bất kỳ ngôn ngữ hoặc framework nào cũng có thể sử dụng để thiết kế một RESTful API.

2.4.3 RESTful hoạt động



HÌNH 2.5: RESTful hoạt động

REST hoạt động chủ yếu dựa vào giao thức HTTP. Các hoạt động cơ bản nêu trên sẽ sử dụng những phương thức HTTP riêng.

- GET (SELECT): Trả về một Resource hoặc một danh sách Resource.
- POST (CREATE): Tạo mới một Resource.

- PUT (UPDATE): Cập nhật thông tin cho Resource.
- DELETE (DELETE): Xóa một Resource.

Những phương thức hay hoạt động này thường được gọi là CRUD tương ứng với Create, Read, Update, Delete – Tạo, Đọc, Sửa, Xóa.

2.4.4 Authentication và dữ liệu trả về

RESTful API không sử dụng session và cookie, nó sử dụng một access_token với mỗi request. Dữ liệu trả về thường có cấu trúc như sau:

```
{
  "data" : {
    "id": "1",
    "name": "TopDev"
  }
}
```

2.4.5 Status code

Khi chúng ta request một API nào đó thường thì sẽ có vài status code để nhận biết sau:

- 200 OK – Trả về thành công cho những phương thức GET, PUT, PATCH hoặc DELETE.
- 201 Created – Trả về khi một Resource vừa được tạo thành công.
- 204 No Content – Trả về khi Resource xóa thành công.
- 304 Not Modified – Client có thể sử dụng dữ liệu cache.
- 400 Bad Request – Request không hợp lệ
- 401 Unauthorized – Request cần có auth.
- 403 Forbidden – bị từ chối không cho phép.
- 404 Not Found – Không tìm thấy resource từ URI
- 405 Method Not Allowed – Phương thức không cho phép với user hiện tại.
- 410 Gone – Resource không còn tồn tại, Version cũ đã không còn hỗ trợ.
- 415 Unsupported Media Type – Không hỗ trợ kiểu Resource này.
- 422 Unprocessable Entity – Dữ liệu không được xác thực
- 429 Too Many Requests – Request bị từ chối do bị giới hạn

2.4.6 Xây dựng API với Laravel

Lấy việc xây dựng api trên Laravel để làm ví dụ, trước khi đi vào ta tổng quan về Http Request.

❖ HTTP Request

HTTP request có tất cả 9 loại method, 2 loại được sử dụng phổ biến nhất là GET và POST

- GET: được sử dụng để lấy thông tin từ server theo URI đã cung cấp.
- HEAD: giống với GET nhưng response trả về không có body, chỉ có header.
- POST: gửi thông tin tới sever thông qua các biểu mẫu http.
- PUT: ghi đè tất cả thông tin của đối tượng với những gì được gửi lên.
- PATCH: ghi đè các thông tin được thay đổi của đối tượng.
- DELETE: xóa tài nguyên trên server.
- CONNECT: thiết lập một kết nối tới server theo URI.
- OPTIONS: mô tả các tùy chọn giao tiếp cho resource.
- TRACE: thực hiện một bài test loop – back theo đường dẫn đến resource.

❖ RESTful Route

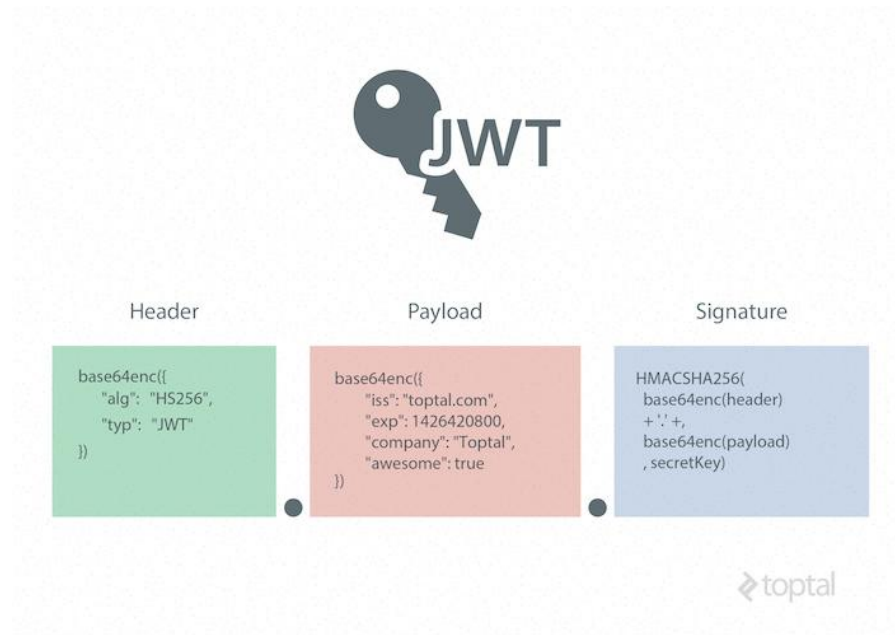
Viết Api thì sẽ khai báo router vào file routes/api.php thay vì sử dụng file routes/web.php. Các setting mặc cho file api.php trong laravel:

- Url: những route được khai báo trong file này mặc định có prefix url là api (ví dụ: topdev.vn/api/products)
- Middleware: mặc định sẽ được gán Middleware Group là api, trong file app/Http/Kernel sẽ thấy 2 middleware thuộc Middleware Group: api là throttle (giới hạn request / time) và bindings (model binding).

2.5. Tìm hiểu về JSON Web Token

2.5.1 Json Web Token - jwt

JWT là một phương tiện đại diện cho các yêu cầu chuyển giao giữa hai bên Client – Server , các thông tin trong chuỗi JWT được định dạng bằng JSON. Trong đó chuỗi Token phải có 3 phần là header, phần payload và phần signature được ngăn bằng dấu “.”[17]



HÌNH 2.6: JSON Web Token

2.5.2 Cấu trúc của JSON Web Token:

Như ở trên đã nói JSON Web Token bao gồm 3 phần, được ngăn cách nhau bởi dấu chấm (.):

- Header
- Payload
- Signature (chữ ký)

❖ Header

Phần header sẽ chứa kiểu dữ liệu , và thuật toán sử dụng để mã hóa ra chuỗi JWT

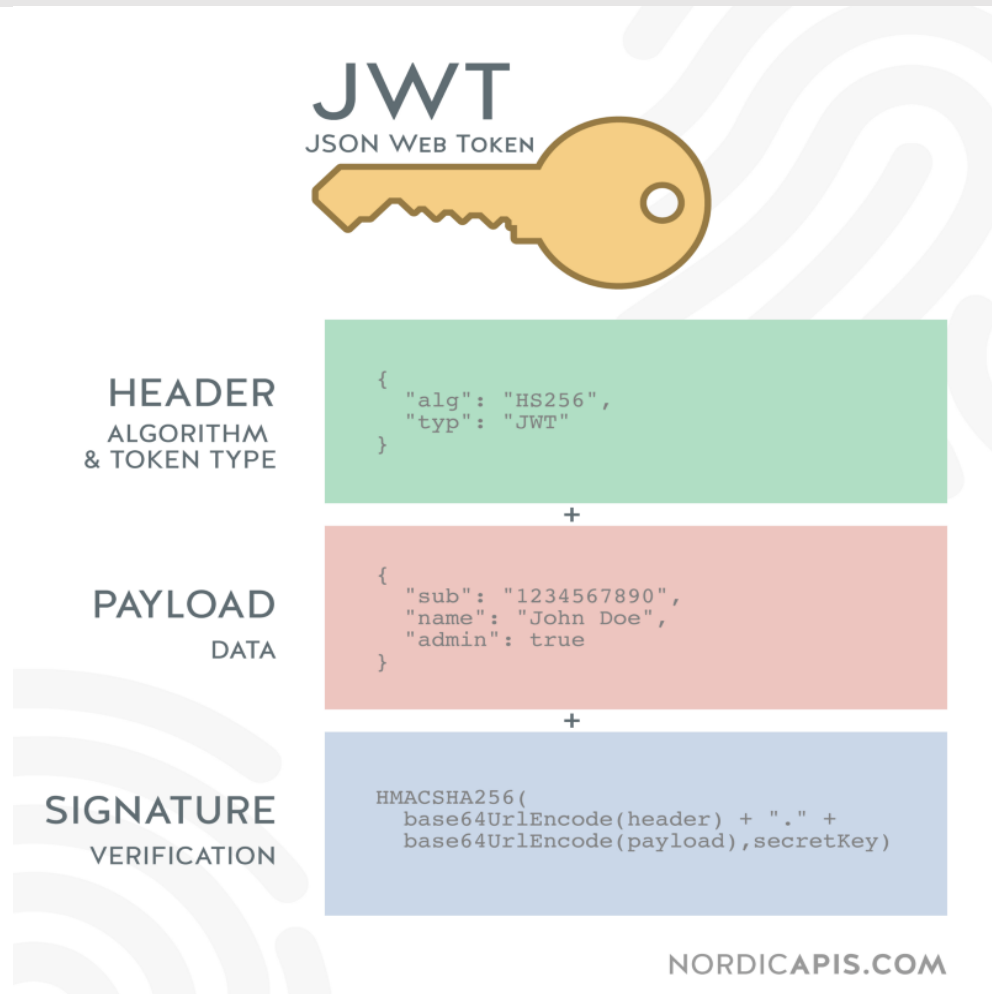
```
{  
  "typ": "JWT",  
  "alg": "HS256"  
}
```

- “typ” (type) chỉ ra rằng đối tượng là một JWT
- “alg” (algorithm) xác định thuật toán mã hóa cho chuỗi là HS256

❖ Payload

Phần payload sẽ chứa các thông tin mình muốn đặt trong chuỗi Token như username , userId , author , ... ví dụ:

```
{
  "user_name": "admin",
  "user_id": "1513717410",
  "authorities": "ADMIN_USER",
  "jti": "474cb37f-2c9c-44e4-8f5c-1ea5e4cc4d18"
}
```



HÌNH 2.7: Cấu trúc của JSON Web Token

2.5.3 Signature

Phần chữ ký này sẽ được tạo ra bằng cách mã hóa phần header , payload kèm theo một chuỗi secret (khóa bí mật) , ví dụ:

```
data      =   base64urlEncode(   header   )   +   "."   +
base64urlEncode( payload )
signature = Hash( data, secret );
```

base64UrlEncoder : thuật toán mã hóa **header** và **payload**

Đoạn code trên sau khi mã hóa header và payload bằng thuật toán base64UrlEncode ta sẽ có chuỗi như sau:

```
// header
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9
// payload
eyJhdWQiOi0lsidGVzdGp3dHJlc291cmNlaWQiXSwidXNlcl9uYW11IjoieYWRtaW4iLCJzY29wZSI6WyJyZWFKIiwid3JpdGUixSwiZXhwIjoxNTEzNzE
```

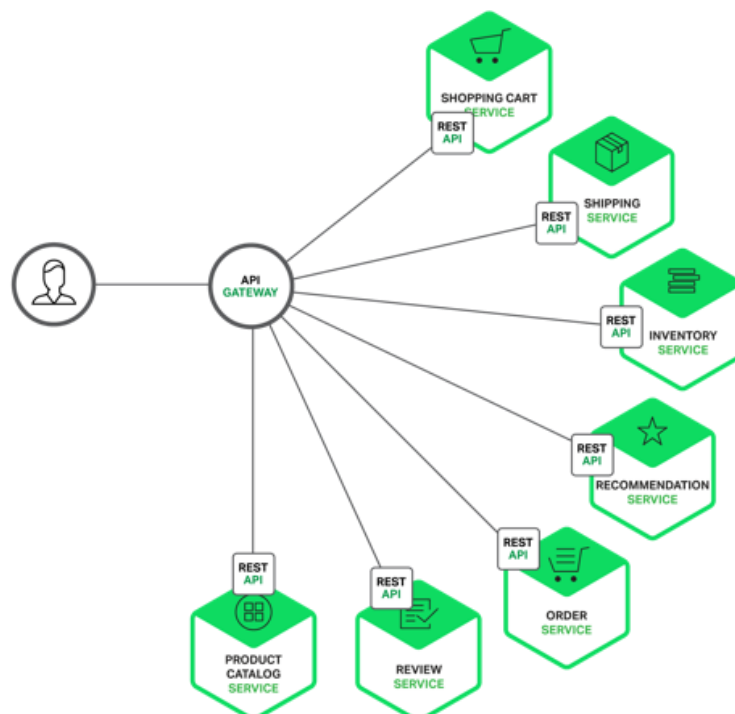
Sau đó mã hóa 2 chuỗi trên kèm theo secret (khóa bí mật) bằng thuật toán HS256 ta sẽ có chuỗi signature như sau:

```
9nRhBWiRoryc8fV5xRpTmw9iyJ6EM7WTGTjvCM1e36Q
```

2.6. Tổng quan API Gateway

2.6.1 API Gateway

Như mình nói ở trên, API Gateway có thể coi là một cổng trung gian, nó là cổng vào duy nhất tới hệ thống microservices của chúng ta, api gateway sẽ nhận các requests từ phía client, chỉnh sửa, xác thực và điều hướng chúng đến các API cụ thể trên các services phía sau. Khi này sơ đồ hệ thống của chúng ta sẽ trông như này.[10]



HÌNH 2.8: API Gateway

Ngoài nhiệm vụ chính là proxy request thì một hệ thống API Gateway thường sẽ đảm nhận luôn vài vai trò khác như bảo mật API, monitoring, analytics số lượng requests cũng như tình trạng hệ thống phía sau.

2.6.2 Lợi ích của việc sử dụng API Gateway

Che dấu được cấu trúc của hệ thống microservices với bên ngoài: Clients sẽ tương tác với hệ thống của chúng ta thông qua api gateway chứ không gọi trực tiếp tới một services cụ thể, các endpoints của các services sẽ chỉ được gọi nội bộ, tức là gọi giữa các services với nhau hoặc được gọi từ API gateway, người dùng sẽ gọi các api này thông qua các public endpoints từ API Gateway. Chính vì vậy cho nên phía client không cần và cũng không thể biết được các services phía backend được phân chia như thế nào, việc refactor code frontend cũng dễ dàng hơn đối với lập trình viên.

Phần code phía frontend sẽ gọn gàng hơn: Vì không phải tracking nhiều endpoints, tất cả chỉ việc gọi đến api gateway nên phần code frontend sẽ gọn gàng hơn so với việc phải tracking hàng tá endpoints trên từng services một, nhất là khi mà hệ thống ngày một PHÌNH to ra.

Dễ dàng theo dõi và quản lý traffic: Hầu hết các hệ thống API gateway phổ biến hiện nay đều sẽ đi kèm tính năng theo dõi và quản lý lượng traffic bằng GUI hoặc thông qua các APIs của hệ thống Gateway, VD như với Kong (bản EE).

Requests caching và cân bằng tải: API Gateway sẽ kiêm luôn vai trò load balancer của hệ thống, requests sẽ không được gửi trực tiếp đến backend nên sẽ giảm thiểu được rủi ro hệ thống bị quá tải.

Thêm một lớp bảo mật nữa cho hệ thống: API gateway giúp ngăn chặn các cuộc tấn công bằng cách thêm một lớp bảo vệ các loại tấn công như ddos, sql injections,...

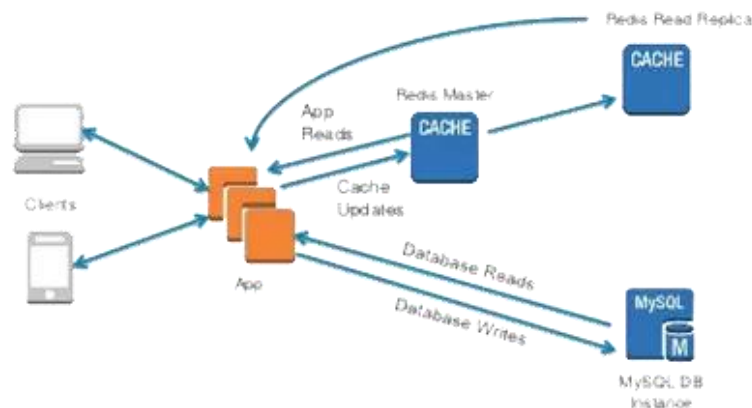
Thay thế authentication services: API gateway thường cung cấp nhiều cơ chế xác thực, chúng ta có thể sử dụng nó để xác thực người dùng luôn, giúp tiết kiệm thời gian và làm hệ thống chúng ta đơn giản hơn. VD một vài cơ chế xác thực hỗ trợ bởi Kong API gateway

2.7. Tổng quan Redis

2.7.1 Giới thiệu

Thông thường, cứ mỗi request từ phía client thì lại chạy vào database lôi đống kết quả về cho client. Cho tới khi số lượng request tăng lên đáng kể sẽ khiến cho thằng CPU và Hard Disk phải gánh cái đống việc lặp đi lặp lại ấy, có khi vừa mới tính toán xong xuôi, có kết quả rồi mà bị gọi đến thì vẫn phải đi làm lại y hệt công việc cũ trong uất ức. Để giải quyết nỗi uất ức trên thì Caching đã làm tốt nhiệm vụ của nó bằng việc giúp server ghi nhớ kết quả của cả quá trình tính toán, để trả về cho những lần request tương tự từ client trong tương lai.[5][13]

Redis: Architecture



HÌNH 2.9: Redis

Redis là tên viết tắt của Remote Dictionary Server (Máy chủ từ điển từ xa), là kho dữ liệu key-value, trong bộ nhớ, mã nguồn mở và có tốc độ truy cập nhanh để dùng làm cơ sở dữ liệu, cache, message broker và queue. Redis có thể được sử dụng như một database được lưu trữ trong ram để tăng tốc độ xử lý.

Redis hiện cung cấp thời gian phản hồi ở tốc độ chưa đến một mili giây, vì Redis cho phép bạn lưu trữ trên RAM của mình nhanh hơn so với khi lưu trữ trên các ổ cứng, khoảng 150.000 lần so với truy cập ổ HDD và nhanh hơn 500 lần so với truy cập SSD.

Thông thường khi ta sử dụng cơ sở dữ liệu thì vẫn lưu trữ trên ổ đĩa cứng. Ví dụ khi query một cơ sở dữ liệu để đọc 10.000 bản ghi. Nếu dữ liệu được lưu trữ trên đĩa, thì nó sẽ mất trung bình 30 giây, trong khi chỉ mất khoảng 0,0002 giây để đọc từ

RAM, đó chính là lí do mà Redis dùng RAM để lưu trữ. Redis thường được tạo trên một máy chủ riêng hoặc set giới hạn bộ nhớ nhất định được sử dụng trên máy chủ dùng chung cho nên sẽ không bị tình trạng thiếu RAM khi chạy đồng thời các ứng dụng khác.

Tuy nhiên, khi lưu trữ trên RAM thì nhanh thật nhưng lại gặp 1 vấn đề là bị mất điện thì dữ liệu cũng mất tiêu. Để ngăn chặn việc mất dữ liệu xảy ra, có một mô-đun được tích hợp sẵn để ghi trạng thái trong bộ nhớ vào file trên đĩa trong những trường hợp nhất định. Các file này được tải lại khi khởi động lại redis. Vì vậy, dữ liệu sẽ không bị mất. Ngoài ra, để tăng tính sẵn sàng và khả năng chịu lỗi của hệ thống. Redis có thể cấu hình theo dạng Cluster với kỹ thuật Master-Slave giúp hệ thống redis luôn sẵn sàng đáp ứng sao lưu dữ liệu trên đĩa cứng và phục hồi dữ liệu khi gặp sự cố.

2.7.2 Lợi ích của Redis

❖ In-Memory Data Store

Toàn bộ dữ liệu Redis nằm trong bộ nhớ chính của máy chủ, khác với các cơ sở dữ liệu thường lưu phần lớn dữ liệu trên ổ đĩa hoặc ổ SSD. So với cơ sở dữ liệu trên ổ đĩa truyền thống trong đó phần lớn các tác vụ đều yêu cầu truy cập lặp lại tới ổ đĩa, còn Redis thì chỉ việc lấy ra kết quả đã tính toán. Do đó hiệu suất nhanh thấy rõ rệt với các tác vụ đọc hoặc ghi thông thường mất chưa đầy một mili giây và hỗ trợ hàng triệu tác vụ mỗi giây.

❖ Cấu trúc dữ liệu linh hoạt

Khác với những kho dữ liệu key-value đơn giản có cấu trúc dữ liệu giới hạn, Redis có nhiều cấu trúc dữ liệu khác nhau nên đáp ứng được nhu cầu ứng dụng của bạn. Kiểu dữ liệu Redis gồm có:

- String – văn bản hoặc dữ liệu nhị phân có kích thước lên tới 512MB
- List – một tập hợp các chuỗi được sắp xếp theo thứ tự như khi được thêm vào
- Set – một tập hợp chưa được sắp xếp các chuỗi, có khả năng giao cắt, liên kết và khác với các kiểu tập khác
- Sorted Set – Tập được sắp xếp theo giá trị
- Hash – một cấu trúc dữ liệu dùng để lưu trữ danh sách các trường và giá trị
- Bitmap – một kiểu dữ liệu cho phép thực hiện các tác vụ quy mô bit

– HyperLogLogs – một cấu trúc dữ liệu xác suất để ước tính các thành phần duy nhất trong một tập dữ liệu

❖ Đơn giản và dễ sử dụng

Redis đơn giản hóa code của bạn bằng cách cho phép bạn viết ít dòng code hơn để lưu trữ, truy cập và sử dụng dữ liệu trong các ứng dụng của mình. Ví dụ: nếu ứng dụng của bạn có dữ liệu được lưu dạng hashmap và bạn muốn lưu trữ dữ liệu đó trong DB thì bạn chỉ cần sử dụng cấu trúc dữ liệu hashes để lưu chúng. Thao tác tương tự trên DB không có cấu trúc dữ liệu hashes sẽ yêu cầu nhiều dòng code để chuyển đổi từ định dạng này sang định dạng khác. Với mỗi kiểu dữ liệu thì Redis thêm nhiều tùy chọn để thao tác và tương tác với dữ liệu của bạn.

❖ Replication - Persistence

Redis sử dụng kiến trúc primary replica và hỗ trợ sao chép không đồng bộ, dữ liệu có thể được sao chép sang nhiều máy chủ khác nhau. Điều này giúp hiệu suất đọc được cải thiện (vì các yêu cầu có thể được phân chia giữa các máy chủ) và phục hồi nhanh hơn khi máy chủ chính gặp sự cố ngừng hoạt động. Để duy trì, Redis hỗ trợ sao lưu point-in-time backups (sao chép dữ liệu Redis vào đĩa).

❖ Độ khả dụng cao và quy mô linh hoạt

Redis có kiến trúc primary replica theo cấu trúc liên kết dạng một nút chính hoặc cụm. Kiến trúc này cho phép bạn xây dựng những giải pháp có độ khả dụng cao, đảm bảo hiệu suất ổn định và tin cậy. Khi cần điều chỉnh kích thước cụm, bạn có nhiều tùy chọn khác nhau để thay đổi quy mô theo chiều dọc và thay đổi quy mô theo chiều ngang. Việc này cho phép tăng cụm theo nhu cầu của bạn.

❖ Khả năng mở rộng

Redis là dự án mã nguồn mở đã có cộng đồng lớn tin dùng. Không có giới hạn về nhà cung cấp hoặc công nghệ vì Redis được có tính tiêu chuẩn mở, hỗ trợ các định dạng dữ liệu mở và có tập hợp máy khách phong phú.

2.8. Tìm hiểu về hệ quản trị cơ sở dữ liệu PostgreSQL

2.8.1 PostgreSQL

PostgreSQL là một hệ thống quản trị cơ sở dữ liệu quan hệ và đối tượng (object-relational database management system) miễn phí và nguồn mở (RDBMS) tiên tiến nhất hiện nay. khả năng mở rộng cao và tuân thủ các tiêu chuẩn kỹ thuật. Nó

được thiết kế để xử lý một loạt các khối lượng công việc lớn, từ các máy tính cá nhân đến kho dữ liệu hoặc dịch vụ Web có nhiều người dùng đồng thời.

- PostgreSQL được phát triển bởi PostgreSQL Global Development Group, Phát hành lần đầu: 08/07/1996

- PostgreSQL linh động có thể chạy được trên nhiều nền tảng khác nhau như Mac OS X, Solaris và Windows.

- PostgreSQL là một phần mềm mã nguồn mở miễn phí bởi vậy PostgreSQL có thể được dùng, sửa đổi và phổ biến bởi bất kỳ ai cho bất kỳ mục đích nào.

- PostgreSQL có tính ổn định cao.

- PostgreSQL là hệ thống quản lý cơ sở dữ liệu đầu tiên triển khai tính năng kiểm soát đồng thời nhiều phiên bản (MVCC).

2.8.2 Tính năng PostgreSQL

PostgreSQL tích hợp nhiều tính năng tuyệt vời giúp hỗ trợ nhà phát triển xây dựng app đáp ứng các chức năng phức tạp, truy vấn nhanh chóng và bảo mật duy trì tính toàn vẹn và độ tin cậy. Để đáng tin cậy hơn, Postgresql cung cấp các tùy chọn bảo mật, xác thực và khôi phục thảm họa khác nhau. PostgreSQL được chứng minh là có khả năng mở rộng cao cả về số lượng dữ liệu và số lượng người dùng có thể thao tác cùng lúc.

Dưới đây là một số các tính năng nổi bật mình tổng hợp lại.

- Câu truy vấn phức hợp (complex query)

- Thủ tục sự kiện (trigger)

- Các khung nhìn (view)

- Tính toàn vẹn của các giao dịch (integrity transactions)

- Việc kiểm tra truy cập đồng thời đa phiên bản (multiversion concurrency control)

- Truy vấn xử lý song song (parallel query)

- Sao chép dữ liệu dạng luồng (Streaming replication)

❖ Kiểu dữ liệu

- Nguyên hàm: Số nguyên, số, chuỗi, Boolean

- Cấu trúc: Date/Time, Array, Phạm vi, UUID

- Document: JSON/JSONB, XML, Key-value (Hstore)

- Hình học: Điểm, Đường thẳng, Vòng tròn, Đa giác
- Tùy chỉnh: Composite, Các kiểu tùy chỉnh

❖ Toàn vẹn dữ liệu

- UNIQUE, NOT NULL
- Primary Keys
- Foreign Keys
- Ràng buộc loại trừ
- Khóa hàm số, Khóa khuyến nghị
- Đồng quy, hiệu suất
- Lập danh mục: B-tree, Multicolumn, Expressions, Partial
- Lập danh mục nâng cao: GiST, SP-Gist, KNN Gist, GIN, BRIN, Bloom filters
- Trình lập kế hoạch / trình tối ưu hóa truy vấn phức tạp, quét index-only, thống kê số liệu trên nhiều cột.

kê số liệu trên nhiều cột.

- Giao tác, Giao tác dạng nest (thông qua lưu điểm)
- Điều khiển đồng thời nhiều phiên bản (MVCC)
- Truy vấn đọc song song
- Phân vùng bảng
- Tất cả các mức độ giao dịch độc lập được xác định trong tiêu chuẩn SQL, bao

gồm cả Serializable

- Độ tin cậy, phục hồi sau thảm họa
- Ghi nhật ký ghi trước (Write-ahead Logging - WAL)
- Replication: Không đồng bộ, Đồng bộ, Logical
- Khôi phục điểm-theo-thời gian (Point-in-time-recovery - PITR), active

standbys

- Không gian bảng

❖ Bảo mật

- Xác thực: GSSAPI, SSPI, LDAP, SCRAM-SHA-256, Certificate và các hình thức khác

- Hệ thống kiểm soát truy cập mạnh mẽ
- Bảo mật cấp độ cột và hàng

❖ Khả năng mở rộng

- Phương pháp lưu trữ
- Ngôn ngữ thủ tục: PL / PGSQL, Perl, Python (và nhiều ngôn ngữ khác)
- Trình wrapper dữ liệu ngoài: kết nối với các cơ sở dữ liệu hoặc luồng khác với giao diện SQL chuẩn
- Và nhiều tiện ích mở rộng cung cấp chức năng bổ sung, bao gồm cả PostGIS
- Tìm kiếm văn bản
- Hỗ trợ các bộ ký tự quốc tế, ví dụ: thông qua ICU collations
- Tìm kiếm văn bản đầy đủ

2.9. Tìm hiểu và tổng quan về Docker

2.9.1 Docker

Docker là một nền tảng giúp đóng gói ứng dụng cùng với toàn bộ môi trường chạy (thư viện, cấu hình, dependencies) vào một Container.[12]

Nguyên tắc hoạt động:

Build Once – Run Anywhere

Xây dựng một lần và chạy được trên nhiều môi trường khác nhau mà không cần cấu hình lại.

Ví dụ:

- Máy lập trình viên chạy được.
- Máy kiểm thử chạy được.
- Server triển khai thực tế cũng chạy được.

2.9.2 Docker giải quyết vấn đề

Trước đây khi triển khai ứng dụng cần:

- Cài Java
- Cài MySQL
- Cài Redis
- Cấu hình môi trường
- Cấu hình phiên bản

Dễ xảy ra xung đột giữa các phần mềm.

Docker giúp:

- Đóng gói toàn bộ ứng dụng.
- Tạo môi trường chạy độc lập.

- Triển khai nhanh chóng.
- Dễ mở rộng hệ thống.

2.9.3 Container

Container là môi trường chạy độc lập chứa:

- Source code
- Runtime
- Thư viện
- Cấu hình

Mỗi container hoạt động như một máy tính nhỏ nhưng dùng chung Kernel của hệ điều hành Host nên nhẹ và nhanh hơn máy ảo.

2.9.4 Docker và Virtual Machine

BẢNG 2.2: Docker và Virtual Machine

Docker	Virtual Machine
Dùng chung Kernel	Có Guest OS riêng
Nhẹ	Nặng
Khởi động vài giây	Khởi động vài chục giây
Tiêu tốn ít RAM	Tiêu tốn nhiều RAM
Dễ scale	Scale chậm

2.9.5 Các thành phần quan trọng của Docker

A. Docker Image

Là khuôn mẫu (template) để tạo Container.

Ví dụ:

- Ubuntu Image
- MySQL Image
- Redis Image
- OpenJDK Image

B. Docker Container

Là một instance đang chạy của Image.

Ví dụ:

```
docker run mysql
```

C. Dockerfile

Là file chứa các lệnh dùng để tạo Docker Image.

Ví dụ:

```
FROM openjdk:21
COPY target/app.jar app.jar
ENTRYPOINT ["java", "-jar", "app.jar"]
```

D. Docker Hub

Kho lưu trữ Docker Image.

Tương tự:

- GitHub lưu source code.
- Docker Hub lưu Docker Images.

2.9.6 Docker Compose

Docker Compose giúp chạy nhiều Container cùng lúc bằng một file cấu hình.

Ví dụ hệ thống đồ án:

```
API Gateway
User Service
Student Service
MySQL
Redis
```

Thay vì chạy từng container riêng lẻ:

```
docker run ...
docker run ...
docker run ...
```

Chỉ cần:

```
docker-compose up
```

2.10. Tìm hiểu Apache Jmeter

2.10.1 Apache Jmeter

Apache JMeter là một công cụ mã nguồn mở được phát triển bởi Apache Software Foundation, được sử dụng để kiểm thử tải (Load Testing), kiểm thử hiệu năng (Performance Testing) và đánh giá độ ổn định của các ứng dụng web, ứng dụng di động và hệ thống phân tán.

JMeter cho phép tạo ra các yêu cầu (request) đến hệ thống và đo lường thời gian phản hồi, từ đó đánh giá khả năng xử lý của ứng dụng khi có nhiều người dùng truy cập cùng lúc. Công cụ này cung cấp các báo cáo chi tiết về thời gian phản hồi, tỷ lệ lỗi, thông lượng (Throughput) và mức sử dụng tài nguyên hệ thống.

JMeter hỗ trợ nhiều giao thức khác nhau như:

- HTTP/HTTPS
- FTP
- JDBC (Cơ sở dữ liệu)
- SOAP
- REST API
- TCP
- LDAP
- SMTP

Ngoài ra, JMeter còn hỗ trợ mở rộng thông qua Plugin và Script, cho phép người dùng xây dựng các kịch bản kiểm thử phức tạp bằng các ngôn ngữ như Java, Groovy và JavaScript.

2.10.2 Apache JMeter có thể làm

JMeter được sử dụng để thực hiện nhiều nhiệm vụ liên quan đến kiểm thử tải và hiệu năng, bao gồm:

- Kiểm tra hiệu suất ứng dụng Web và Mobile: JMeter tạo ra các yêu cầu giả lập từ người dùng và đo lường thời gian phản hồi của hệ thống để đánh giá hiệu năng hoạt động.
- Kiểm tra độ ổn định của hệ thống: Công cụ có thể chạy các bài kiểm thử trong thời gian dài nhằm đánh giá tính ổn định của ứng dụng dưới áp lực tải liên tục.

- Mô phỏng hành vi người dùng: JMeter cho phép mô phỏng nhiều người dùng thực hiện các thao tác khác nhau như đăng nhập, tìm kiếm dữ liệu, gửi biểu mẫu hoặc gọi API.
- Phát hiện lỗi và sự cố: Thông qua các báo cáo chi tiết về thời gian phản hồi, tỷ lệ lỗi và hiệu suất hệ thống, JMeter giúp phát hiện các điểm nghẽn và lỗi tiềm ẩn.
- Kiểm tra hiệu năng hệ thống phân tán: JMeter có thể đánh giá hiệu suất của các hệ thống gồm nhiều máy chủ, ứng dụng và cơ sở dữ liệu hoạt động đồng thời.
- Tự động hóa kiểm thử: Người dùng có thể xây dựng các kịch bản kiểm thử tự động, giúp tiết kiệm thời gian và nâng cao độ chính xác của quá trình kiểm thử.

2.10.3 Đặc trưng của Apache JMeter

Apache JMeter sở hữu nhiều tính năng nổi bật giúp nó trở thành một trong những công cụ kiểm thử hiệu năng phổ biến nhất hiện nay.

Hỗ trợ nhiều giao thức: JMeter hỗ trợ nhiều giao thức như HTTP, HTTPS, FTP, JDBC, SOAP, REST API và nhiều giao thức khác.

Mô phỏng hành vi người dùng: Cho phép mô phỏng nhiều người dùng thực hiện các hành động khác nhau trên hệ thống nhằm đánh giá khả năng đáp ứng của ứng dụng.

Tạo kịch bản kiểm thử phức tạp: Người dùng có thể xây dựng các kịch bản kiểm thử với nhiều bước, điều kiện và luồng xử lý khác nhau.

Đo thời gian phản hồi: JMeter có khả năng ghi nhận và phân tích thời gian phản hồi của từng yêu cầu gửi đến hệ thống.

Báo cáo và phân tích kết quả

Cung cấp các báo cáo chi tiết về:

- Response Time
- Throughput
- Error Rate
- Latency
- Performance Metrics
- Dễ dàng phân tích kết quả kiểm thử.

Khả năng mở rộng và tùy chỉnh: JMeter hỗ trợ Plugin và Script, cho phép mở rộng tính năng theo nhu cầu thực tế.

Hỗ trợ kiểm thử API: Người dùng có thể kiểm thử các API RESTful và SOAP bằng cách gửi các yêu cầu HTTP và đánh giá phản hồi nhận được từ hệ thống.

2.11. Tổng quan Swagger

2.11.1 Swagger

Swagger là một bộ công cụ mã nguồn mở xoay quanh OpenAPI Specification, dùng để thiết kế, mô tả, tạo tài liệu và kiểm thử các REST API. Thay vì phải viết tài liệu API thủ công, bạn mô tả API trong một file OpenAPI (YAML/JSON), Swagger sẽ dùng đặc tả này để sinh ra tài liệu tương tác, sinh code client/server và hỗ trợ quy trình phát triển API theo hướng “design first”.[7]

Trong các dự án thực tế, Swagger thường được tích hợp sẵn vào ứng dụng (ví dụ qua Swagger UI trong ASP.NET Core, [Spring Boot](#), [NestJS](#),...) để đội ngũ dev, tester và đối tác truy cập tài liệu API thống nhất, cập nhật theo đúng phiên bản code đang chạy. Khi đặt API trên VPS hoặc server riêng, việc chuẩn hóa tài liệu với Swagger giúp việc debug, mở rộng tính năng và bàn giao giữa các nhóm trở nên rõ ràng và ít sai sót hơn.

❖ Swagger UI

Swagger UI là công cụ hiển thị tài liệu API ở dạng giao diện web tương tác dựa trên file OpenAPI, giúp chuyển nội dung YAML/JSON khô cứng thành danh sách endpoint có thể click, xem tham số, schema và ví dụ request/response rõ ràng. Swagger UI còn cho phép gửi thử request trực tiếp lên API, rất hữu ích cho dev, QA và đối tác khi thử nghiệm nhanh mà không cần tự viết script gọi API.

❖ Swagger Editor

Swagger Editor là trình soạn thảo giúp viết và chỉnh sửa file OpenAPI/Swagger ngay trên trình duyệt. Công cụ này hỗ trợ highlight cú pháp, gợi ý, validate theo chuẩn OAS và báo lỗi trực tiếp khi đặc tả sai cấu trúc. Nhờ đó team backend có thể thiết kế API theo hướng API-first rồi chia sẻ file OpenAPI cho frontend, mobile, QA và đối tác tích hợp.

❖ Swagger Codegen

Swagger Codegen là công cụ tự động tạo mã nguồn (SDK client, server stub và tài liệu) từ các tệp đặc tả OpenAPI (Swagger). Công cụ này giúp rút ngắn thời gian

phát triển, đảm bảo API và mã nguồn luôn đồng bộ, đồng thời hỗ trợ hơn 40 ngôn ngữ lập trình phổ biến như Java, Python, PHP, Nodejs,...

❖ **SwaggerHub**

SwaggerHub là nền tảng hợp tác trực tuyến cho đội ngũ phát triển API dựa trên chuẩn OpenAPI. Nó kết hợp Swagger Editor, Swagger UI, Codegen, quản lý phiên bản, phân quyền, comment và tích hợp với Git để mọi thay đổi file OpenAPI được kiểm soát tập trung. Nhờ một không gian làm việc chung, dev, QA, kỹ thuật viết tài liệu và bên liên quan có thể cùng chỉnh sửa, review và xuất bản tài liệu API một cách có kiểm soát.

❖ **Metadata**

Trong một file Swagger/OpenAPI, phần metadata thường nằm trong object info. Phần này mô tả thông tin chung về API như tên, mô tả ngắn, phiên bản, contact và license. Metadata giúp người dùng API hiểu nhanh họ đang làm việc với API nào, phiên bản nào và ai là đơn vị chịu trách nhiệm. Khi làm tài liệu cho nhiều môi trường hoặc nhiều service, metadata rõ ràng giúp tránh nhầm lẫn giữa các phiên bản và hệ thống.

❖ **Servers**

Phần servers dùng để khai báo một hoặc nhiều URL nơi API được triển khai, ví dụ môi trường production, staging, development. Mỗi server có thể gồm URL, tên hiển thị và mô tả, từ đó client biết phải gửi request tới đâu. OpenAPI cho phép dùng biến trong URL server để linh hoạt với subdomain, port hoặc basePath, phù hợp với các hệ thống nhiều vùng hoặc nhiều khách hàng.

Ví dụ rút gọn:

```
servers:
  - name: Production
    url: https://api.example.com/v1
  - name: Staging
    url: https://staging-api.example.com/v1
```

❖ **Paths**

Phần paths là nơi khai báo toàn bộ endpoint của API, mỗi key là một đường dẫn tương đối như /users hoặc /orders/{id}. Bên trong mỗi path, bạn định nghĩa các

method như GET, POST, PUT, DELETE cùng với tham số, request body, response và mã trạng thái HTTP. Paths là nơi nhóm dev và client tập trung thời gian nhiều nhất, vì paths mô tả trực tiếp API có những chức năng gì và cách gọi chi tiết ra sao.

Ví dụ:

```
paths:
  /users:
    get:
      summary: List users
      responses:
        '200':
          description: OK
```

❖ Schema

Phần schema thường nằm dưới components.schemas và mô tả cấu trúc dữ liệu dùng cho request và response. Mỗi schema định nghĩa kiểu dữ liệu, thuộc tính, ràng buộc như required, format, enum giúp client và server thống nhất cấu trúc mang đi qua API. Khi tái sử dụng cùng một schema cho nhiều endpoint, file Swagger gọn hơn, dễ bảo trì và giảm nguy cơ sai lệch dữ liệu giữa các phần của hệ thống.

Ví dụ:

```
Copy
components:
  schemas:
    User:
      type: object
      properties:
        id:
          type: integer
        username:
          type: string
        email:
          type: string
```

2.11.2 Lợi ích của Swagger với nhà phát triển API

Swagger mang lại nhiều lợi ích cho nhà phát triển API trong suốt vòng đời thiết kế, triển khai và bảo trì như:

Tiêu chuẩn hóa cách mô tả API: Swagger dựa trên OpenAPI Specification, giúp toàn bộ team dùng chung một định dạng để mô tả endpoint, tham số, dữ liệu và mã phản hồi. Nhờ đó frontend, backend, QA và đối tác có cùng “ngôn ngữ” khi trao đổi về API.

Rút ngắn thời gian viết tài liệu: Tài liệu được sinh tự động từ file OpenAPI thay vì viết tay trên wiki hoặc file riêng lẻ. Khi cập nhật API, bạn chỉ cần chỉnh file đặc tả, Swagger UI sẽ hiển thị tài liệu mới mà không phải chỉnh sửa ở nhiều nơi.

Hỗ trợ thiết kế API-first: Nhà phát triển có thể bắt đầu từ file đặc tả, review với team, chốt hợp đồng API rồi mới triển khai code. Cách làm này giúp phát hiện lỗi thiết kế sớm và cho phép các team phát triển song song dựa trên một bản mô tả thống nhất.

Tăng tốc kiểm thử và debug: Swagger UI cung cấp giao diện web để gửi request thử, xem request/response chi tiết và các mã lỗi mà không cần viết script hoặc dùng tool khác. Dev và QA có thể test nhanh theo từng endpoint, phát hiện lỗi mapping dữ liệu hoặc status code ngay trong giai đoạn phát triển.

Giảm lỗi khi tích hợp đa ngôn ngữ: Với Swagger Codegen hoặc các generator tương tự, bạn sinh được client SDK và server stub cho nhiều ngôn ngữ dựa trên một file OpenAPI. Việc này giảm sai lệch giữa tài liệu và code, đồng thời giúp team sử dụng nhiều stack khác nhau vẫn bám đúng hợp đồng API.

Cải thiện cộng tác nội bộ và với đối tác: Tài liệu API dạng tương tác giúp cả team dev, QA, technical writer và đối tác bên ngoài hiểu chính xác API đang cung cấp những gì. Việc on-board dev mới, mở API cho partner hoặc khách hàng bên ngoài cũng thuận tiện hơn vì chỉ cần chia sẻ một đường dẫn tài liệu Swagger.

CHƯƠNG 3. HIỆN THỰC HÓA NGHIÊN CỨU

3.1. Mô tả bài toán

3.1.1 Bối cảnh thực tế

Hiện nay, công nghệ thông tin đang được ứng dụng rộng rãi trong lĩnh vực giáo dục nhằm hỗ trợ công tác quản lý và nâng cao hiệu quả hoạt động của các cơ sở đào tạo. Tại các trường đại học, số lượng sinh viên ngày càng tăng, kéo theo khối lượng dữ liệu cần quản lý cũng ngày càng lớn. Các thông tin như hồ sơ sinh viên, kết quả học tập, điểm rèn luyện, thông tin lớp học và giảng viên cố vấn cần được lưu trữ và quản lý một cách khoa học.

Tuy nhiên, nhiều hệ thống quản lý hiện nay vẫn còn tồn tại một số hạn chế như khả năng mở rộng chưa cao, việc phân quyền người dùng chưa chặt chẽ và chưa có nhiều cơ chế bảo vệ dữ liệu quan trọng. Trong khi đó, các thông tin của sinh viên như số CCCD, địa chỉ, số điện thoại, email hay kết quả học tập đều là những dữ liệu cần được bảo mật.

Bên cạnh đó, cùng với sự phát triển của các ứng dụng web, các hệ thống trực tuyến thường xuyên phải đối mặt với những nguy cơ như truy cập trái phép, dò tìm mật khẩu, gửi yêu cầu quá mức cho phép hoặc các hình thức tấn công làm ảnh hưởng đến hoạt động của hệ thống. Vì vậy, việc xây dựng một hệ thống quản lý sinh viên có tính bảo mật cao, dễ mở rộng và đáp ứng nhu cầu sử dụng thực tế là cần thiết.

Từ những lý do trên, đề tài lựa chọn xây dựng hệ thống quản lý sinh viên theo kiến trúc Microservice sử dụng Spring Boot, kết hợp các cơ chế bảo mật như JWT, BCrypt, AES-GCM, Redis Rate Limiting và API Gateway nhằm nâng cao khả năng bảo vệ hệ thống cũng như tối ưu việc quản lý người dùng.

3.1.2 Các vấn đề cần giải quyết

Quản lý thông tin phân tán: Thông tin sinh viên, giảng viên và lớp học thường được lưu trữ ở nhiều nơi khác nhau, gây khó khăn cho việc tra cứu và cập nhật.

Bảo mật thông tin: Dữ liệu cá nhân và học tập của sinh viên cần được bảo vệ khỏi các cuộc tấn công như brute-force, SQL injection, và truy cập trái phép.

Phân quyền truy cập: Cần phân biệt rõ ràng quyền truy cập giữa Admin, Giảng viên và Sinh viên để đảm bảo tính bảo mật.

Khả năng mở rộng: Hệ thống cần đủ linh hoạt để thêm các chức năng mới mà không ảnh hưởng đến các module hiện tại.

Hiệu suất xử lý: Với số lượng người dùng lớn, hệ thống cần xử lý nhiều request đồng thời mà vẫn đảm bảo thời gian phản hồi nhanh.

Giám sát và audit: Cần theo dõi các hoạt động của người dùng để phát hiện và ngăn chặn các hành vi đáng ngờ.

3.1.3 Mục tiêu của hệ thống

Xây dựng hệ thống quản lý tập trung: Tích hợp quản lý sinh viên, giảng viên, lớp học và điểm số trong một nền tảng thống nhất.

Đảm bảo bảo mật cao: Áp dụng các cơ chế bảo mật hiện đại như JWT RS256, BCrypt, Rate Limiting, IP Blacklist và Account Locking.

Tối ưu hiệu suất: Sử dụng Redis để cache dữ liệu và xử lý rate limiting, giảm tải cho database.

Dễ dàng mở rộng: Áp dụng kiến trúc Microservices để dễ dàng thêm các service mới mà không làm gián đoạn hệ thống hiện tại.

Truy vết và giám sát: Ghi log chi tiết các hoạt động bảo mật để có thể phân tích và xử lý sự cố.

Giao diện thân thiện: Cung cấp API RESTful chuẩn để dễ dàng tích hợp với frontend và các hệ thống khác.

3.1.4 Yêu cầu chức năng

Quản lý tài khoản và xác thực:

- Đăng ký tài khoản mới với vai trò Student, Teacher hoặc Admin.
- Đăng nhập với username hoặc email.
- Đổi mật khẩu cho người dùng đã xác thực.
- Refresh token để duy trì phiên đăng nhập.
- Đăng xuất và thu hồi refresh token.
- Khóa tài khoản tự động khi nhập sai mật khẩu nhiều lần.
- Admin có thể mở khóa tài khoản và IP bị chặn.

Quản lý sinh viên:

- Thêm, sửa, xóa thông tin sinh viên.
- Tìm kiếm sinh viên theo tên, mã sinh viên, email.

- Lọc sinh viên theo trạng thái (đang học, nghỉ học, tốt nghiệp, bị đình chỉ).
- Upload và cập nhật ảnh đại diện.
- Xem danh sách lớp học mà sinh viên đang đăng ký.
- Xem điểm số và kết quả học tập.

Quản lý giảng viên:

- Thêm, sửa, xóa thông tin giảng viên.
- Quản lý thông tin khoa, chuyên môn, phòng làm việc.
- Lọc giảng viên theo khoa.
- Xem danh sách lớp học mà giảng viên đang giảng dạy.
- Phân công cố vấn học tập cho sinh viên.

Quản lý lớp học:

- Tạo, cập nhật, xóa lớp học.
- Phân công giảng viên cho lớp học.
- Quản lý sĩ số (số sinh viên tối đa, số sinh viên hiện tại).
- Đăng ký/bỏ lớp học cho sinh viên.
- Xem danh sách sinh viên trong lớp.
- Cập nhật điểm, tỷ lệ điểm danh cho sinh viên.

Giám sát bảo mật:

- Ghi log tất cả các hoạt động đăng nhập, đăng xuất.
- Theo dõi số lần đăng nhập thất bại.
- Tự động khóa tài khoản và chặn IP khi vượt ngưỡng.
- Xem lịch sử security log theo người dùng, IP, thời gian.

3.1.5 Yêu cầu phi chức năng

Bảo mật:

- Mã hóa mật khẩu: Sử dụng BCrypt với strength 12 để hash password.
- JWT RS256: Sử dụng RSA 2048-bit key pair để sign và verify token.
- Rate Limiting: Giới hạn 100 requests/giây/IP để chống DDoS.
- Brute-force Protection: Khóa tài khoản theo cơ chế progressive (5, 10, 15, 20 lần sai).
- IP Blacklist: Chặn vĩnh viễn IP sau 20 lần đăng nhập sai.
- Audit Logging: Ghi log đầy đủ các hoạt động bảo mật.

Hiệu suất:

- Thời gian phản hồi: API response time < 200ms cho 95% requests.
- Throughput: Hỗ trợ ít nhất 1000 concurrent users.
- Caching: Sử dụng Redis để cache các dữ liệu thường xuyên truy cập.
- Database Indexing: Đánh index các trường thường xuyên query.

Khả năng mở rộng:

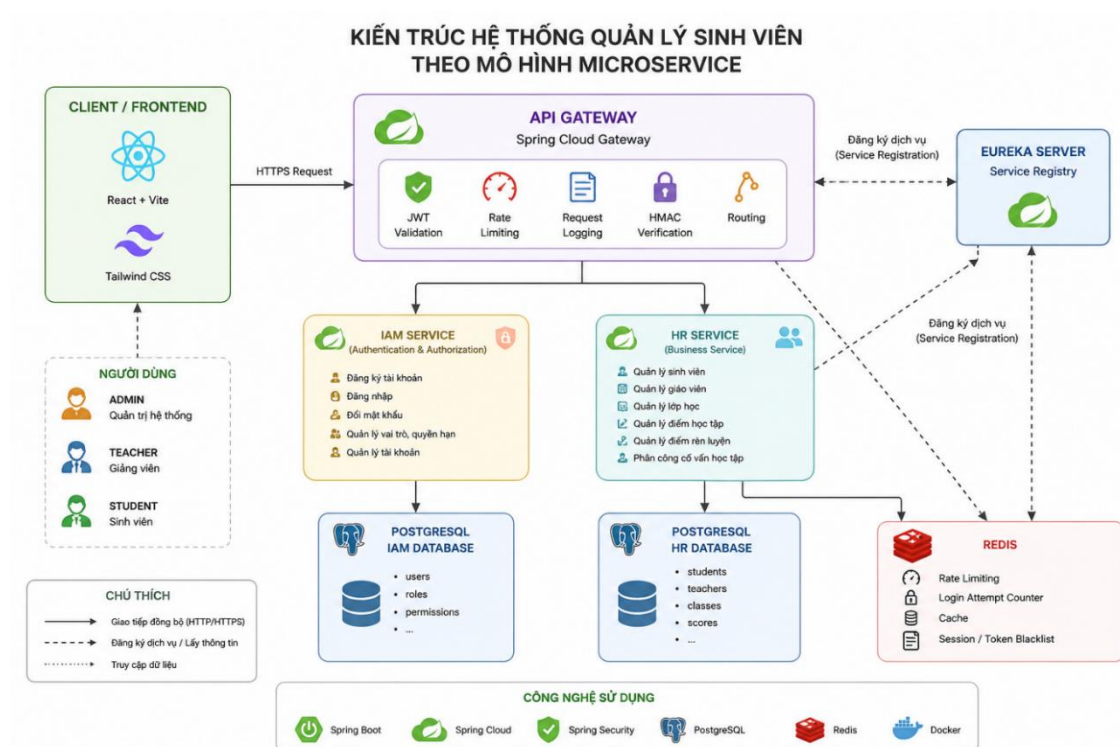
- Microservices: Mỗi service độc lập, có thể scale riêng biệt.
- Service Discovery: Sử dụng Eureka để tự động phát hiện service instances.
- Load Balancing: Spring Cloud LoadBalancer cho việc phân phối request.
- Containerization: Docker containers để dễ dàng deploy.

Tính sẵn sàng:

- Uptime: Đảm bảo 99.5% uptime (cho phép downtime tối đa 3.65 giờ/tháng).
- Health Check: Mỗi service có endpoint health check.
- Graceful Degradation: Hệ thống tiếp tục hoạt động khi một service gặp sự cố.

3.2. Kiến Trúc Hệ Thống

3.2.1 Mô hình kiến trúc Microservice



HÌNH 3.1: Kiến trúc hệ thống quản lý sinh viên

3.2.2 Các thành phần trong hệ thống

3.2.2.1 Eureka Server

Port: 8761

Vai trò: Service Registry và Discovery cho toàn bộ hệ thống.

Công nghệ: Spring Cloud Netflix Eureka.

Chức năng chính:

- Đăng ký tất cả các microservice instances.
- Cung cấp thông tin về các service đang hoạt động.
- Health check định kỳ để phát hiện service down.
- Hỗ trợ load balancing thông qua service discovery.

Cấu hình:

```
eureka: instance: hostname: localhost client: register-with-eureka: false fetch-registry: false server: enable-self-preservation: false eviction-interval-timer-in-ms: 30000
```

3.2.2.2 API Gateway

Port: 8080

Vai trò: Single entry point cho tất cả client requests.

Công nghệ: Spring Cloud Gateway (Reactive).

Chức năng chính:

- Routing: Định tuyến request đến service phù hợp dựa trên URL path.
- Authentication: Xác thực JWT token cho mọi request (trừ public endpoints).
- Rate Limiting: Giới hạn 100 requests/giây/IP.
- IP Blacklist: Chặn các IP bị blacklist.
- CORS: Xử lý Cross-Origin Resource Sharing.
- Load Balancing: Phân phối request đến các service instances.

Filter Chain (theo thứ tự ưu tiên):

- IpBlacklistFilter (Order: -1): Kiểm tra IP có bị chặn không.
- RateLimitFilter: Kiểm tra rate limit.
- JwtAuthenticationFilter: Xác thực JWT và thêm user info vào headers.

Public Endpoints (không cần JWT):

/api/auth/** - Authentication endpoints.

/actuator/** - Health check endpoints.

/eureka/** - Eureka dashboard.

3.2.2.3 IAM Service

Port: 8081

Vai trò: Quản lý xác thực, phân quyền và bảo mật.

Công nghệ: Spring Boot 3.5, Spring Security 6.

Chức năng chính:

Authentication:

- Đăng ký tài khoản mới (register).
- Đăng nhập (login) với username hoặc email.
- Refresh access token.
- Đăng xuất và thu hồi refresh token.

JWT Management:

- Generate access token (TTL: 1 giờ).
- Generate refresh token (TTL: 7 ngày).
- Sử dụng RS256 với RSA 2048-bit key pair.

Security Features:

- BCrypt password hashing (strength 12).
- Progressive account locking (5, 10, 15, 20 lần sai).
- IP blacklist management.
- Failed login tracking (Redis + Database).

Audit Logging:

- Ghi log mọi hoạt động bảo mật.
- Lưu trữ IP, user agent, timestamp.
- Hỗ trợ truy vết và phân tích.

Database Tables:

- `nguoi_dung` (users).
- `token_lam_moi` (refresh_tokens).
- `nhat_ky_bao_mat` (security_logs).
- `danh_sach_den_ip` (ip_blacklist).

3.2.2.4 HR Service

Port: 8082

Vai trò: Quản lý sinh viên, giảng viên, lớp học.

Công nghệ: Spring Boot 3.5, Spring Data JPA.

Chức năng chính:

Student Management:

- CRUD operations cho sinh viên.
- Tìm kiếm theo tên, mã sinh viên, email.
- Lọc theo trạng thái (đang học, tốt nghiệp, v.v.).
- Upload avatar (max 5MB).
- Thống kê số lượng theo trạng thái.

Teacher Management:

- CRUD operations cho giảng viên.
- Quản lý khoa, chuyên môn, phòng làm việc.
- Lọc theo khoa.
- Thống kê số lượng theo khoa.

Class Management:

- CRUD operations cho lớp học.
- Quản lý sĩ số (max students, current students).
- Phân công giảng viên.
- Lọc theo giảng viên, năm học, học kỳ.

Enrollment Management:

- Đăng ký lớp học cho sinh viên.
- Bỏ lớp học.
- Cập nhật điểm, tỷ lệ điểm danh.
- Xem danh sách sinh viên trong lớp.
- Xem danh sách lớp của sinh viên.

Database Tables:

- sinh_vien (students)
- giang_vien (teachers)
- lop_hoc (classes)

- `dang_ky_lop_hoc` (`class_enrollments`)

3.2.2.5 Redis

Port: 6379

Vai trò: In-memory data store cho caching và session management.

Công nghệ: Redis 7.2 Alpine.

Chức năng chính:

Rate Limiting:

- Key pattern: `rate_limit:{ip}`.
- TTL: 1 giây.
- Value: counter.

IP Blacklist:

- Key pattern: `ip_blacklist:{ip}`.
- TTL: permanent.

Account Locking:

- Key pattern: `account_lock:{username}`.
- TTL: lock duration (30s, 1m, 1h)

Failed Login Tracking:

- Key pattern: `failed_login:user:{username}`.
- Key pattern: `failed_login:ip:{ip}`.
- Value: counter.

3.2.2.6 PostgreSQL

Port: 5432

Vai trò: Relational database chính của hệ thống.

Công nghệ: PostgreSQL 16 Alpine.

Database: `student_management_db`

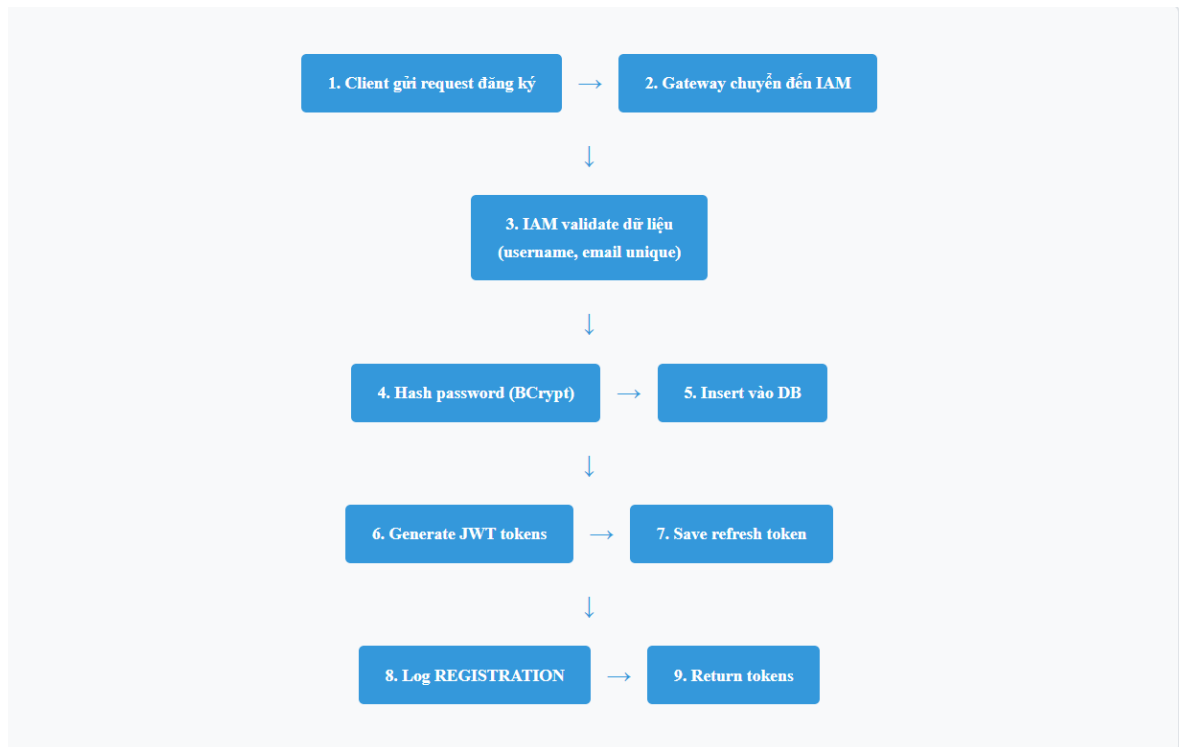
Đặc điểm:

- Shared Database: IAM và HR services dùng chung 1 database.
- Foreign Keys: Các bảng sinh viên, giảng viên có FK đến bảng users.
- Indexing: Index các trường thường xuyên query (`username`, `email`, `student_code`, v.v.).
- Migration: Sử dụng Flyway để quản lý database schema version.

- UUID Primary Keys: Tất cả bảng sử dụng UUID thay vì auto-increment integer.
- Audit Columns: Các bảng có created_at, updated_at, created_by, updated_by.

3.2.3 Luồng xử lý của hệ thống

3.2.3.1 Luồng đăng ký tài khoản

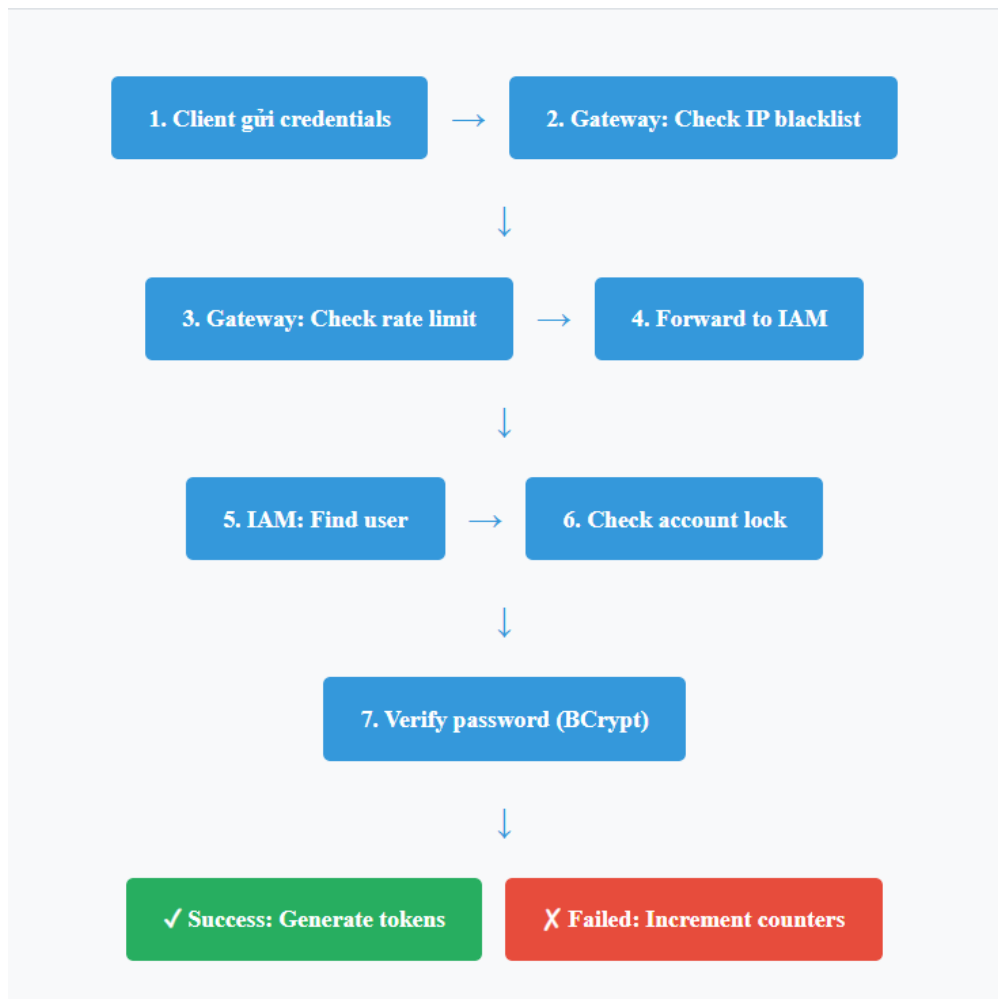


HÌNH 3.2: Luồng đăng ký tài khoản

```

POST /api/auth/register Request: { "username": "nguyenvana",
"email": "nguyenvana@example.com", "password": "password123",
"role": "STUDENT" } Response: { "success": true, "data":
{ "accessToken": "eyJhbGciOiJIUzI1NiIs... ", "refreshToken":
"550e8400-e29b-41d4... ", "tokenType": "Bearer", "expiresIn":
3600, "user": { "id": "uuid", "username": "nguyenvana",
"email": "nguyenvana@example.com", "role": "STUDENT" } } }
  
```

3.2.3.2 Luồng đăng nhập

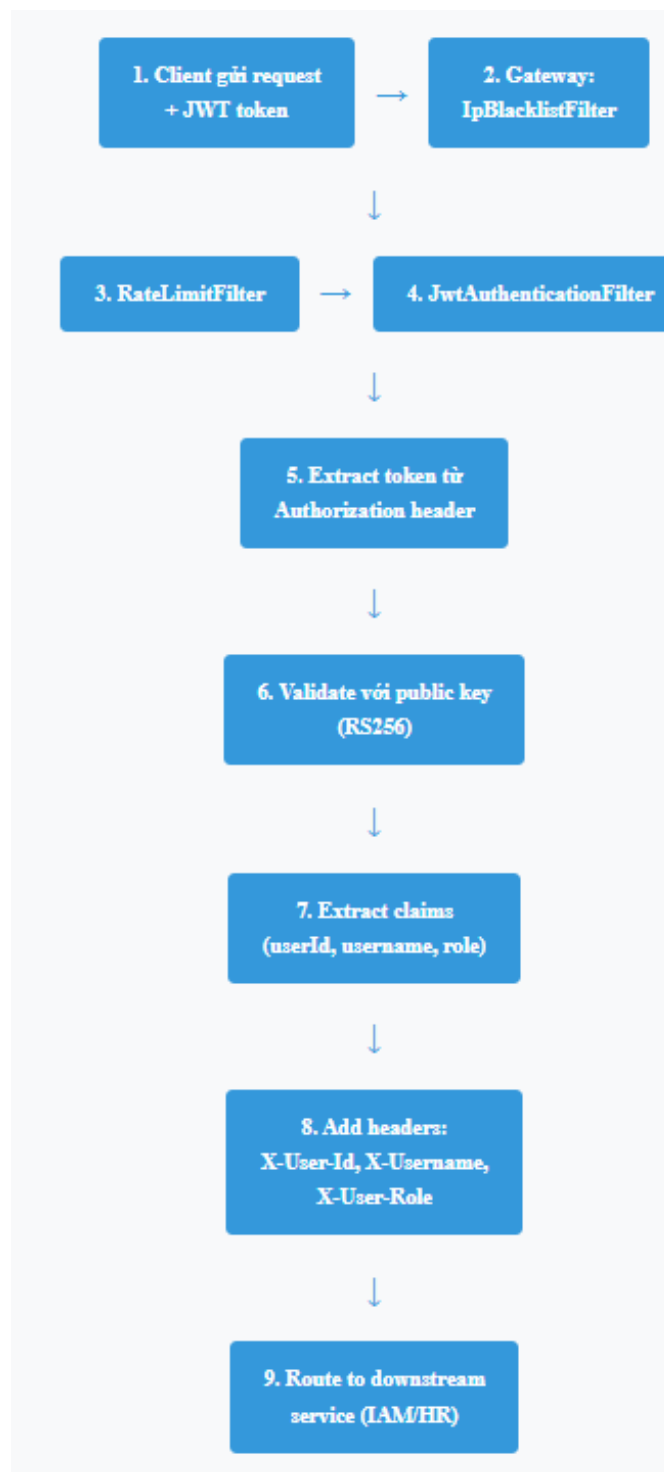


HÌNH 3.3: Luồng đăng nhập

Progressive Locking:

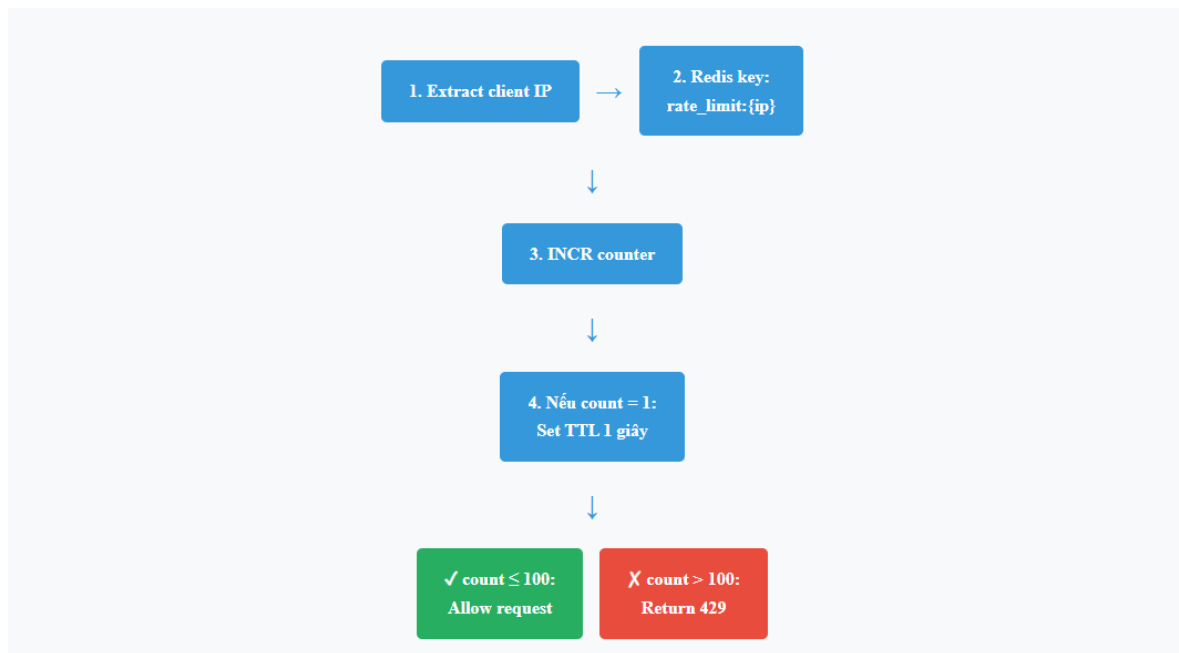
- 5 lần sai → Khóa 30 giây
- 10 lần sai → Khóa 1 phút
- 15 lần sai → Khóa 1 giờ
- 20 lần sai → Khóa vĩnh viễn + Chặn IP vĩnh viễn

3.2.3.3 Luồng xác thực JWT



HÌNH 3.4: Luồng xác thực JWT

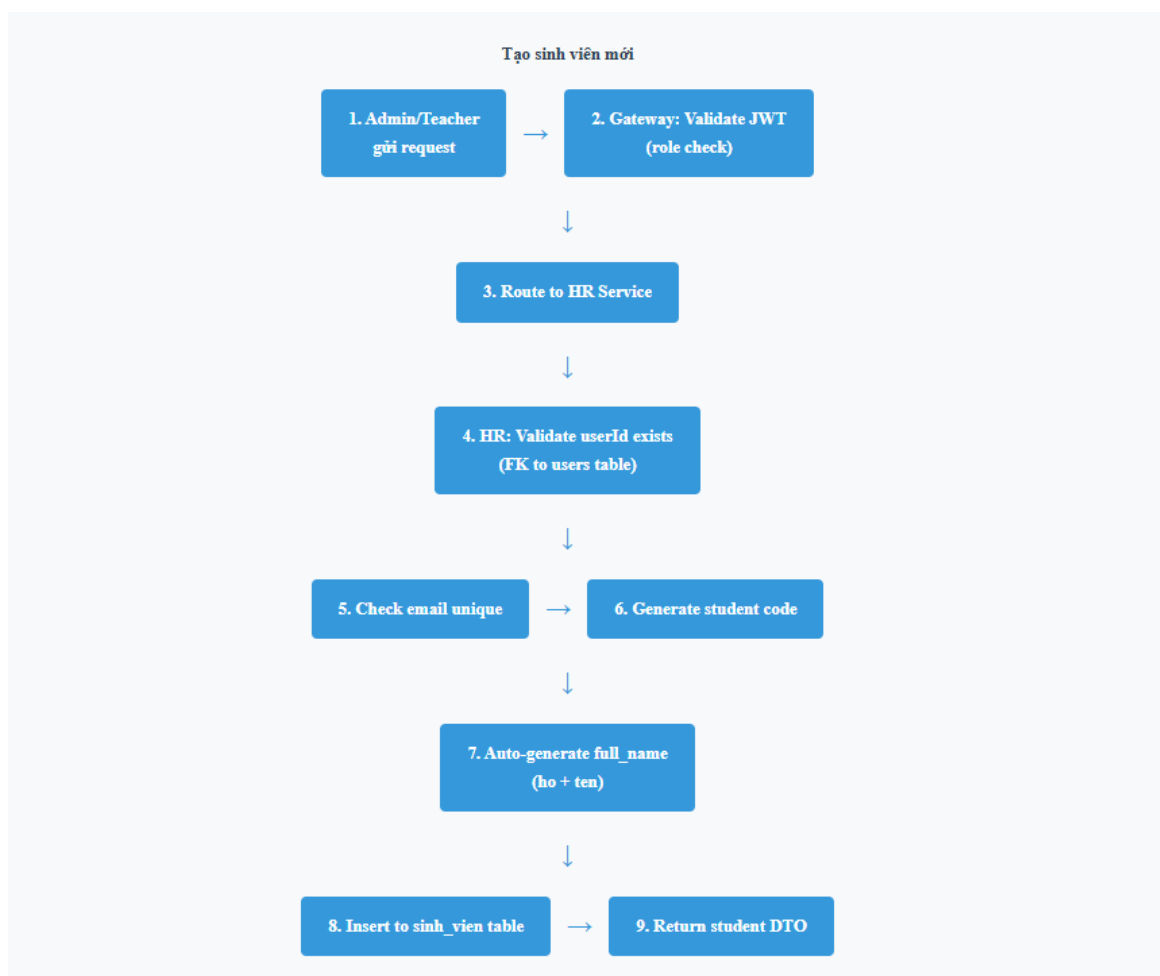
3.2.3.4 Luồng Rate Limiting



HÌNH 3.5: Luồng Rate Limiting

Implementation: Redis INCR với TTL 1 giây, giới hạn 100 requests/giây/IP

3.2.3.5 Luồng quản lý sinh viên



HÌNH 3.6: Luồng quản lý sinh viên

3.2.4 Kiến trúc bảo mật

Hệ thống áp dụng nhiều lớp bảo mật (defense in depth) để đảm bảo an toàn thông tin:

Layer 1: Network Security

- IP Blacklist: Chặn IP đáng ngờ.
- Rate Limiting: Chống DDoS.
- CORS: Kiểm soát cross-origin requests.

Layer 2: Authentication & Authorization

- JWT RS256: Token-based authentication.
- Role-based: Access Admin/Teacher/Student.
- Token Expiry: Access token: 1h, Refresh: 7d.

Layer 3: Data Protection

- Bcrypt: Password hashing (strength 12).
- HTTPS: Encrypted communication.
- SHA-256: Refresh token hashing.

Layer 4: Attack Prevention

- Brute-force Protection: Progressive account locking.
- SQL Injection: Prepared statements (JPA).
- XSS Protection: Input validation.

Layer 5: Monitoring & Audit

- Security Logs: Comprehensive audit trail.
- Failed Login Tracking: Redis + Database.
- Health Monitoring: Actuator endpoints.

3.2.4.1 JWT RS256

Hệ thống sử dụng JWT (JSON Web Token) với thuật toán RS256 (RSA Signature with SHA-256) để xác thực người dùng.

Ưu điểm của RS256:

Asymmetric encryption: Private key để sign, public key để verify.

Bảo mật cao: Private key chỉ IAM service có, các service khác chỉ có public key.

Khó giả mạo: Cần có private key mới có thể tạo token hợp lệ.

Scalability: Gateway có thể verify token độc lập, không cần gọi IAM.

Token Structure:

```
// Header { "alg": "RS256", "typ": "JWT" } // Payload { "userId":  
"550e8400-e29b-41d4-a716-446655440000", "sub": "nguyenvana",  
"role": "STUDENT", "iss": "enterprise-iam", "iat": 1640000000,  
"exp": 1640003600 } // Signature  
RSASHA256( base64UrlEncode(header) + "." +  
base64UrlEncode(payload), privateKey )
```

3.2.4.2 Bcrypt

BCrypt là thuật toán hash password được thiết kế đặc biệt để chống brute-force attack.

Đặc điểm:

- Adaptive hashing: Có thể tăng cost factor theo thời gian.
- Salt tự động: Mỗi password có salt riêng, ngăn rainbow table attack.
- Slow by design: Hashing chậm giúp làm chậm brute-force attack.
- Strength 12: $2^{12} = 4096$ rounds, cân bằng giữa bảo mật và hiệu suất.

Implementation:

```
// Hash password khi register String passwordHash =  
passwordEncoder.encode(request.getPassword()); // Output:  
$2a$12$N9qo8uLOickgx2ZMRZoMyeIjZAgcfl7p92ldGxad68LJZdL17lhWy //  
Verify password khi login boolean matches =  
passwordEncoder.matches( request.getPassword(),  
user.getPasswordHash() );
```

3.2.4.3 HMAC Signature

Hiện tại hệ thống sử dụng SHA-256 để hash refresh token trước khi lưu database.

```
// Hash refresh token public String hash(String token)  
{ MessageDigest digest = MessageDigest.getInstance("SHA-256");  
byte[] hash =  
digest.digest(token.getBytes(StandardCharsets.UTF_8)); return  
bytesToHex(hash); }
```

3.2.4.4 Rate Limiting

Rate Limiting giới hạn số lượng requests mà một IP có thể gửi trong một khoảng thời gian, giúp chống DDoS và abuse.

Cấu hình:

- Limit: 100 requests/giây/IP.
- Storage: Redis với TTL 1 giây.
- Algorithm: Token bucket (simplified).
- Response: 429 Too Many Requests.

Flow:

- Extract client IP từ request headers (hỗ trợ X-Forwarded-For).
- Redis key: rate_limit:{ip}.
- INCR counter, set TTL 1s nếu lần đầu.

- Nếu counter > 100 → reject và log vào security_logs.

3.2.4.5 Brute-force Protection

Số lần sai	Hành động	Thời gian khóa	Thông báo
1-4	Cảnh báo	-	Còn X lần nữa sẽ bị khóa 30 giây
5	Khóa Level 1	30 giây	Tài khoản đã bị khóa 30 giây
6-9	Cảnh báo	-	Còn X lần nữa sẽ bị khóa 1 phút
10	Khóa Level 2	1 phút	Tài khoản đã bị khóa 1 phút
11-14	Cảnh báo	-	Còn X lần nữa sẽ bị khóa 1 giờ
15	Khóa Level 3	1 giờ	Tài khoản đã bị khóa 1 giờ
16-19	Cảnh báo nghiêm trọng	-	Còn X lần nữa IP sẽ bị chặn vĩnh viễn
20	Khóa vĩnh viễn + Chặn IP	Permanent	Tài khoản và IP bị chặn vĩnh viễn

HÌNH 3.7: Mức độ khóa tăng dần theo số lần đăng nhập sai

3.2.4.6 Account Locking

Storage: Redis (real-time) + Database (persistent).

Redis:

- Key: account_lock: {username}
- TTL: Lock duration (30s, 60s, 3600s)
- Auto-expire khi hết thời gian khóa.

Database:

- Field: bi_khoa (boolean)
- Field: khoa_den_luc (timestamp)
- Field: so_lan_dang_nhap_that_bai (integer)
- Sử dụng để recovery khi Redis restart

3.3. Phân tích tác nhân và Use Case

3.3.1 Tác nhân của hệ thống

BẢNG 3.1: Tác nhân của hệ thống

Tác nhân	Mô tả và quyền hạn
Admin	Vai trò: Quản trị viên hệ thống Quyền hạn: - Quản lý toàn bộ sinh viên, giảng viên, lớp học - Tạo, sửa, xóa tài khoản

	- Mở khóa tài khoản bị khóa - Gỡ chặn IP từ blacklist - Xem security logs và audit trail - Quản lý phân quyền - Xem thống kê, báo cáo toàn hệ thống
Teacher	Vai trò: Giảng viên Quyền hạn: - Xem danh sách sinh viên thuộc các lớp mình giảng dạy - Cập nhật điểm học tập cho sinh viên - Cập nhật tỷ lệ điểm danh - Xem báo cáo lớp học - Quản lý sinh viên được phân công cố vấn - Xem và cập nhật thông tin cá nhân
Student	Vai trò: Sinh viên Quyền hạn: - Đăng ký và đăng nhập tài khoản - Đổi mật khẩu - Xem thông tin cá nhân - Cập nhật ảnh đại diện - Xem danh sách lớp học đã đăng ký - Xem điểm số và kết quả học tập - Xem thông tin cố vấn học tập

3.3.2 Danh sách Use Case

BẢNG 3.2: Use Case Admin

Use Case ID	Tên Use Case	Mô tả ngắn
UC-A01	Quản lý sinh viên	Thêm, sửa, xóa, tìm kiếm thông tin sinh viên
UC-A02	Quản lý giảng viên	Thêm, sửa, xóa, tìm kiếm thông tin giảng viên
UC-A03	Quản lý lớp học	Tạo lớp, phân công giảng viên, quản lý sĩ số
UC-A04	Phân công cố vấn học tập	Gán giảng viên làm cố vấn cho sinh viên
UC-A05	Mở khóa tài khoản	Mở khóa tài khoản người dùng bị khóa
UC-A06	Gỡ chặn IP	Xóa IP khỏi blacklist
UC-A07	Theo dõi thống kê	Xem báo cáo số lượng sinh viên, giảng viên, lớp học
UC-A08	Xem security logs	Theo dõi các hoạt động bảo mật, phát hiện bất thường

BẢNG 3.3: Use Case Teacher

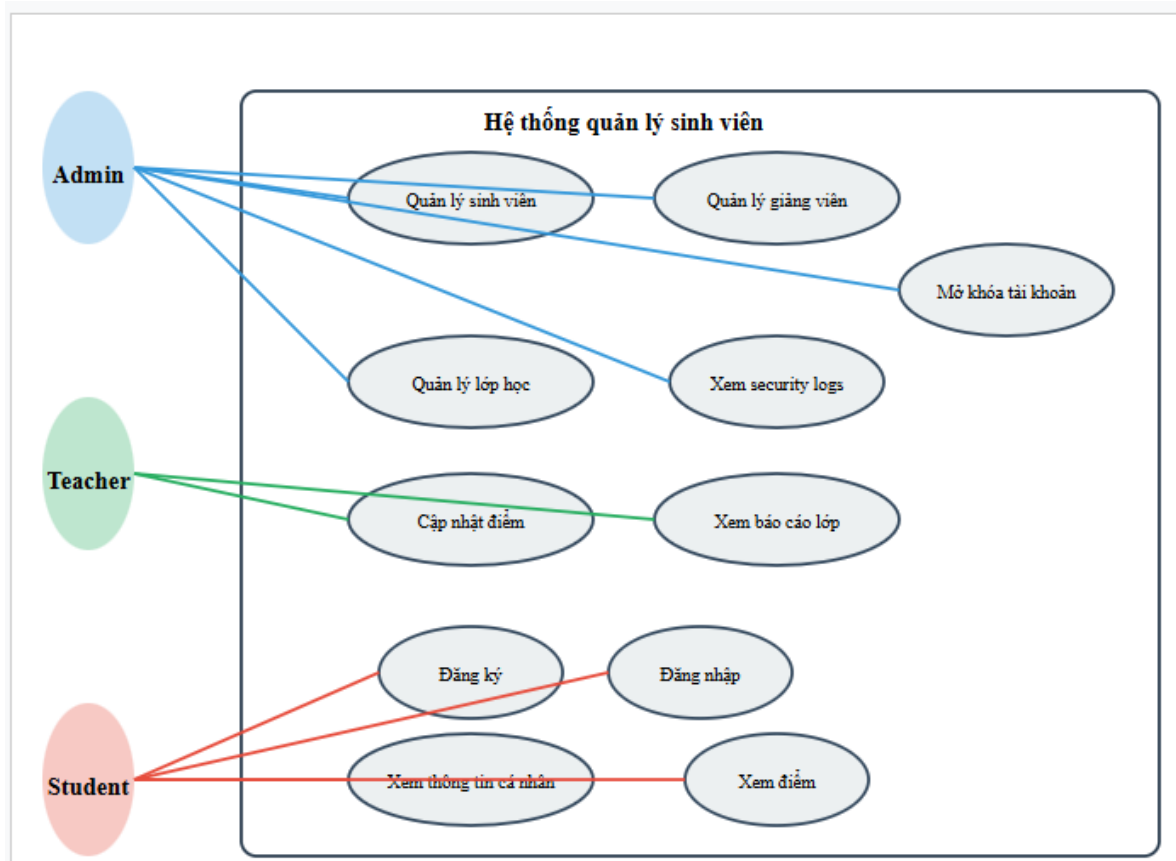
Use Case ID	Tên Use Case	Mô tả ngắn
UC-T01	Xem sinh viên thuộc quyền quản lý	Xem danh sách sinh viên trong các lớp mình dạy
UC-T02	Cập nhật điểm học tập	Nhập điểm cho sinh viên trong lớp
UC-T03	Cập nhật điểm rèn luyện	Đánh giá điểm rèn luyện và tỷ lệ điểm danh
UC-T04	Xem báo cáo lớp	Xem thống kê điểm số và kết quả học tập của lớp

BẢNG 3.4: Use Case Student

se Case ID	Tên Use Case	Mô tả ngắn
UC-S01	Đăng ký	Tạo tài khoản mới trong hệ thống
UC-S02	Đăng nhập	Xác thực và truy cập vào hệ thống
UC-S03	Đổi mật khẩu	Thay đổi mật khẩu tài khoản
UC-S04	Xem thông tin cá nhân	Xem và cập nhật profile
UC-S05	Xem điểm	Xem điểm các môn học và kết quả học tập

UC-S06	Xem cổ vấn học tập	Xem thông tin giảng viên cổ vấn
--------	--------------------	---------------------------------

3.3.3 Sơ đồ Use Case tổng quát



HÌNH 3.8: Use Case Diagram - Hệ thống quản lý sinh viên

3.4. Thiết kế chức năng hệ thống

3.4.1 Chức năng quản lý tài khoản

Đăng ký

Endpoint: POST /api/auth/register

Mô tả: Tạo tài khoản mới với vai trò Student, Teacher hoặc Admin

Request:

```
{ "username": "nguyenvana", "email": "nguyenvana@example.com",
"password": "Password@123", "role": "STUDENT" }
```

Validation:

- Username: 3-50 ký tự, chỉ chữ cái, số và underscore
- Email: Định dạng email hợp lệ, unique

- Password: Tối thiểu 8 ký tự
- Role: Phải là một trong STUDENT, TEACHER, ADMIN

Đăng nhập

Endpoint: POST /api/auth/login

Mô tả: Xác thực người dùng và cấp JWT tokens

Request:

```
{ "usernameOrEmail": "nguyenvana", "password": "Password@123" }
```

Security Checks:

1. Kiểm tra IP có bị blacklist không.
2. Kiểm tra rate limit (100 req/s).
3. Kiểm tra tài khoản có bị khóa không.
4. Verify password với Bcrypt.
5. Nếu sai: Increment failed attempts counter → Apply locking nếu vượt threshold.
6. Nếu đúng: Reset counters → Generate tokens.

Đổi mật khẩu

Endpoint: PUT /api/users/me/password

Mô tả: Thay đổi mật khẩu cho người dùng đã xác thực.

Authorization: Requires valid JWT token

Request:

```
{ "oldPassword": "Password@123", "newPassword": "NewPassword@456" }
```

Refresh Token

Endpoint: POST /api/auth/refresh

Mô tả: Làm mới access token khi hết hạn.

Request:

```
{ "refreshToken": "550e8400-e29b-41d4-a716-446655440000" }
```

Process:

1. Hash refresh token với SHA-256
2. Tìm trong database table token_lam_moi
3. Validate: not expired, not revoked
4. Generate new access token

5. Return new access token (keep same refresh token)

3.4.2 Chức năng phân quyền

BẢNG 3.5: Phân quyền

hức năng	Admin	Teacher	Student
Quản lý sinh viên (CRUD)	✓	✗	✗
Quản lý giảng viên (CRUD)	✓	✗	✗
Quản lý lớp học (CRUD)	✓	✗	✗
Xem sinh viên trong lớp	✓	✓ (lớp mình dạy)	✗
Cập nhật điểm	✓	✓ (lớp mình dạy)	✗
Xem điểm	✓	✓	✓ (điểm của mình)
Mở khóa tài khoản	✓	✗	✗
Gỡ chặn IP	✓	✗	✗
Xem security logs	✓	✗	✗

3.4.3 Chức năng quản lý sinh viên

Các API endpoints:

GET /api/students: Danh sách sinh viên (pagination, sort).

GET /api/students/{id}: Chi tiết sinh viên theo ID.

GET /api/students/code/{code}: Chi tiết theo mã sinh viên.

GET /api/students/search?name=xxx: Tìm kiếm theo tên.

GET /api/students/status/{status}: Lọc theo trạng thái.

POST /api/students: Tạo sinh viên mới.

PUT /api/students/{id}: Cập nhật thông tin.

DELETE /api/students/{id}: Xóa sinh viên.

POST /api/students/{id}/avatar: Upload ảnh đại diện.

Business Rules:

- Mỗi user chỉ có 1 student profile (FK userId unique).
- Email phải unique trong bảng sinh_vien.
- Mã sinh viên tự động generate hoặc nhập thủ công.
- Full name tự động generate từ họ + tên.
- Avatar max 5MB, allowed types: jpg, jpeg, png, gif.

3.5. Cơ sở dữ liệu

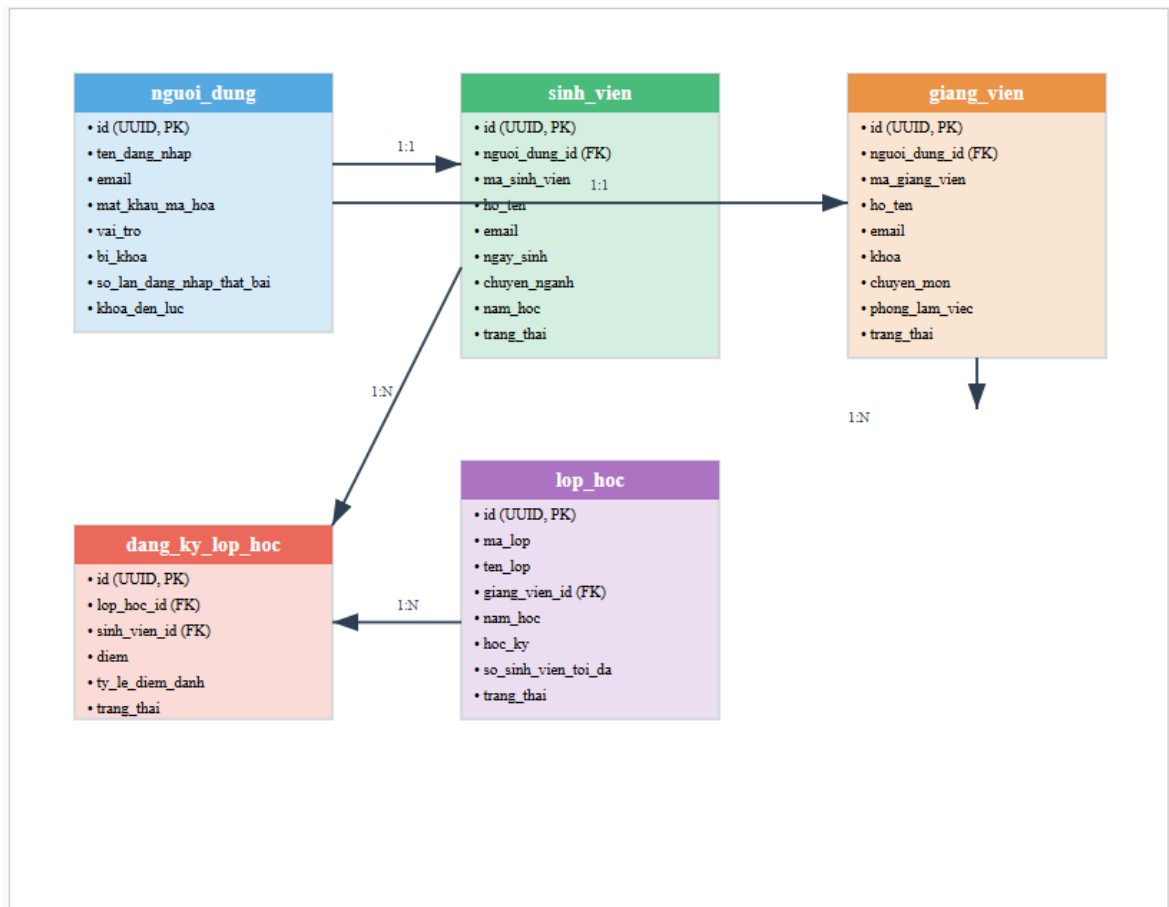
3.5.1 Phân tích dữ liệu

Hệ thống sử dụng PostgreSQL 16 làm cơ sở dữ liệu quan hệ chính. Database được thiết kế theo chuẩn Third Normal Form (3NF) để đảm bảo tính nhất quán và giảm thiểu dư thừa dữ liệu.

Đặc điểm thiết kế:

- UUID Primary Keys: Sử dụng UUID thay vì auto-increment integer để tránh enumeration attack.
- Indexes: Đánh index các trường thường xuyên query (username, email, student_code, v.v.).
- Foreign Keys: Ràng buộc referential integrity giữa các bảng.
- Audit Columns: Mọi bảng có created_at, updated_at, created_by, updated_by.
- Soft Delete: Sử dụng status field thay vì xóa vật lý.
- Tiếng Việt: Tất cả tên bảng và cột sử dụng tiếng Việt không dấu.

3.5.2 Mô hình ERD



HÌNH 3.9: Entity Relationship Diagram

Quan hệ chính:

- 1 nguoi_dung ↔ 1 sinh_vien (One-to-One)
- 1 nguoi_dung ↔ 1 giang_vien (One-to-One)
- 1 giang_vien → N lop_hoc (One-to-Many)
- 1 sinh_vien → N dang_ky_lop_hoc (One-to-Many)
- 1 lop_hoc → N dang_ky_lop_hoc (One-to-Many)

3.5.3 Danh sách bảng dữ liệu

Tên bảng	Mô tả
nguoi_dung	Lưu trữ thông tin tài khoản người dùng
token_lam_moi	Lưu trữ refresh tokens
nhat_ky_bao_mat	Ghi log các hoạt động bảo mật
danh_sach_den_ip	Danh sách IP bị chặn
sinh_vien	Thông tin chi tiết sinh viên
giang_vien	Thông tin chi tiết giảng viên
lop_hoc	Thông tin lớp học/môn học
dang_ky_lop_hoc	Đăng ký lớp học của sinh viên

HÌNH 3.10: Danh sách bảng dữ liệu

3.5.4 Thiết kế bảng nguoi_dung

Tên cột	Kiểu dữ liệu	Ràng buộc	Mô tả
id	UUID	PRIMARY KEY	ID duy nhất của người dùng
ten_dang_nhap	VARCHAR(50)	NOT NULL, UNIQUE	Tên đăng nhập
email	VARCHAR(100)	NOT NULL, UNIQUE	Địa chỉ email
mat_khau_ma_hoa	VARCHAR(255)	NOT NULL	Mật khẩu đã hash bằng BCrypt
vai_tro	VARCHAR(20)	NOT NULL	Vai trò: ADMIN, TEACHER, STUDENT
bi_khoa	BOOLEAN	NOT NULL, DEFAULT FALSE	Trạng thái khóa tài khoản
khoa_den_luc	TIMESTAMP	NULL	Thời gian hết khóa
kich_hoat	BOOLEAN	NOT NULL, DEFAULT TRUE	Tài khoản có kích hoạt không
so_lan_dang_nhap_that_bai	INTEGER	NOT NULL, DEFAULT 0	Số lần đăng nhập sai (tích lũy)
lan_dang_nhap_cuoi	TIMESTAMP	NULL	Thời gian đăng nhập gần nhất
ip_dang_nhap_cuoi	VARCHAR(45)	NULL	IP đăng nhập gần nhất
ngay_tao	TIMESTAMP	NOT NULL	Thời gian tạo tài khoản
ngay_cap_nhat	TIMESTAMP	NULL	Thời gian cập nhật cuối

HÌNH 3.11: Bảng thiết kế bảng nguoi_dung

3.5.5 Thiết kế bảng sinh_vien

Tên cột	Kiểu dữ liệu	Ràng buộc	Mô tả
id	UUID	PRIMARY KEY	ID duy nhất của sinh viên
nguoai_dung_id	UUID	NOT NULL, UNIQUE, FK	Foreign key đến bảng nguoai_dung
ma_sinh_vien	VARCHAR(20)	NOT NULL, UNIQUE	Mã sinh viên (student code)
ho	VARCHAR(50)	NOT NULL	Họ và tên đệm
ten	VARCHAR(50)	NOT NULL	Tên
ho_ten	VARCHAR(101)	NOT NULL	Họ tên đầy đủ (auto-generated)
email	VARCHAR(100)	NOT NULL, UNIQUE	Email sinh viên
ngay_sinh	DATE	NULL	Ngày sinh
gioi_tinh	VARCHAR(10)	NULL	Giới tính: NAM, NU, KHAC
so_dien_thoai	VARCHAR(20)	NULL	Số điện thoại
dia_chi	VARCHAR(255)	NULL	Địa chỉ
anh_dai_dien	VARCHAR(500)	NULL	URL ảnh đại diện
ngay_nhap_hoc	DATE	NULL	Ngày nhập học
chuyen_nganh	VARCHAR(100)	NULL	Chuyên ngành
nam_hoc	VARCHAR(20)	NULL	Năm học (vd: 2023-2024)
trang_thai	VARCHAR(20)	NOT NULL	HOAT_DONG, KHONG_HOAT_DONG, DA_TOT_NGHIEP, BI_DINH_CHI

HÌNH 3.12: Bảng thiết kế bảng sinh_vien

3.5.6 Thiết kế bảng giang_vien

Tên cột	Kiểu dữ liệu	Ràng buộc	Mô tả
id	UUID	PRIMARY KEY	ID duy nhất của giảng viên
nguoai_dung_id	UUID	NOT NULL, UNIQUE, FK	Foreign key đến bảng nguoai_dung
ma_giang_vien	VARCHAR(20)	NOT NULL, UNIQUE	Mã giảng viên
ho_ten	VARCHAR(101)	NOT NULL	Họ tên đầy đủ
email	VARCHAR(100)	NOT NULL, UNIQUE	Email giảng viên
khoa	VARCHAR(100)	NULL	Khoa (department)
chuyen_mon	VARCHAR(100)	NULL	Chuyên môn (specialization)
ngay_tuyen_dung	DATE	NULL	Ngày tuyển dụng
phong_lam_viec	VARCHAR(100)	NULL	Phòng làm việc
trang_thai	VARCHAR(20)	NOT NULL	HOAT_DONG, KHONG_HOAT_DONG, NGHI_PHEP, DA_NGHI_HUU

HÌNH 3.13: Bảng thiết kế bảng giang_vien

3.5.7 Thiết kế bảng lop_hoc

Tên cột	Kiểu dữ liệu	Ràng buộc	Mô tả
id	UUID	PRIMARY KEY	ID duy nhất của lớp học
ma_lop	VARCHAR(20)	NOT NULL, UNIQUE	Mã lớp học
ten_lop	VARCHAR(100)	NOT NULL	Tên lớp học
giang_vien_id	UUID	NULL, FK	Foreign key đến bảng giang_vien
nam_hoc	VARCHAR(20)	NOT NULL	Năm học (vd: 2023-2024)
hoc_ky	INTEGER	NOT NULL	Học kỳ (1, 2, 3)
mon_hoc	VARCHAR(100)	NULL	Tên môn học
phong_hoc	VARCHAR(50)	NULL	Phòng học
lich_hoc	VARCHAR(255)	NULL	Lịch học (vd: Thứ 2, 7h-9h)
so_sinh_vien_toi_da	INTEGER	NULL	Sĩ số tối đa
so_sinh_vien_hien_tai	INTEGER	DEFAULT 0	Số sinh viên hiện tại
ngay_bat_dau	DATE	NULL	Ngày bắt đầu
ngay_ket_thuc	DATE	NULL	Ngày kết thúc
mo_ta	VARCHAR(500)	NULL	Mô tả lớp học
trang_thai	VARCHAR(20)	NOT NULL	HOAT_DONG, KHONG_HOAT_DONG, DA_HOAN_THANH, DA_HUY

HÌNH 3.14: Bảng thiết kế bảnglop_hoc

3.5.8 Thiết kế bảng dang_ky_lop_hoc

Tên cột	Kiểu dữ liệu	Ràng buộc	Mô tả
id	UUID	PRIMARY KEY	ID duy nhất
lop_hoc_id	UUID	NOT NULL, FK	Foreign key đến bảng lop_hoc
sinh_vien_id	UUID	NOT NULL, FK	Foreign key đến bảng sinh_vien
ngay_dang_ky	TIMESTAMP	NOT NULL	Ngày đăng ký lớp
trang_thai	VARCHAR(20)	NOT NULL	DA_DANG_KY, DA_HOAN_THANH, DA_BO_HOC, THAT_BAI
diem	VARCHAR(10)	NULL	Điểm môn học (A, B, C, D, F)
ty_le_diem_danh	DOUBLE PRECISION	NULL	Tỷ lệ điểm danh (0-100)
ghi_chu	VARCHAR(500)	NULL	Ghi chú
ngay_bo_hoc	TIMESTAMP	NULL	Ngày bỏ học (nếu có)

HÌNH 3.15: Bảng thiết kế dang_ky_lop_hoc

3.5.9 Thiết kế bảng nhật_ky_bao_mat

Tên cột	Kiểu dữ liệu	Ràng buộc	Mô tả
id	UUID	PRIMARY KEY	ID duy nhất của log
nguoi_dung_id	UUID	NULL	ID người dùng (nếu có)
ten_dang_nhap	VARCHAR(50)	NULL	Tên đăng nhập
dia_chi_ip	VARCHAR(45)	NULL	Địa chỉ IP (hỗ trợ IPv6)
hanh_dong	VARCHAR(50)	NOT NULL	LOGIN_ATTEMPT, LOGIN_SUCCESS, ACCOUNT_LOCKED, v.v.
ket_qua	VARCHAR(20)	NOT NULL	SUCCESS, FAILURE, WARNING, INFO
thong_bao	VARCHAR(500)	NULL	Thông báo chi tiết
trinh_duyet	VARCHAR(255)	NULL	User Agent string
ngay_tao	TIMESTAMP	NOT NULL	Thời gian ghi log

HÌNH 3.16: Bảng thiết kế nhật_ky_bao_mat

3.6. Thiết kế API

3.6.1 API Authentication

Register API

Endpoint: POST /api/auth/register

Mô tả: Đăng ký tài khoản mới

Authentication: None (Public)

Request Body:

```
{ "username": "nguyenvana", "email": "nguyenvana@example.com",  
  "password": "Password@123", "role": "STUDENT" }
```

Response (201 Created):

```
{ "success": true, "message": "User registered successfully",  
  "data": { "accessToken": "eyJhbGciOiJSUzI1NiIs...",  
    "refreshToken": "550e8400-e29b-41d4-a716-446655440000",  
    "tokenType": "Bearer", "expiresIn": 3600, "user": { "id":  
    "550e8400-e29b-41d4-a716-446655440000", "username": "nguyenvana",  
    "email": "nguyenvana@example.com", "role": "STUDENT" } } }
```

Error Responses:

- 400 Bad Request: Username hoặc email đã tồn tại
- 400 Bad Request: Validation errors

Login API

Endpoint: POST /api/auth/login

Mô tả: Đăng nhập và nhận JWT tokens

Authentication: None (Public)

Request Body:

```
{ "usernameOrEmail": "nguyenvana", "password": "Password@123" }
```

Response (200 OK):

```
{ "success": true, "message": "Login successful", "data":  
{ "accessToken": "eyJhbGciOiJSUzI1NiIs... ", "refreshToken":  
"550e8400-e29b-41d4-a716-446655440000", "tokenType": "Bearer",  
"expiresIn": 3600, "user": { "id": "550e8400-e29b-41d4-a716-  
446655440000", "username": "nguyenvana", "email":  
"nguyenvana@example.com", "role": "STUDENT" } } }
```

Error Responses:

- 401 Unauthorized: Sai username/password
- 403 Forbidden: Tài khoản bị khóa
- 403 Forbidden: IP bị chặn
- 429 Too Many Requests: Vượt rate limit

Change Password API

Endpoint: PUT /api/users/me/password

Mô tả: Đổi mật khẩu

Authentication: Required (JWT)

Request Headers:

```
Authorization: Bearer eyJhbGciOiJSUzI1NiIs...
```

Request Body:

```
{ "oldPassword": "Password@123", "newPassword":  
"NewPassword@456" }
```

Response (200 OK):

```
{ "success": true, "message": "Password changed successfully" }
```

3.6.2 Student API

BẢNG 3.6: Student API

Method	Endpoint	Mô tả	Auth
GET	/api/students	Danh sách sinh viên (pagination)	JWT
GET	/api/students/{id}	Chi tiết sinh viên theo ID	JWT
GET	/api/students/code/{code}	Chi tiết theo mã sinh viên	JWT
GET	/api/students/search	Tìm kiếm theo tên (query param: name)	JWT
GET	/api/students/status/{status}	Lọc theo trạng thái	JWT
POST	/api/students	Tạo sinh viên mới	JWT (Admin)
PUT	/api/students/{id}	Cập nhật thông tin sinh viên	JWT (Admin)
DELETE	/api/students/{id}	Xóa sinh viên	JWT (Admin)
GET	/api/students/count	Đếm tổng số sinh viên	JWT
GET	/api/students/count/status/{status}	Đếm theo trạng thái	JWT
POST	/api/students/{id}/avatar	Upload ảnh đại diện	JWT

Example: Create Student

```
POST /api/students Authorization: Bearer eyJhbGciOiJSUzI1NiIs...
Request Body: { "userId": "550e8400-e29b-41d4-a716-446655440000",
"studentCode": "SV2024001", "firstName": "An", "lastName": "Nguyen
Van", "email": "nguyenvanan@example.com", "dateOfBirth": "2005-01-
15", "gender": "NAM", "phoneNumber": "0912345678", "address": "Ha
Noi, Viet Nam", "major": "Cong nghe thong tin", "academicYear":
"2023-2024", "status": "HOAT_DONG" } Response (201 Created):
{ "success": true, "message": "Student created successfully",
"data": { "id": "660e8400-e29b-41d4-a716-446655440001", "userId":
"550e8400-e29b-41d4-a716-446655440000", "studentCode":
"SV2024001", "fullName": "Nguyen Van An", "email":
"nguyenvanan@example.com", "major": "Cong nghe thong tin",
"status": "HOAT_DONG" } }
```

3.6.3 Teacher API

BẢNG 3.7: Teacher API

Method	Endpoint	Mô tả	Auth
GET	/api/teachers	Danh sách giảng viên	JWT
GET	/api/teachers/{id}	Chi tiết giảng viên theo ID	JWT
GET	/api/teachers/code/{code}	Chi tiết theo mã giảng viên	JWT
GET	/api/teachers/departement/{dept}	Lọc theo khoa	JWT
POST	/api/teachers	Tạo giảng viên mới	JWT (Admin)
PUT	/api/teachers/{id}	Cập nhật thông tin	JWT (Admin)
DELETE	/api/teachers/{id}	Xóa giảng viên	JWT (Admin)

3.6.4 Class API

BẢNG 3.8: Class API

Method	Endpoint	Mô tả	Auth
GET	/api/classes	Danh sách lớp học	JWT
GET	/api/classes/{id}	Chi tiết lớp học	JWT
GET	/api/classes/teacher/{teacherId}	Lớp của giảng viên	JWT
POST	/api/classes	Tạo lớp học mới	JWT (Admin)
PUT	/api/classes/{id}	Cập nhật lớp học	JWT (Admin)
DELETE	/api/classes/{id}	Xóa lớp học	JWT (Admin)
POST	/api/classes/{classId}/enroll	Đăng ký sinh viên vào lớp	JWT (Admin)
DELETE	/api/classes/{classId}/students/{studentId}	Bỏ lớp học	JWT (Admin)

GET	/api/classes/{classId}/enrollments	Danh sách đăng ký	JWT
GET	/api/classes/{classId}/students	Sinh viên trong lớp	JWT
PUT	/api/classes/enrollments/{id}	Cập nhật điểm, điểm danh	JWT (Teacher)
GET	/api/classes/students/{studentId}/enrollments	Lớp của sinh viên	JWT

Example: Enroll Student

```
POST /api/classes/{classId}/enroll Authorization: Bearer
eyJhbGciOiJSUzI1NiIs... Request Body: { "studentId": "660e8400-
e29b-41d4-a716-446655440001" } Response (200 OK): { "success":
true, "message": "Student enrolled successfully", "data": { "id":
"770e8400-e29b-41d4-a716-446655440002", "classId": "classIdHere",
"studentId": "660e8400-e29b-41d4-a716-446655440001",
"enrollmentDate": "2024-01-15T10:00:00Z", "status":
"DA_DANG_KY" } }
```

Example: Update Grade

```
PUT /api/classes/enrollments/{id} Authorization: Bearer
eyJhbGciOiJSUzI1NiIs... Request Body: { "grade": "A",
"attendanceRate": 95.5, "notes": "Sinh vien hoc tap tot" }
Response (200 OK): { "success": true, "message": "Enrollment
updated successfully" }
```

3.6.5 Security API

BẢNG 3.9: Security API

Method	Endpoint	Mô tả	Auth
POST	/api/users/{id}/unlock	Mở khóa tài khoản	JWT (Admin)
DELETE	/api/users/ip-blacklist/{ip}	Gỡ chặn IP	JWT (Admin)
GET	/api/security/logs	Xem security logs	JWT (Admin)
GET	/api/security/logs/user/{username}	Logs theo username	JWT (Admin)
GET	/api/security/logs/ip/{ip}	Logs theo IP	JWT (Admin)

CHƯƠNG 4. KẾT QUẢ NGHIÊN CỨU

4.1. Môi trường triển khai

4.1.1 Yêu cầu phần cứng

BẢNG 4.1: Phần cứng

Thông số	Tối thiểu	Khuyến nghị
CPU	Intel Core i5 Gen 8 hoặc tương đương	Intel Core i7 Gen 10+ hoặc AMD Ryzen 7
RAM	8 GB	16 GB
Ổ cứng	10 GB trống (SSD)	20 GB trống (NVMe SSD)
Network	100 Mbps	1 Gbps

4.1.2 Yêu cầu phần mềm

BẢNG 4.2: Phần mềm

Phần mềm	Phiên bản	Mục đích
Java Development Kit (JDK)	21 LTS	Runtime cho Spring Boot applications
Docker Desktop	24.0+	Container orchestration
Docker Compose	2.20+	Multi-container deployment
PostgreSQL	16	Relational database

Redis	7.2	In-memory cache & rate limiting
Node.js	20 LTS	Frontend build tools
Git	2.40+	Version control

4.1.3 Công nghệ sử dụng

Trong quá trình xây dựng hệ thống quản lý sinh viên theo kiến trúc Microservice, đề tài sử dụng các công nghệ phổ biến trong phát triển phần mềm backend hiện nay. Các công nghệ được lựa chọn nhằm đảm bảo hệ thống có khả năng bảo mật, dễ mở rộng và phù hợp với mô hình triển khai thực tế.

❖ Backend

Hệ thống được phát triển bằng ngôn ngữ Java 21 kết hợp với framework Spring Boot 3.4.1. Spring Boot giúp xây dựng các API backend nhanh chóng, hỗ trợ quản lý cấu hình, kết nối cơ sở dữ liệu và phát triển các dịch vụ độc lập theo kiến trúc Microservice.

Ngoài ra, hệ thống sử dụng các thành phần thuộc hệ sinh thái Spring như Spring Security để xử lý xác thực, phân quyền người dùng và bảo vệ API. Spring Data JPA được sử dụng để kết nối và thao tác với cơ sở dữ liệu thông qua mô hình ORM, giúp việc quản lý dữ liệu thuận tiện hơn.

❖ Kiến trúc Microservice

Hệ thống được triển khai theo mô hình Microservice với các thành phần chính gồm:

- Eureka Server: được sử dụng để quản lý và đăng ký các dịch vụ trong hệ thống. Eureka giúp các Microservice có thể tìm kiếm và giao tiếp với nhau.
- API Gateway: đóng vai trò là cổng trung gian tiếp nhận toàn bộ request từ phía người dùng. Gateway thực hiện điều hướng request, kiểm tra xác thực JWT, giới hạn truy cập API và ghi nhận log hệ thống.
- IAM Service: quản lý tài khoản người dùng, đăng nhập, đăng ký, xác thực và phân quyền trong hệ thống.

- HR Service: xử lý các chức năng nghiệp vụ liên quan đến quản lý sinh viên, giáo viên, lớp học và điểm học tập.

❖ Database và Cache

Hệ thống sử dụng PostgreSQL làm cơ sở dữ liệu chính để lưu trữ các thông tin như tài khoản người dùng, sinh viên, giáo viên, lớp học và dữ liệu liên quan.

Bên cạnh đó, Redis được sử dụng để hỗ trợ lưu trữ tạm thời dữ liệu và triển khai cơ chế bảo mật như API Rate Limiting, giới hạn số lượng request gửi đến hệ thống, giúp hạn chế spam API và tăng hiệu năng xử lý.

❖ Bảo mật hệ thống

Về bảo mật, hệ thống sử dụng JWT (JSON Web Token) với thuật toán RS256 để xác thực người dùng khi truy cập API. JWT giúp kiểm tra danh tính người dùng mà không cần lưu trạng thái đăng nhập trên server.

Mật khẩu người dùng được bảo vệ bằng thuật toán BCrypt trước khi lưu vào cơ sở dữ liệu nhằm tránh lộ thông tin khi xảy ra sự cố.

Ngoài ra, hệ thống áp dụng AES-GCM Encryption để mã hóa các dữ liệu nhạy cảm và triển khai cơ chế khóa tài khoản khi đăng nhập sai nhiều lần nhằm hạn chế các hành vi dò mật khẩu.

❖ Công cụ triển khai và kiểm thử

Hệ thống sử dụng Docker và Docker Compose để đóng gói và triển khai các thành phần như Backend Service, PostgreSQL và Redis trong môi trường độc lập.

Các công cụ hỗ trợ trong quá trình phát triển gồm:

- Swagger/OpenAPI: xây dựng tài liệu và kiểm thử API trực tiếp trên trình duyệt.
- Postman: kiểm tra hoạt động của các API.
- Apache JMeter: kiểm thử hiệu năng và khả năng chịu tải của hệ thống.
- Redis Insight: theo dõi dữ liệu Redis.
- pgAdmin: quản lý cơ sở dữ liệu PostgreSQL.
- IntelliJ IDEA: môi trường phát triển chính cho dự án.

4.2. Cài đặt và cấu hình hệ thống

4.2.1 Clone project từ repository

```
$ git clone https://github.com/your-repo/student-management-system.git $ cd student-management-system
```

4.2.2 Cấu hình Environment Variables

Tạo file .env trong thư mục backend/:

```
# Database Configuration DB_PASSWORD=app_password
POSTGRES_DB=student_management_db POSTGRES_USER=app_user # Redis
Configuration REDIS_PASSWORD= # JWT Configuration JWT_PUBLIC_KEY=--
----BEGIN PUBLIC KEY-----
MIIBIjANBgkqhkiG9w0BAQEFAAOCAQ8AMIIBCgKCAQEA... -----END PUBLIC
KEY----- JWT_PRIVATE_KEY=-----BEGIN PRIVATE KEY-----
MIIEvQIBADANBgkqhkiG9w0BAQEFAASCBCwggSjAgEA... -----END PRIVATE
KEY----- # Eureka Server EUREKA_SERVER_URL=http://eureka-
server:8761/eureka/
```

4.2.3 Build và khởi động services

Bước 1: Build project với Maven

```
$ cd backend $ mvn clean install -DskipTests
```

Kết quả mong đợi: BUILD SUCCESS cho tất cả modules

Bước 2: Khởi động Docker containers

```
$ docker-compose up -d Creating network "backend_backend-
network" ... done Creating postgres ... done Creating redis ...
done Creating eureka-server ... done Creating api-gateway ... done
Creating iam-service ... done Creating hr-service ... done
```

Bước 3: Kiểm tra trạng thái services

```
$ docker-compose ps NAME STATUS PORTS postgres Up 2 minutes
0.0.0:5432->5432/tcp redis Up 2 minutes 0.0.0:6379->6379/tcp
eureka-server Up 1 minute 0.0.0:8761->8761/tcp api-gateway Up 1
minute 0.0.0:8080->8080/tcp iam-service Up 1 minute
0.0.0:8081->8081/tcp hr-service Up 1 minute 0.0.0:8082->8082/tcp
```

4.2.4 Verify deployment

BẢNG 4.3: Bảng các server đã chạy

Service	URL	Health Check
Eureka Server	http://localhost:8761	http://localhost:8761/actuator/health
API Gateway	http://localhost:8080	http://localhost:8080/actuator/health
IAM Service	http://localhost:8081	http://localhost:8081/actuator/health
HR Service	http://localhost:8082	http://localhost:8082/actuator/health

4.3. Demo chức năng hệ thống

4.3.1 Demo Authentication - Đăng ký và Đăng nhập

Test Case 1: Đăng ký tài khoản mới

Endpoint: POST http://localhost:8080/api/auth/register

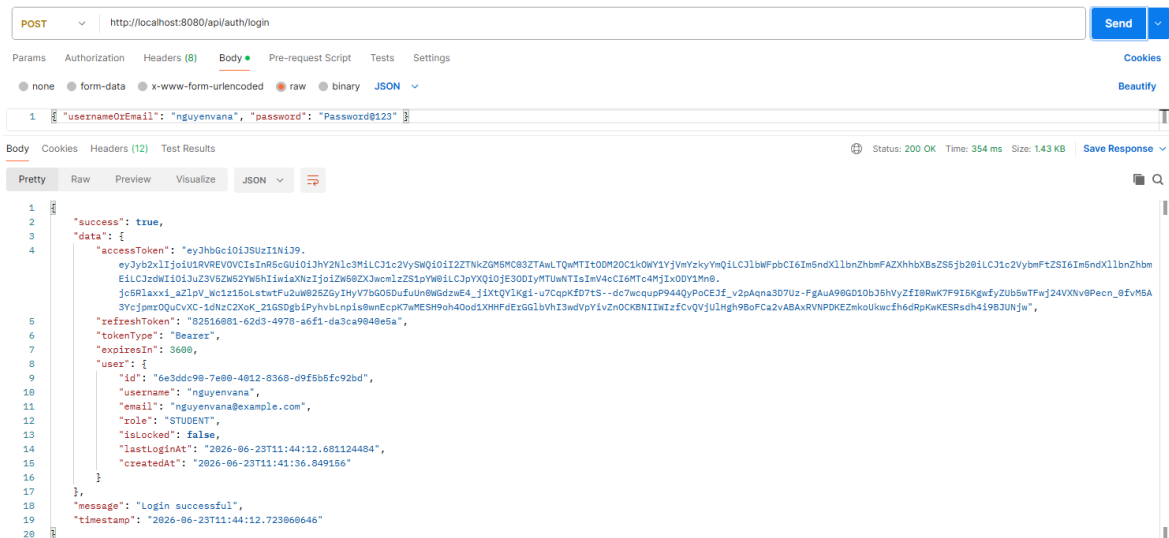
Request Body:

```
{ "username": "nguyenvana", "email":  
"nguyenvana@example.com", "password": "Password@123", "role":  
"STUDENT" }
```

Response (201 Created):

```
{ "success": true, "message": "User registered successfully",  
"data": { "accessToken":  
"eyJhbGciOiJSUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VySWQiOiI1NTBl...  
", "refreshToken": "550e8400-e29b-41d4-a716-446655440000",  
"tokenType": "Bearer", "expiresIn": 3600, "user": { "id":  
"550e8400-e29b-41d4-a716-446655440000", "username":  
"nguyenvana", "email": "nguyenvana@example.com", "role":  
"STUDENT", "enabled": true, "locked": false } } }
```

Screenshot: Đăng ký thành công



HÌNH 4.2: Postman - Đăng nhập thành công với JWT token

Test Case 3: Đăng nhập sai mật khẩu

Request Body:

```
{ "usernameOrEmail": "nguyenvana", "password":  
"WrongPassword123" }
```

Response (401 Unauthorized):

```
{ "success": false, "message": "Tên đăng nhập hoặc mật khẩu  
không đúng (1/20). Còn 19 lần nữa sẽ bị khóa 30 giây.",  
"timestamp": "2024-01-15T10:30:00Z" }
```

Lỗi xác thực: Hệ thống phát hiện mật khẩu sai và cảnh báo số lần còn lại.

4.3.2 Demo Student Management - Quản lý sinh viên

Test Case 4: Tạo sinh viên mới

Prerequisite: Đã có JWT token từ đăng nhập Admin

Endpoint: POST http://localhost:8080/api/students

Request Headers:

```
Authorization: Bearer eyJhbGciOiJSUzI1NiIsInR5cCI6IkpXVCJ9...  
Content-Type: application/json
```

Request Body:

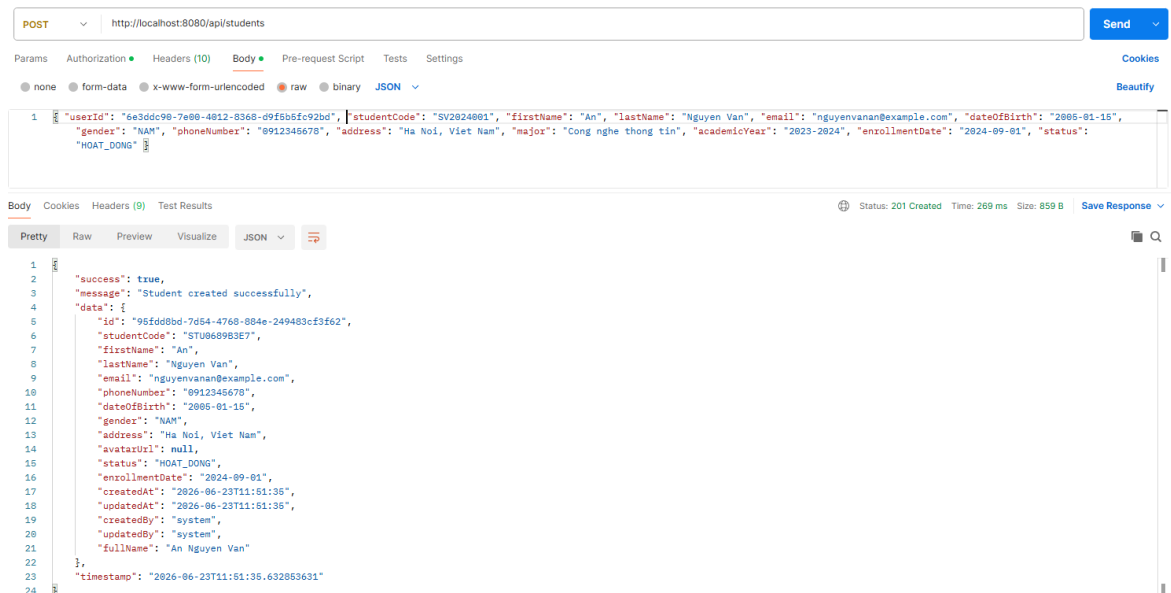
```
{ "userId": "550e8400-e29b-41d4-a716-446655440000",  
"studentCode": "SV2024001", "firstName": "An", "lastName":  
"Nguyen Van", "email": "nguyenvanan@example.com",  
"dateOfBirth": "2005-01-15", "gender": "NAM", "phoneNumber":  
"0912345678", "address": "Ha Noi, Viet Nam", "major": "Cong
```

```
nghe thông tin", "academicYear": "2023-2024",  
"enrollmentDate": "2024-09-01", "status": "HOAT_DONG" }
```

Response (201 Created):

```
{ "success": true, "message": "Student created successfully",  
"data": { "id": "660e8400-e29b-41d4-a716-446655440002",  
"userId": "550e8400-e29b-41d4-a716-446655440000",  
"studentCode": "SV2024001", "fullName": "Nguyen Van An",  
"email": "nguyenvanan@example.com", "dateOfBirth": "2005-01-  
15", "gender": "NAM", "phoneNumber": "0912345678", "address":  
"Ha Noi, Viet Nam", "major": "Cong nghe thông tin",  
"academicYear": "2023-2024", "status": "HOAT_DONG",  
"createdAt": "2024-01-15T10:45:00Z" } }
```

Screenshot: Tạo sinh viên mới thành công



HÌNH 4.3: Postman - POST Student API với JWT Authorization

Hiện thị: Headers với Bearer token, Request body, Response 201 Created

Test Case 5: Lấy danh sách sinh viên (Pagination)

Endpoint: GET

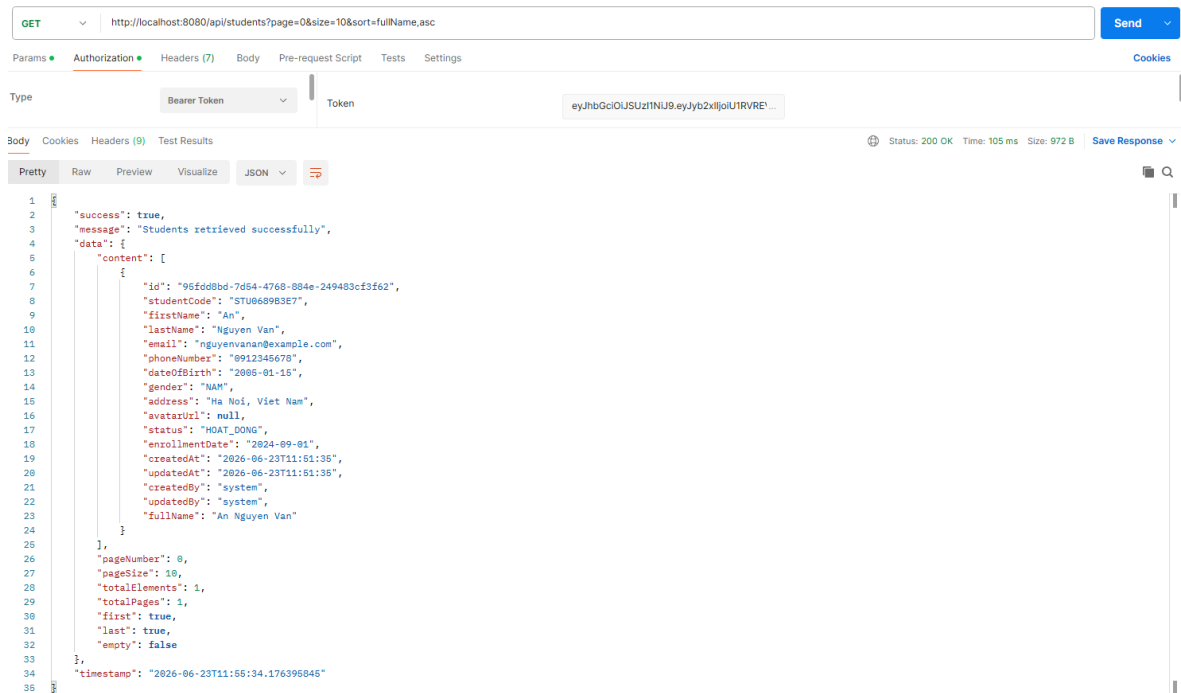
```
http://localhost:8080/api/students?page=0&size=10&sort=fullName,asc
```

Response (200 OK):

```
{ "success": true, "data": { "content": [ { "id": "660e8400-  
e29b-41d4-a716-446655440002", "studentCode": "SV2024001",  
"fullName": "Nguyen Van An", "email":  
"nguyenvanan@example.com", "major": "Cong nghe thông tin",  
"status": "HOAT_DONG" }, { "id": "670e8400-e29b-41d4-a716-  
446655440003", "studentCode": "SV2024002", "fullName": "Tran
```

```
Thi Binh", "email": "tranthiibinhexample.com", "major": "Quan  
tri kinh doanh", "status": "HOAT_DONG" } ], "pageable":  
{ "pageNumber": 0, "pageSize": 10, "sort": { "sorted": true,  
"unsorted": false } }, "totalElements": 25, "totalPages": 3,  
"last": false, "first": true } }
```

Screenshot: Danh sách sinh viên với pagination



HÌNH 4.4: Postman - GET Students với pagination params

Hiện thị: Query parameters (page, size, sort), Response với paginated data

Test Case 6: Tìm kiếm sinh viên theo tên

Endpoint:

GET `http://localhost:8080/api/students/search?name=nguyen`

Response (200 OK):

```
{ "success": true, "data": [ { "id": "660e8400-e29b-41d4-a716-446655440002", "studentCode": "SV2024001", "fullName":  
"Nguyen Van An", "email": "nguyenvanan@example.com" },  
{ "id": "680e8400-e29b-41d4-a716-446655440004",  
"studentCode": "SV2024003", "fullName": "Nguyen Thi Cam",  
"email": "nguyenthicam@example.com" } ] }
```

4.3.3 Demo Class Enrollment - Quản lý đăng ký lớp

Test Case 7: Tạo lớp học mới

Endpoint:

POST `http://localhost:8080/api/classes`

Request Body:

```
{ "classCode": "CNTT2024-01", "className": "Lap trinh Java nang cao", "teacherId": "bd4a30ce-e62d-4e44-9785-1d9350b1263c", "academicYear": "2023-2024", "semester": 1, "subject": "Lap trinh Java", "room": "A301", "schedule": "Thu 2, 7h-9h", "maxStudents": 40, "startDate": "2024-01-15", "endDate": "2024-05-15", "status": "HOAT_DONG" }
```

Response (201 Created):

```
{ "success": true, "message": "Class created successfully", "data": { "id": "bd4a30ce-e62d-4e44-9785-1d9350b1263c", "classCode": "CNTT2024-01", "className": "Lap trinh Java nang cao", "teacherId": "770e8400-e29b-41d4-a716-446655440005", "teacherName": "Nguyen Van Hieu", "maxStudents": 40, "currentStudents": 0, "status": "HOAT_DONG" } }
```

Test Case 8: Đăng ký sinh viên vào lớp

Endpoint:

```
POST http://localhost:8080/api/classes/780e8400-e29b-41d4-a716-446655440006/enroll
```

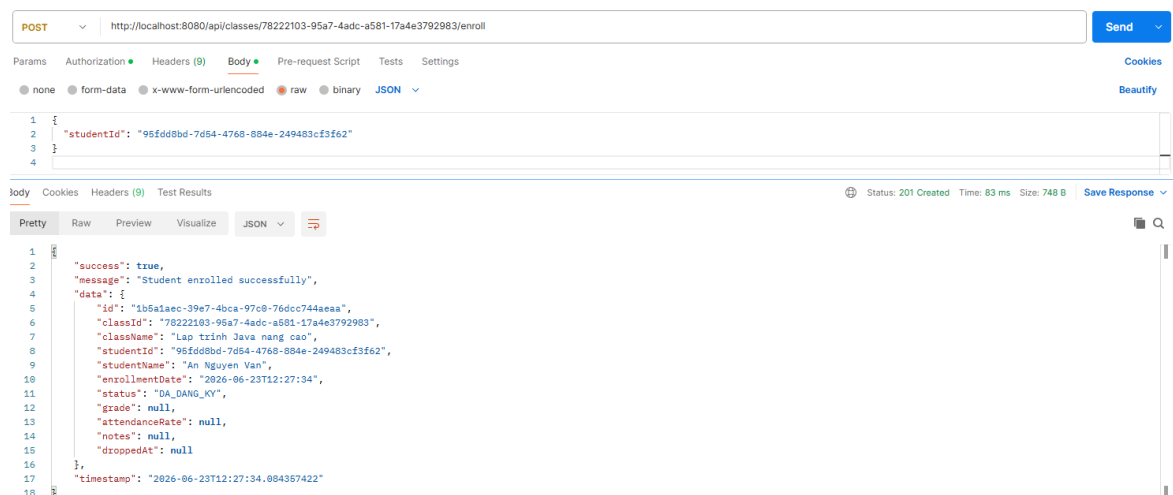
Request Body:

```
{ "studentId": "660e8400-e29b-41d4-a716-446655440002" }
```

Response (200 OK):

```
{ "success": true, "message": "Student enrolled successfully", "data": { "id": "790e8400-e29b-41d4-a716-446655440007", "classId": "780e8400-e29b-41d4-a716-446655440006", "studentId": "660e8400-e29b-41d4-a716-446655440002", "enrollmentDate": "2024-01-15T11:00:00Z", "status": "DA_DANG_KY", "grade": null, "attendanceRate": null } }
```

Screenshot: Đăng ký lớp thành công



HÌNH 4.5: Postman - POST Enroll Student API

Hiện thị: Enrollment response với status DA_DANG_KY

Test Case 9: Cập nhật điểm và điểm danh

Endpoint:

```
PUT http://localhost:8080/api/classes/enrollments/790e8400-e29b-41d4-a716-446655440007
```

Request Body:

```
{ "grade": "A", "attendanceRate": 95.5, "notes": "Sinh vien hoc tap tot, tích cuc tham gia" }
```

Response (200 OK):

```
{ "success": true, "message": "Enrollment updated successfully", "data": { "id": "790e8400-e29b-41d4-a716-446655440007", "grade": "A", "attendanceRate": 95.5, "notes": "Sinh vien hoc tap tot, tích cuc tham gia", "status": "DA_HOAN_THANH" } }
```

Chức năng Teacher: Giảng viên có thể cập nhật điểm và điểm danh cho sinh viên trong lớp mình dạy.

Test Case 10: Xem danh sách sinh viên trong lớp

Endpoint:

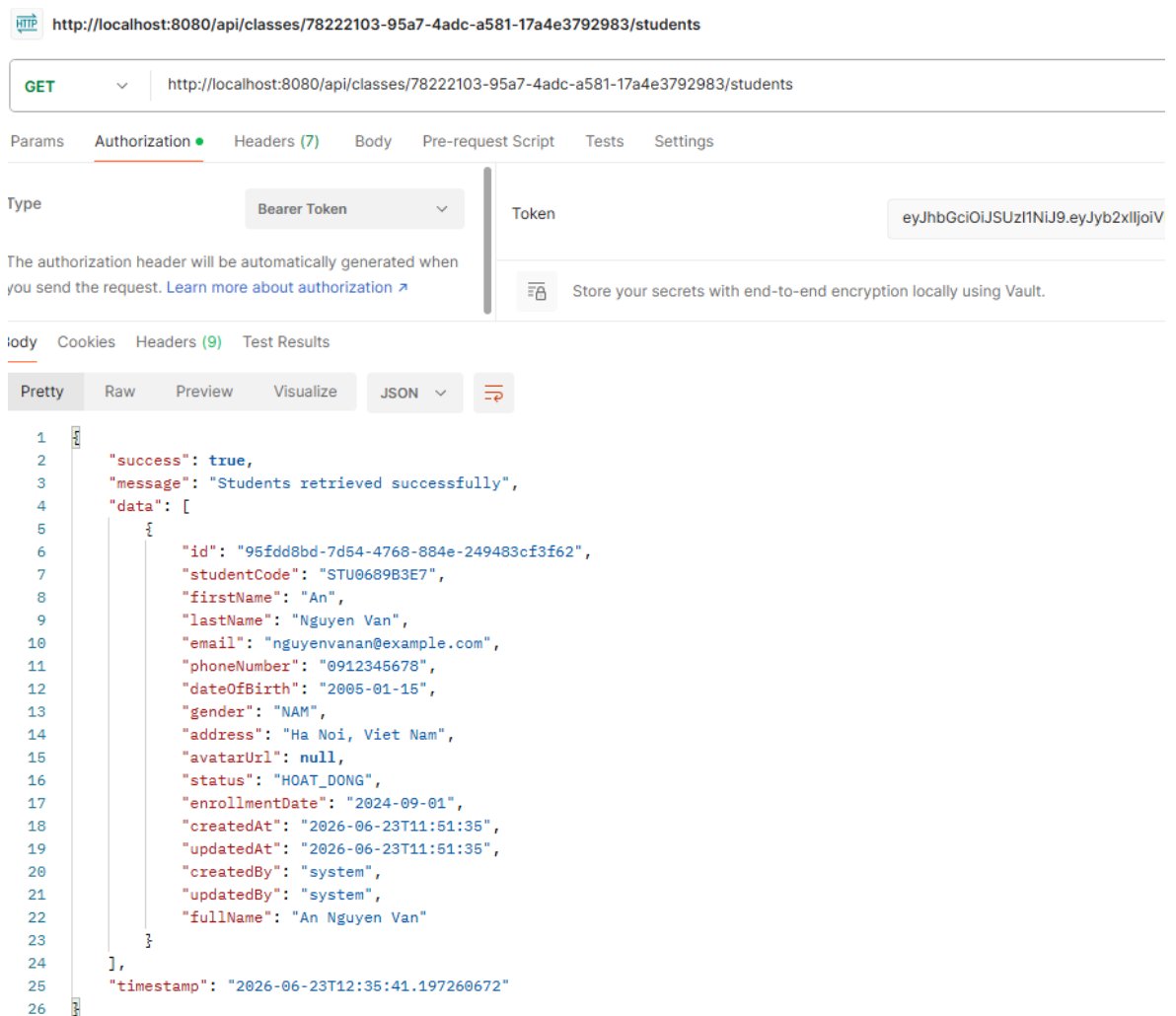
```
GET http://localhost:8080/api/classes/78222103-95a7-4adc-a581-17a4e3792983/students
```

Response (200 OK):

```
{"success": true, "message": "Students retrieved successfully", "data": [{"id": "95fdd8bd-7d54-4768-884e-249483cf3f62", "studentCode": "STU0689B3E7", "firstName":
```

```
"An", "lastName": "Nguyen Van", "email":
"nguyenvanan@example.com", "phoneNumber":
"0912345678", "dateOfBirth": "2005-01-15", "gender":
"NAM", "address": "Ha Noi, Viet Nam", "avatarUrl":
null, "status": "HOAT_DONG", "enrollmentDate": "2024-09-
01", "createdAt": "2026-06-23T11:51:35", "updatedAt": "2026-06-
23T11:51:35", "createdBy": "system", "updatedBy":
"system", "fullName": "An Nguyen Van"}], "timestamp": "2026-06-
23T12:35:41.197260672"}
```

Screenshot: Danh sách sinh viên trong lớp



HÌNH 4.6: Postman - GET Students in Class

Hiện thị: List students với enrollment info, grade, attendance rate

4.4. Demo giao diện Website quản lý sinh viên

4.4.1 Giao diện đăng nhập/ đăng xuất

The screenshot shows a login interface for the 'Hệ thống Quản lý Sinh viên' (Student Management System) of 'ĐẠI HỌC TRÀ VINH' (Trà Vinh University). The page has a dark blue background. At the top center is a logo of a graduation cap inside a blue circle, with the text 'ĐẠI HỌC TRÀ VINH' and 'KHOA KỸ THUẬT VÀ CÔNG NGHỆ (CET)' below it. The main form is white with a light blue border. It contains the title 'Hệ thống Quản lý Sinh viên' and the subtitle 'Đăng nhập cổng thông tin tích hợp'. There are two input fields: 'TÊN ĐĂNG NHẬP HOẶC EMAIL' (Username or Email) and 'MẬT KHẨU' (Password). Below the password field is a checkbox for 'Ghi nhớ' (Remember me) and a link 'Quên mật khẩu?' (Forgot password?). A blue button labeled 'ĐĂNG NHẬP' (Login) is at the bottom of the form. Below the button is a link 'Chưa có tài khoản? Đăng ký ngay' (Don't have an account? Register now). At the very bottom of the form is a small text: '© 2020 Trường Kỹ thuật và Công nghệ - Đại học Trà Vinh'.

HÌNH 4.7: Giao diện đăng nhập

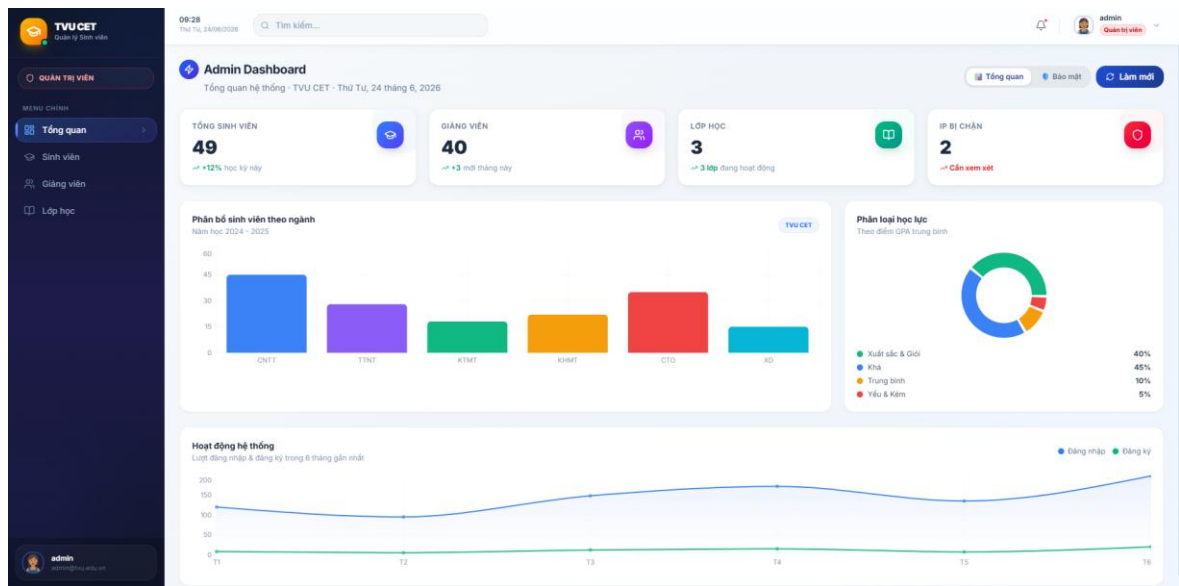
Lưu ý: Người dùng sẽ đăng nhập bằng Gmail của sinh viên của mình.

The screenshot shows a registration interface for the 'Hệ thống Quản lý Sinh viên' (Student Management System) of 'ĐẠI HỌC TRÀ VINH' (Trà Vinh University). The page has a dark blue background. At the top center is a logo of a graduation cap inside a blue circle, with the text 'ĐẠI HỌC TRÀ VINH' and 'KHOA KỸ THUẬT VÀ CÔNG NGHỆ (CET)' below it. The main form is white with a light blue border. It contains the title 'Tạo tài khoản mới' (Create new account) and the subtitle 'Đăng ký tham gia hệ thống quản lý học tập'. There are four input fields: 'TÊN ĐĂNG NHẬP' (Username), 'ĐỊA CHỈ EMAIL' (Email address), 'MẬT KHẨU' (Password), and 'XÁC NHẬN MẬT KHẨU' (Confirm password). There are also two dropdown menus: 'VỊ TRÍ' (Position) with 'Sinh viên' (Student) selected, and 'NGÀNH HỌC TVU CET' (Trà Vinh University CET Major) with 'Công nghệ thông tin' (Information Technology) selected. A blue button labeled 'TAO TÀI KHOẢN' (Create account) is at the bottom of the form. Below the button is a link 'Đã có tài khoản? Đăng nhập ngay' (Already have an account? Login now). At the very bottom of the form is a small text: '© 2020 Trường Kỹ thuật và Công nghệ - Đại học Trà Vinh'.

HÌNH 4.8: Giao diện đăng ký

Lưu ý: Người dùng sẽ đăng ký tài khoản và sau đó được Admin duyệt qua.

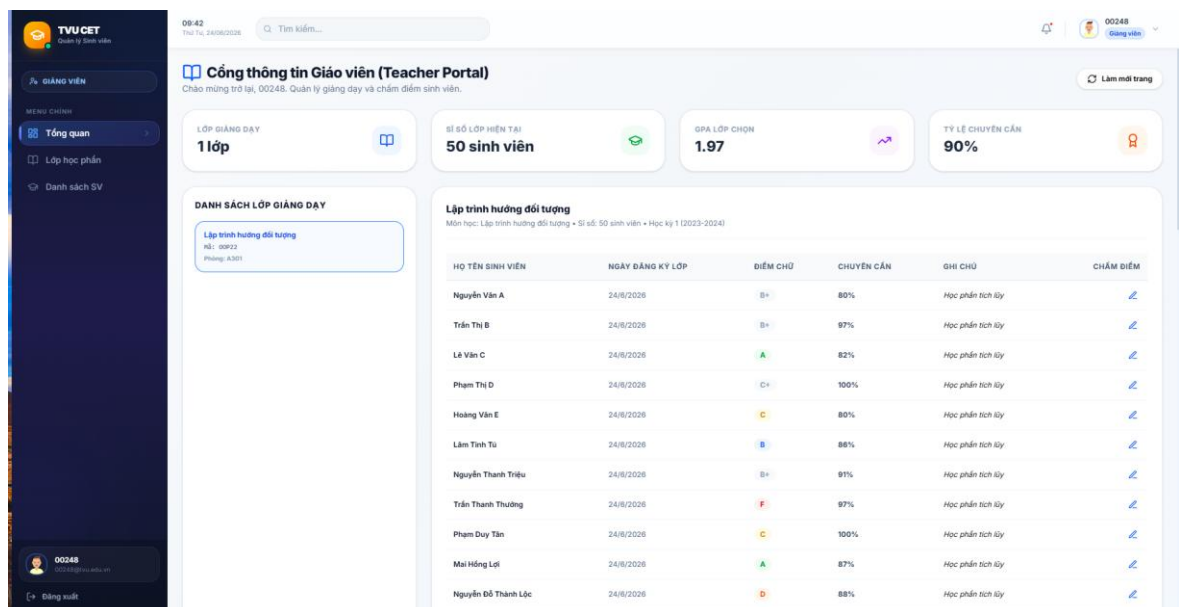
4.4.2 Giao diện Admin



HÌNH 4.9: Giao diện Admin

Lưu ý: Admin sẽ có quyền xem mọi thông tin gồm lớp sinh viên và giảng viên, đặt biệt xem các tài khoản bị khóa và các địa chỉ IP máy bị khóa. Quyền tạo và mở các lớp học phân sinh viên vào đúng lớp học sinh viên đã đăng ký.

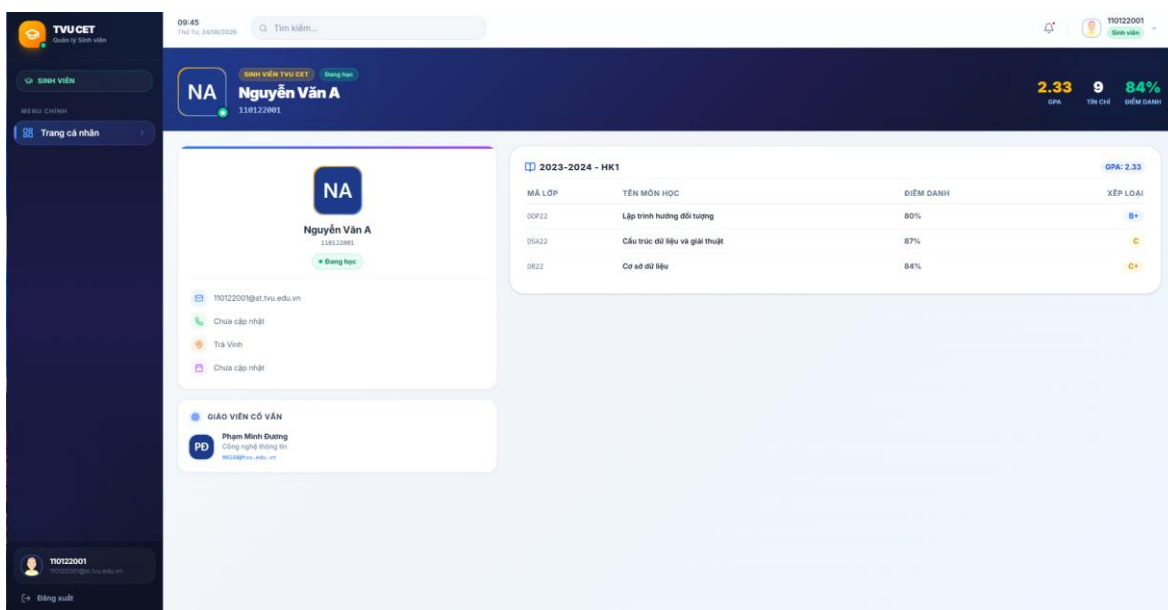
4.4.3 Giao diện Giảng viên



HÌNH 4.10: Giao diện Giảng viên

Lưu ý: Giảng viên sẽ thấy được lớp mình làm cố vấn học tập thấy được sinh viên lớp mình giảng dạy và cho số điểm.

4.4.4 Giao diện Sinh viên



HÌNH 4.11: Giao diện Sinh viên

Lưu ý: Sinh viên sẽ thấy được cô vấn học tập của mình là thầy cô nào và thấy được điểm của mình qua từng môn học.

4.5. Kiểm thử bảo mật

4.5.1 Test Progressive Account Locking

Hệ thống áp dụng cơ chế khóa tài khoản tăng dần (Progressive Locking) để chống brute-force attack. Mức độ khóa tăng dần theo số lần đăng nhập sai:

BẢNG 4.1: Test Progressive Account Locking

Số lần sai	Hành động	Thời gian khóa	Thông báo
1-4	Cảnh báo	-	"...không đúng (X/20). Còn Y lần nữa sẽ bị khóa 30 giây"
5	Khóa Level 1	30 giây	"Tài khoản đã bị khóa 30 giây"
6-9	Cảnh báo	-	"Còn Y lần nữa sẽ bị khóa 1 phút"
10	Khóa Level 2	1 phút	"Tài khoản đã bị khóa 1 phút"
11-14	Cảnh báo	-	"Còn Y lần nữa sẽ bị khóa 1 giờ"
15	Khóa Level 3	1 giờ	"Tài khoản đã bị khóa 1 giờ"
16-19	Cảnh báo nghiêm trọng	-	"Còn Y lần nữa IP sẽ bị chặn vĩnh viễn"
20	Khóa vĩnh viễn + Chặn IP	Permanent	"Tài khoản và IP bị chặn vĩnh viễn"

Test Scenario 1: Khóa 30 giây sau 5 lần sai

Bước 1: Đăng ký tài khoản test

```
POST http://localhost:8080/api/auth/register { "username":  
"testlock1", "email": "testlock1@example.com", "password":  
"Test@123456", "role": "STUDENT" }
```

Bước 2: Đăng nhập sai 5 lần liên tiếp

```
POST http://localhost:8080/api/auth/login  
{ "usernameOrEmail": "testlock1", "password":  
"WrongPassword123" }
```

Screenshot: Account locked sau 5 lần sai

Hiển thị: Status 423, message "Tài khoản đã bị khóa. Vui lòng thử lại sau 30 giây"

Bước 3: Thử đăng nhập với mật khẩu ĐÚNG (vẫn bị chặn)

```
POST http://localhost:8080/api/auth/login  
{ "usernameOrEmail": "testlock1", "password": "Test@123456" }  
Response (423 Locked): { "success": false, "message": "Tài  
khoản đã bị khóa. Vui lòng thử lại sau 25 giây.",  
"timestamp": "2024-01-15T11:15:00Z" }
```

Bước 4: Đợi 30 giây và thử lại → Đăng nhập thành công

Kết quả: Sau 30 giây, tài khoản tự động mở khóa. Đăng nhập thành công với mật khẩu đúng. Counter được reset về 0.

Test Scenario 2: Khóa vĩnh viễn + Chặn IP sau 20 lần PERMANENT LOCK

Quy trình test:

1. Đăng ký tài khoản test mới: testlock2
2. Đăng nhập sai 5 lần → Khóa 30 giây → Đợi 30 giây
3. Đăng nhập sai thêm 5 lần (tổng 10) → Khóa 1 phút → Đợi 1 phút
4. Đăng nhập sai thêm 5 lần (tổng 15) → Khóa 1 giờ → Đợi 1 giờ
5. Đăng nhập sai thêm 5 lần (tổng 20) → Khóa vĩnh viễn + Chặn IP

Response lần thứ 20:

```
{ "success": false, "message": "Tài khoản đã bị khóa vĩnh  
viễn do vi phạm bảo mật nghiêm trọng. Vui lòng liên hệ  
Admin.", "timestamp": "2024-01-15T12:00:00Z" }
```

Permanent Lock:

- Tài khoản bị khóa vĩnh viễn (khoa_den_luc = NULL)

- IP bị thêm vào blacklist trong Redis
- Không thể đăng nhập bất kỳ tài khoản nào từ IP này
- Chỉ Admin mới có thể mở khóa

Screenshot: Permanent lock và IP blacklist

Kiểm tra trong Database:

```
$ docker exec -it postgres psql -U app_user -d
student_management_db student_management_db=# SELECT
ten_dang_nhap, so_lan_dang_nhap_that_bai, bi_khoa,
khoa_den_luc FROM nguoi_dung WHERE ten_dang_nhap =
'testlock2'; ten_dang_nhap | so_lan_dang_nhap_that_bai |
bi_khoa | khoa_den_luc -----+-----
-----+-----+----- testlock2 | 20 | t | (1 row)
```

Kiểm tra trong Redis:

```
$ docker exec -it redis redis-cli 127.0.1:6379> GET
ip_blacklist:127.0.1 "BLOCKED_PERMANENTLY_AT_2024-01-
15T12:00:00Z" 127.0.1:6379> GET failed_login:user:testlock2
"20" 127.0.1:6379> GET failed_login:ip:127.0.1 "20"
```

4.5.2 Test Rate Limiting

Test Scenario 3: Vượt quá 100 requests/giây 429 TOO MANY REQUESTS

Cấu hình: Rate limit = 100 requests/giây/IP

Tool: Apache Bench hoặc Postman Runner

Test với Apache Bench:

```
$ ab -n 150 -c 10 -p login.json -T application/json \
http://localhost:8080/api/auth/login Concurrency Level: 10
Time taken for tests: 0.800 seconds Complete requests: 150
Failed requests: 50 (Rate limited) Requests per second:
187.50 [#/sec]
```

Kết quả:

- 100 requests đầu tiên: 200 OK
- 50 requests tiếp theo: 429 Too Many Requests

Response khi vượt rate limit:

```
{ "success": false, "message": "Rate limit exceeded. Please try again later.", "timestamp": "2024-01-15T12:15:00Z" }
```

Screenshot: Rate limiting in action

Lưu ý: Rate limiting chỉ hoạt động khi gọi qua API Gateway (port 8080). Nếu gọi trực tiếp vào IAM service (port 8081) sẽ bypass rate limiting.

4.5.3 Test JWT Authentication

Test Scenario 4: Truy cập API không có token

Request:

```
GET http://localhost:8080/api/students (không có Authorization header)
```

Response (401 Unauthorized):

```
{ "success": false, "message": "Missing or invalid Authorization header", "timestamp": "2024-01-15T12:20:00Z" }
```

Test Scenario 5: Token hết hạn

Cấu hình: Access token TTL = 3600 giây (1 giờ)

Test: Đợi token hết hạn và gọi API

Response (401 Unauthorized):

```
{ "success": false, "message": "Invalid or expired token", "timestamp": "2024-01-15T13:20:00Z" }
```

Giải pháp: Sử dụng Refresh Token để lấy access token mới:

```
POST /api/auth/refresh với {"refreshToken": "..."} 
```

Test Scenario 6: Token bị giả mạo

Test: Thay đổi 1 ký tự trong JWT token

Original token:

```
eyJhbGciOiJSUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VySWQiOiI1NTBl...
```

Modified token:

```
eyJhbGciOiJSUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VySWQiOiI1NTBm...  
(thay đổi ký tự cuối)
```

Response (401 Unauthorized):

```
{ "success": false, "message": "Invalid token signature", "timestamp": "2024-01-15T13:25:00Z" }
```

Bảo mật: RS256 signature verification phát hiện token bị giả mạo ngay lập tức.

CHƯƠNG 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1. Kết luận

Trong quá trình thực hiện đề tài "Thiết kế và triển khai hệ thống quản lý sinh viên theo kiến trúc Microservice kết hợp cơ chế xác thực và giới hạn truy cập API", em đã nghiên cứu và áp dụng các công nghệ hiện đại trong việc xây dựng hệ thống backend theo mô hình doanh nghiệp.

Đề tài đã xây dựng thành công hệ thống quản lý sinh viên dựa trên kiến trúc Microservice sử dụng Spring Boot 3.4.1, Java 21, PostgreSQL và Redis. Hệ thống được chia thành các dịch vụ độc lập bao gồm Eureka Server, API Gateway, IAM Service và HR Service, giúp tăng khả năng mở rộng, bảo trì và triển khai trong thực tế.

Về mặt chức năng, hệ thống đáp ứng được các yêu cầu cơ bản của bài toán quản lý sinh viên như quản lý thông tin sinh viên, giáo viên, lớp học, điểm học tập, điểm rèn luyện và cố vấn học tập. Hệ thống hỗ trợ ba nhóm người dùng chính gồm Admin, Teacher và Student với cơ chế phân quyền rõ ràng, đảm bảo mỗi đối tượng chỉ được sử dụng các chức năng phù hợp với vai trò của mình.

Về mặt bảo mật, em đã triển khai các cơ chế bảo vệ hệ thống như xác thực JWT RS256, mã hóa mật khẩu bằng BCrypt, mã hóa dữ liệu nhạy cảm bằng AES-GCM, giới hạn truy cập bằng Redis Rate Limiting, khóa tài khoản khi đăng nhập sai nhiều lần và chặn địa chỉ IP có dấu hiệu tấn công.

Thông qua quá trình thực hiện đề tài, em đã có cơ hội tiếp cận và tìm hiểu sâu hơn về kiến trúc Microservice, Spring Security, Spring Cloud Gateway, Redis, PostgreSQL, Docker cũng như các giải pháp bảo mật thường được áp dụng trong các hệ thống doanh nghiệp hiện nay.

Nhìn chung, đề tài đã hoàn thành các mục tiêu đề ra ban đầu, xây dựng được một hệ thống quản lý sinh viên có tính thực tiễn, đảm bảo yêu cầu về bảo mật, khả năng mở rộng và đáp ứng định hướng phát triển của các hệ thống backend hiện đại.

5.2. Hướng phát triển

Mặc dù hệ thống đã hoàn thành các chức năng chính và đáp ứng được các mục tiêu đề ra ban đầu, tuy nhiên trong quá trình thực hiện đề tài, em nhận thấy vẫn

còn nhiều nội dung có thể tiếp tục nghiên cứu và phát triển để nâng cao chất lượng cũng như khả năng ứng dụng thực tế của hệ thống.

Trước hết, hệ thống có thể được mở rộng thêm các chức năng phục vụ công tác quản lý đào tạo như quản lý môn học, quản lý thời khóa biểu, quản lý lịch thi, quản lý học phí và quản lý quá trình học tập của sinh viên theo từng học kỳ. Việc bổ sung các chức năng này sẽ giúp hệ thống trở nên hoàn thiện hơn và đáp ứng tốt hơn nhu cầu quản lý trong môi trường đại học thực tế.

Bên cạnh đó, em có thể phát triển thêm các Microservice chuyên biệt như Notification Service để gửi email hoặc thông báo đến người dùng, Audit Service để theo dõi lịch sử hoạt động của hệ thống và Reporting Service để hỗ trợ thống kê, báo cáo dữ liệu một cách trực quan hơn.

Về mặt bảo mật, hệ thống có thể được nâng cấp bằng cách tích hợp cơ chế xác thực đa yếu tố (Multi-Factor Authentication - MFA) nhằm tăng cường khả năng bảo vệ tài khoản người dùng. Ngoài ra, việc xây dựng KMS (Key Management Service) để quản lý và xoay vòng khóa mã hóa định kỳ cũng là một hướng phát triển phù hợp nhằm nâng cao mức độ an toàn cho dữ liệu.

Trong tương lai, hệ thống có thể được triển khai trên môi trường điện toán đám mây kết hợp với Docker và Kubernetes để hỗ trợ tự động mở rộng tài nguyên khi số lượng người dùng tăng cao. Đây là mô hình đang được nhiều doanh nghiệp áp dụng nhằm đảm bảo tính ổn định và khả năng mở rộng của hệ thống.

Ngoài ra, hệ thống cũng có thể tích hợp các công cụ giám sát như Prometheus và Grafana để theo dõi hiệu năng hoạt động của các Microservice theo thời gian thực, hỗ trợ phát hiện sớm các sự cố và tối ưu quá trình vận hành.

Về khả năng kiểm thử, trong tương lai có thể xây dựng thêm các bộ kiểm thử tự động cho API và triển khai quy trình CI/CD nhằm tự động hóa quá trình kiểm thử, xây dựng và triển khai hệ thống. Điều này giúp nâng cao chất lượng phần mềm cũng như giảm thời gian triển khai khi có các phiên bản mới.

Nhìn chung, với kiến trúc Microservice và nền tảng công nghệ đã xây dựng, hệ thống có khả năng mở rộng và phát triển thành một hệ thống quản lý đào tạo hoàn chỉnh, đáp ứng nhu cầu quản lý trong các trường đại học hoặc các tổ chức giáo dục có quy mô lớn.

TÀI LIỆU THAM KHẢO

- [1] Spring Team, *Spring Boot Reference Documentation*, VMware Inc., 2025. Truy cập tại:
<https://docs.spring.io/spring-boot/reference/>
- [2] Spring Team, *Spring Security Reference Documentation*, VMware Inc., 2025. Truy cập tại:
<https://docs.spring.io/spring-security/reference/>
- [3] Spring Team, *Spring Cloud Gateway Reference Documentation*, VMware Inc., 2025. Truy cập tại:
<https://docs.spring.io/spring-cloud-gateway/reference/>
- [4] Spring Team, *Spring Cloud Netflix Eureka Documentation*, VMware Inc., 2025. Truy cập tại:
<https://docs.spring.io/spring-cloud-netflix/reference/>
- [5] Redis Ltd., *Redis Documentation*, Redis Inc., 2025. Truy cập tại:
<https://redis.io/docs/>
- [6] Docker Inc., *Docker Documentation*, Docker Inc., 2025. Truy cập tại:
<https://docs.docker.com/>
- [7] OpenAPI Initiative, *OpenAPI Specification Documentation*, Linux Foundation, 2025. Truy cập tại:
<https://swagger.io/specification/>
- [8] Apache Software Foundation, *Apache JMeter User Manual*, Apache Software Foundation, 2025. Truy cập tại:
<https://jmeter.apache.org/usermanual/>
- [9] Viblo Asia, "Tìm hiểu về Microservices", Viblo, 2024. Truy cập tại:
<https://viblo.asia/p/tim-hieu-ve-microservices-phan-1-microservices-la-gi-63vKjVjyK2R>
- [10] Viblo Asia, "API Gateway là gì? Tại sao hệ thống Microservices cần API Gateway?", Viblo, 2024. Truy cập tại:
<https://viblo.asia/p/api-gateway-la-gi-tai-sao-mot-he-thong-microservices-lai-can-api-gateway-Do754pDX5M6>
- [11] TopDev, "JSON Web Token (JWT) là gì?", TopDev Việt Nam, 2024. Truy cập tại:
<https://topdev.vn/blog/jwt-la-gi/>
- [12] Viblo Asia, "Tìm hiểu cơ bản về Docker", Viblo, 2024. Truy cập tại:
<https://viblo.asia/p/tim-hieu-co-ban-ve-docker-6J3Zgx1glmB>

- [13] LPTech Asia, "Redis là gì? Các đặc điểm và phân loại dữ liệu trong Redis", LPTech Asia, 2024. Truy cập tại:
<https://lptech.asia/kien-thuc/redis-la-gi-cac-dac-diem-va-phan-loai-du-lieu-trong-redis>
- [14] Vietnix, "Swagger là gì? Hướng dẫn sử dụng Swagger trong phát triển API", Vietnix, 2025. Truy cập tại:
<https://vietnix.vn/swagger/>
- [15] TestMentor, "Apache JMeter là gì? Hướng dẫn sử dụng JMeter kiểm thử hiệu năng Web Server", TestMentor, 2025. Truy cập tại:
<https://testmentor.vn/apache-jmeter-la-gi-huong-dan-su-dung-jmeter-test-hieu-nang-cho-web-server/>
- [16] Mozilla Developer Network (MDN), *HTTP Documentation*, Mozilla Foundation, 2025. Truy cập tại:
<https://developer.mozilla.org/en-US/docs/Web/HTTP>
- [17] JSON Web Token, *Introduction to JSON Web Token (JWT)*, 2025. Truy cập tại:
<https://jwt.io/introduction>